

Deep Learning

Lecture 7

Sequence-to-Sequence with RNNs

Input: Sequence $x_1, \dots, x_T$

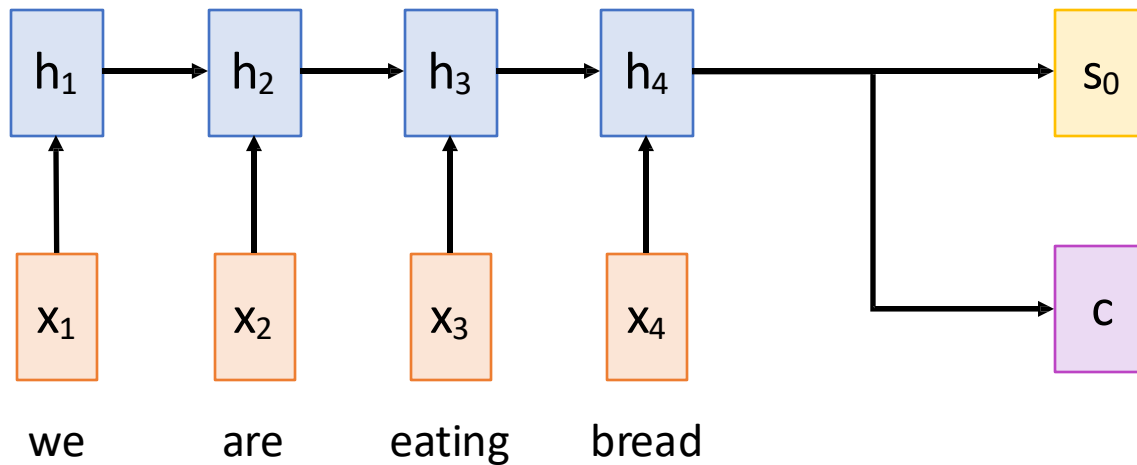
Output: Sequence $y_1, \dots, y_{T'}$

Encoder: $h_t = f_W(x_t, h_{t-1})$

From final hidden state predict:

Initial decoder state s_0

Context vector c (often $c=h_T$)



Sequence-to-Sequence with RNNs

Input: Sequence $x_1, \dots, x_T$

Output: Sequence $y_1, \dots, y_{T'}$

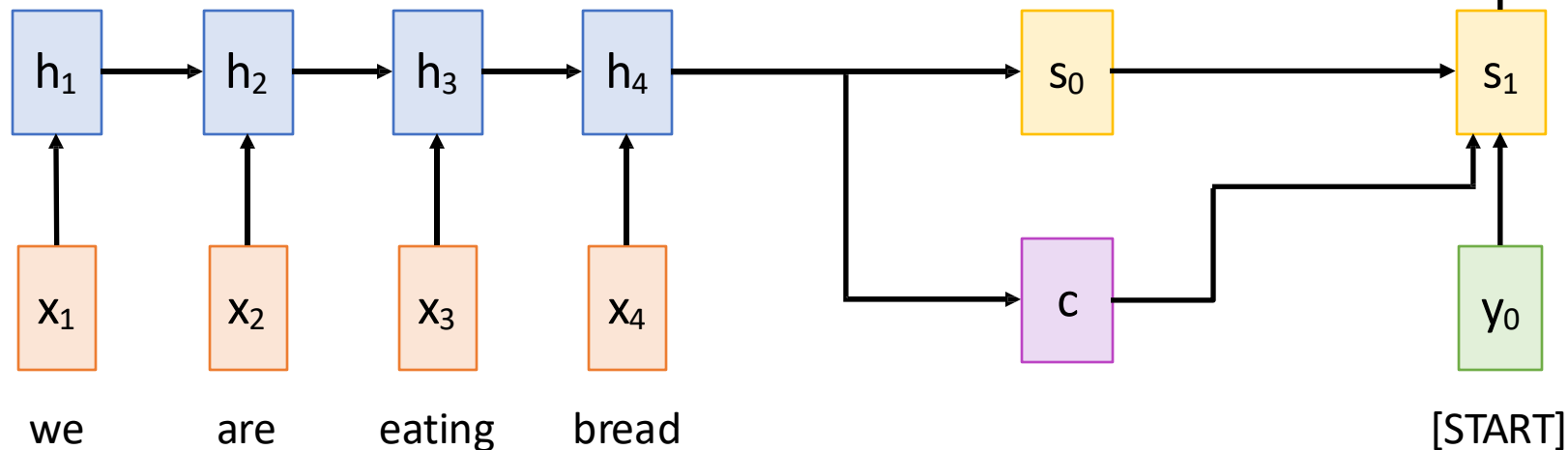
Decoder: $s_t = g_U(y_{t-1}, h_{t-1}, c)$

Encoder: $h_t = f_W(x_t, h_{t-1})$

From final hidden state predict:

Initial decoder state s_0

Context vector c (often $c=h_T$)



Sequence-to-Sequence with RNNs

Input: Sequence $x_1, \dots, x_T$

Output: Sequence $y_1, \dots, y_{T'}$

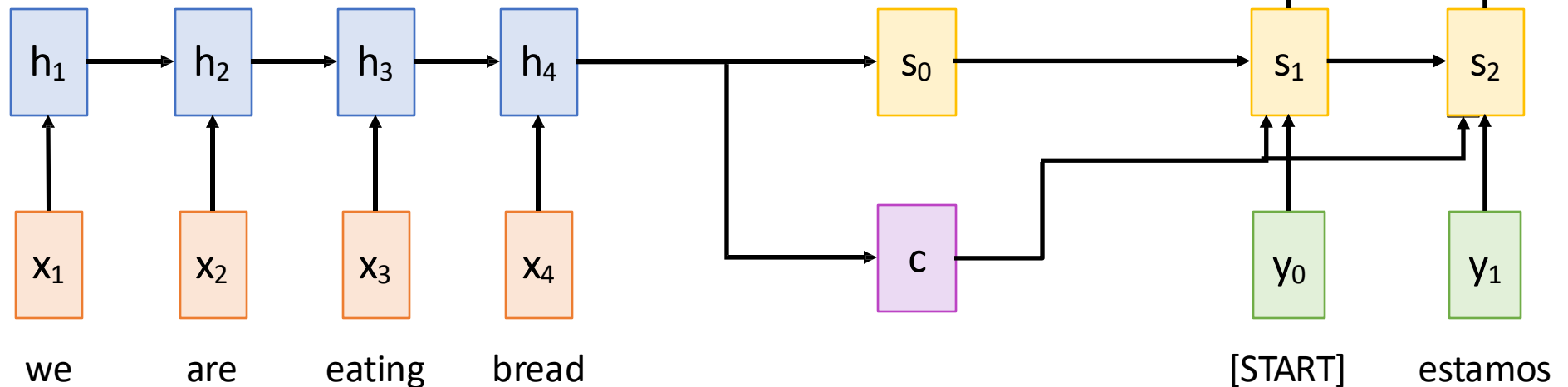
Decoder: $s_t = g_U(y_{t-1}, h_{t-1}, c)$

Encoder: $h_t = f_W(x_t, h_{t-1})$

From final hidden state predict:

Initial decoder state s_0

Context vector c (often $c=h_T$)



Sequence-to-Sequence with RNNs

Input: Sequence $x_1, \dots, x_T$

Output: Sequence $y_1, \dots, y_{T'}$

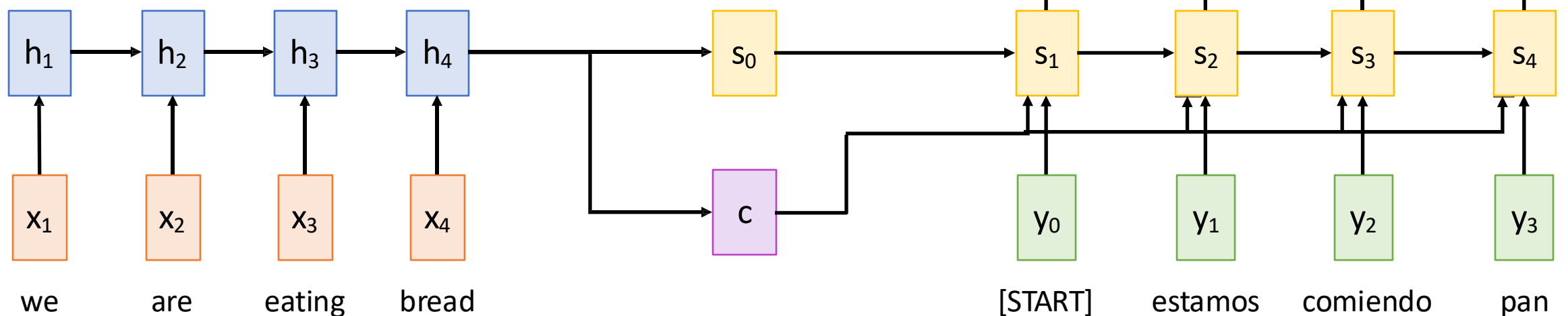
Decoder: $s_t = g_U(y_{t-1}, h_{t-1}, c)$

Encoder: $h_t = f_W(x_t, h_{t-1})$

From final hidden state predict:

Initial decoder state s_0

Context vector c (often $c=h_T$)



Sequence-to-Sequence with RNNs

$$\text{Decoder: } s_t = g_U(y_{t-1}, h_{t-1}, c)$$

Input: Sequence $x_1, \dots, x_T$

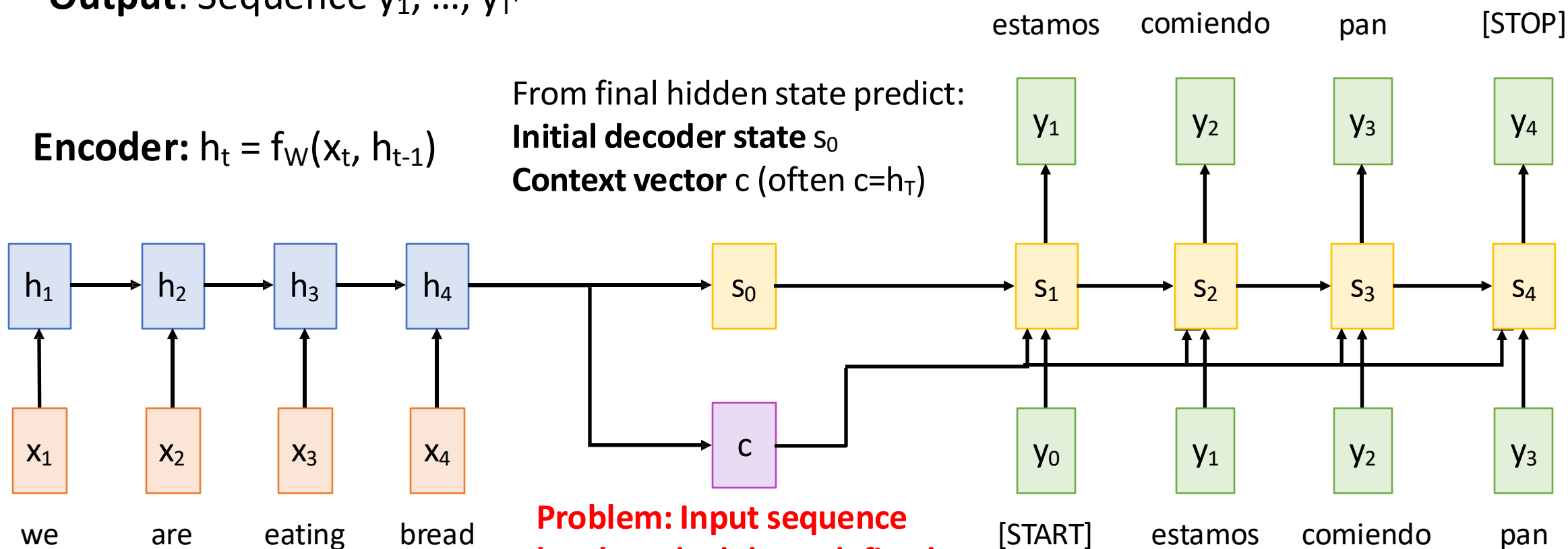
Output: Sequence $y_1, \dots, y_{T'}$

Encoder: $h_t = f_W(x_t, h_{t-1})$

From final hidden state predict:

Initial decoder state s_0

Context vector c (often $c=h_T$)



Problem: Input sequence bottlenecked through fixed-sized vector. What if $T=1000$?

Sequence-to-Sequence with RNNs

Input: Sequence $x_1, \dots, x_T$

Output: Sequence $y_1, \dots, y_{T'}$

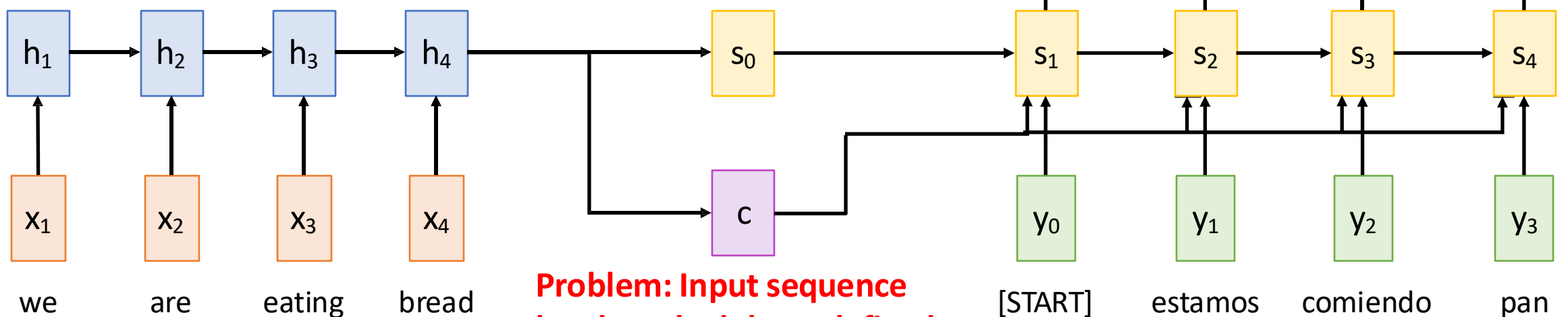
Decoder: $s_t = g_U(y_{t-1}, h_{t-1}, c)$

Encoder: $h_t = f_W(x_t, h_{t-1})$

From final hidden state predict:

Initial decoder state s_0

Context vector c (often $c=h_T$)



Problem: Input sequence bottlenecked through fixed-sized vector. What if $T=1000$?

Idea: use new context vector at each step of decoder!

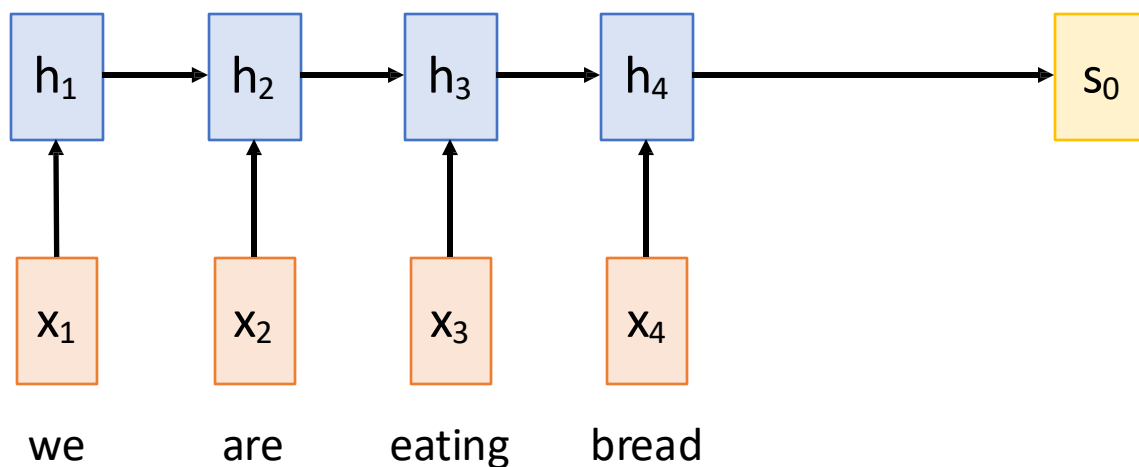
Sequence-to-Sequence with RNNs and Attention

Input: Sequence $x_1, \dots, x_T$

Output: Sequence $y_1, \dots, y_{T'}$

Encoder: $h_t = f_W(x_t, h_{t-1})$

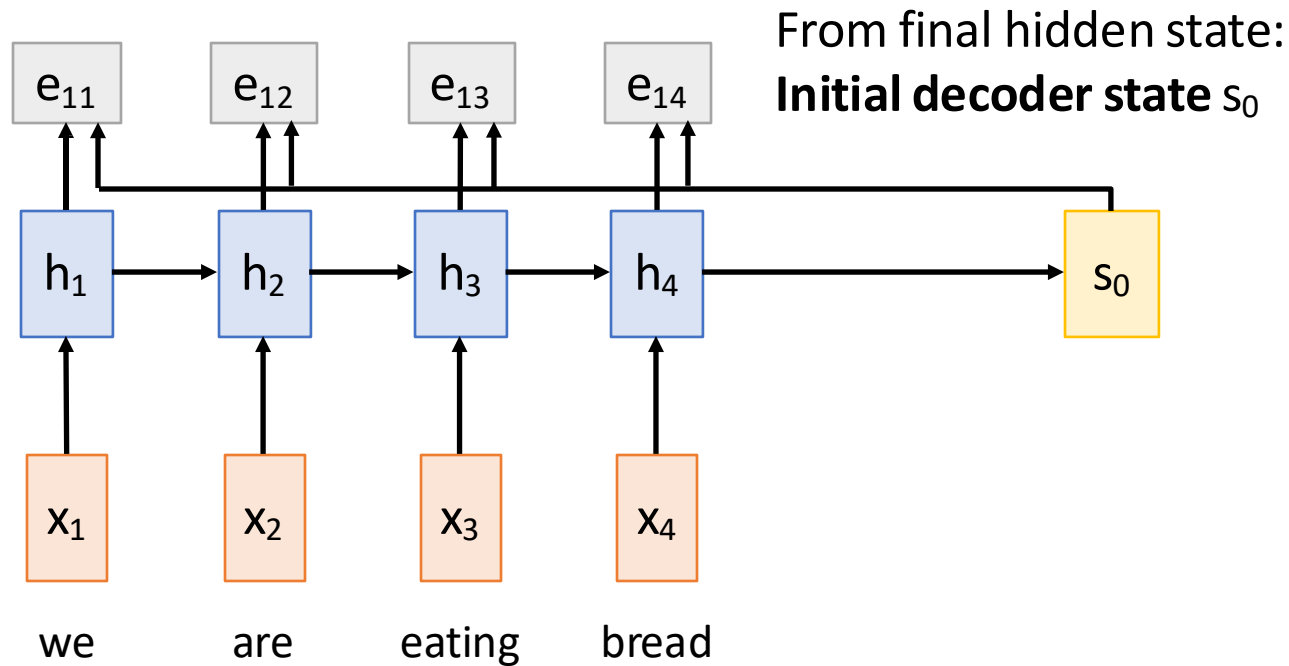
From final hidden state:
Initial decoder state s_0



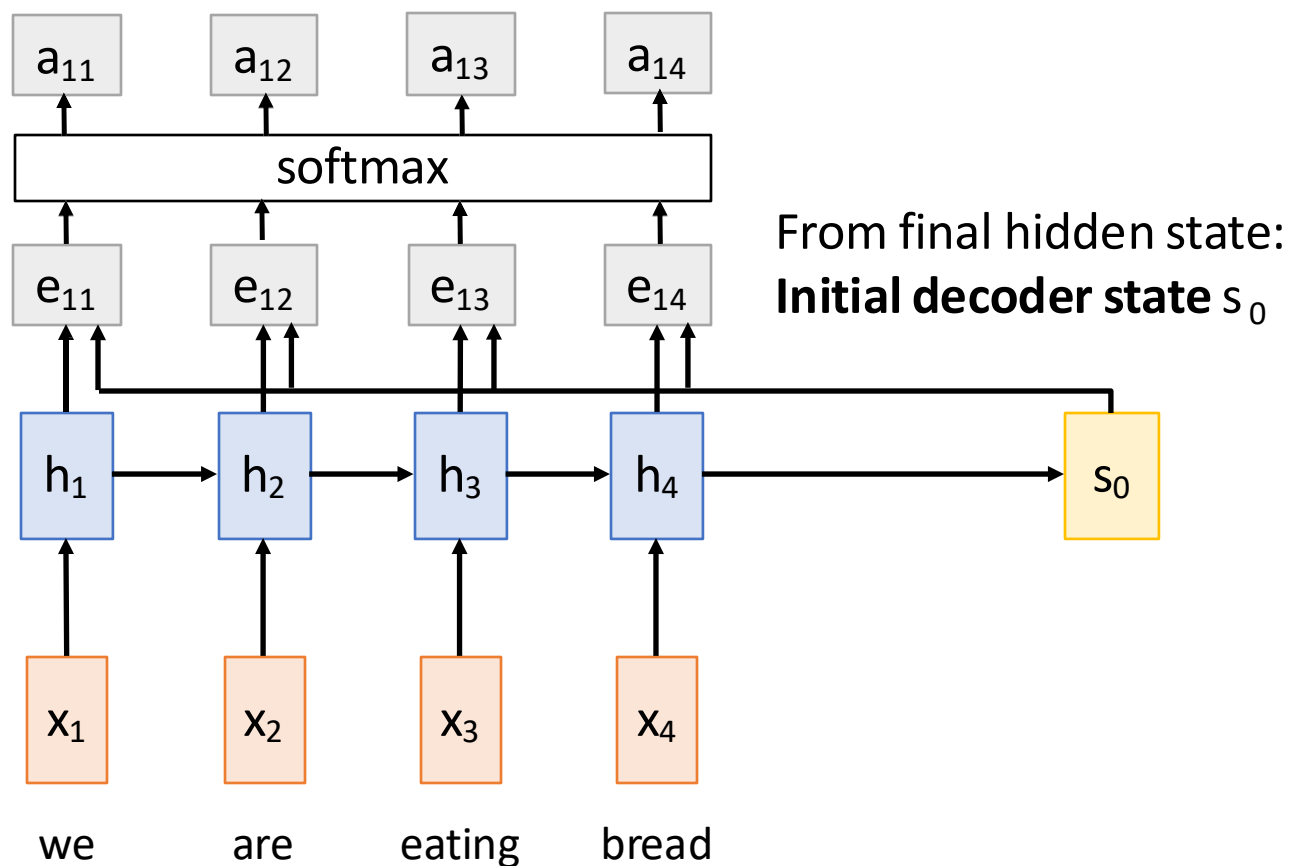
Sequence-to-Sequence with RNNs and Attention

Compute (scalar) **alignment scores**

$$e_{t,i} = f_{\text{att}}(s_{t-1}, h_i) \quad (f_{\text{att}} \text{ is an MLP})$$



Sequence-to-Sequence with RNNs and Attention



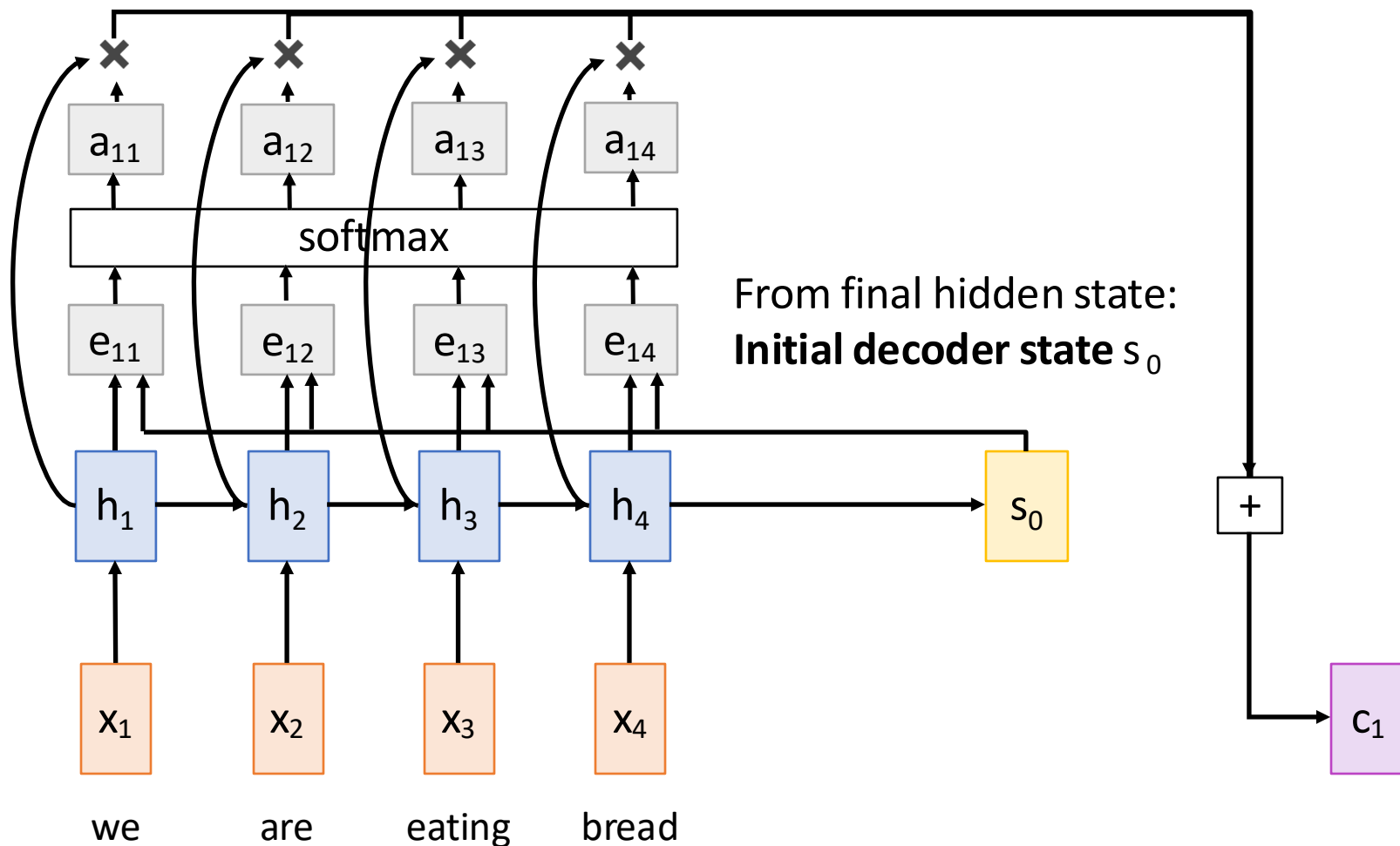
Compute (scalar) **alignment scores**

$$e_{t,i} = f_{\text{att}}(s_{t-1}, h_i) \quad (f_{\text{att}} \text{ is an MLP})$$

Normalize alignment scores
to get **attention weights**

$$0 < a_{t,i} < 1 \quad \sum_i a_{t,i} = 1$$

Sequence-to-Sequence with RNNs and Attention



Compute (scalar) **alignment scores**

$$e_{t,i} = f_{\text{att}}(s_{t-1}, h_i) \quad (f_{\text{att}} \text{ is an MLP})$$

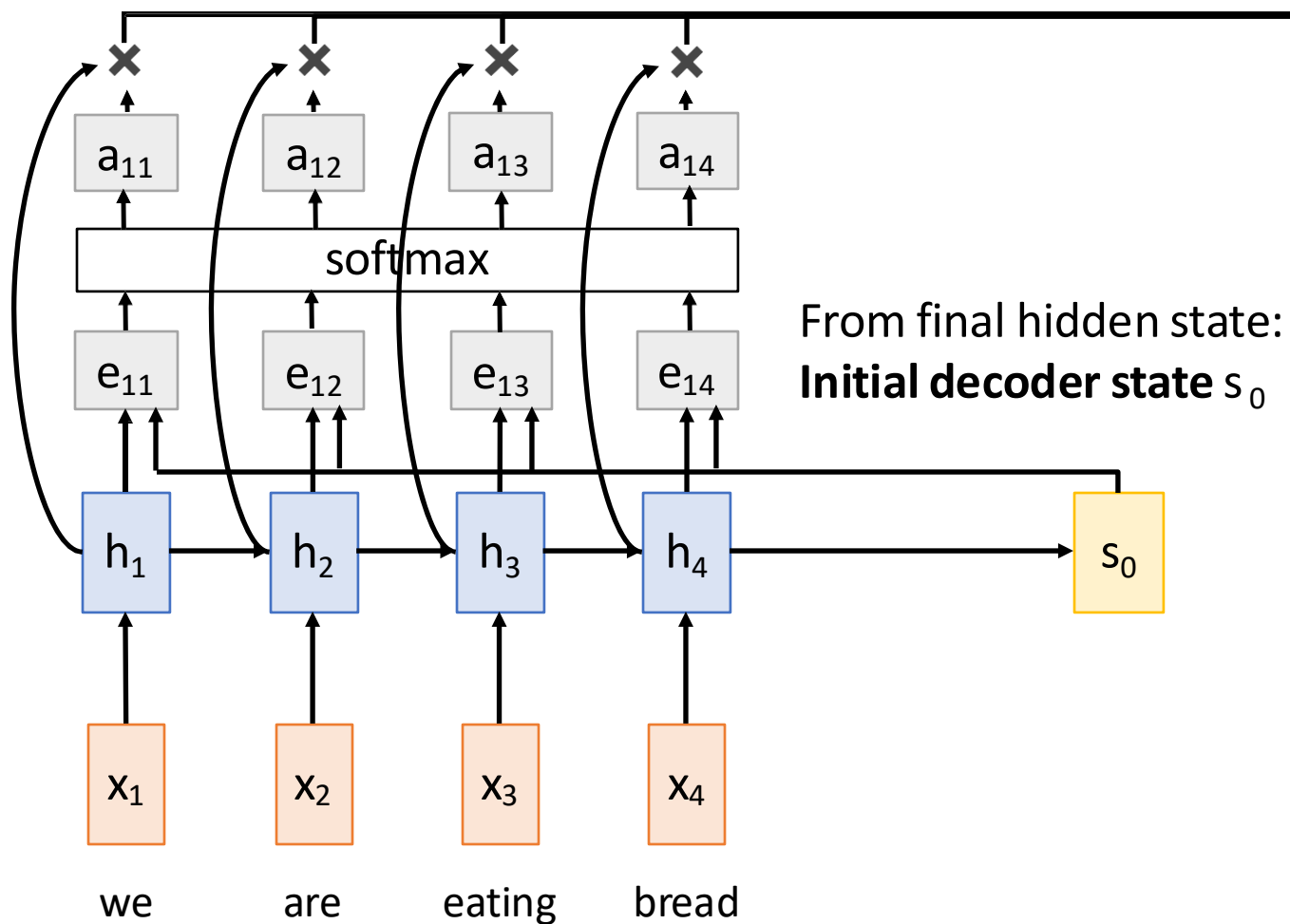
Normalize alignment scores
to get **attention weights**

$$0 < a_{t,i} < 1 \quad \sum_i a_{t,i} = 1$$

Compute context vector as linear
combination of hidden states

$$c_t = \sum_i a_{t,i} h_i$$

Sequence-to-Sequence with RNNs and Attention



Compute (scalar) **alignment scores**
 $e_{t,i} = f_{\text{att}}(s_{t-1}, h_i)$ (f_{att} is an MLP)

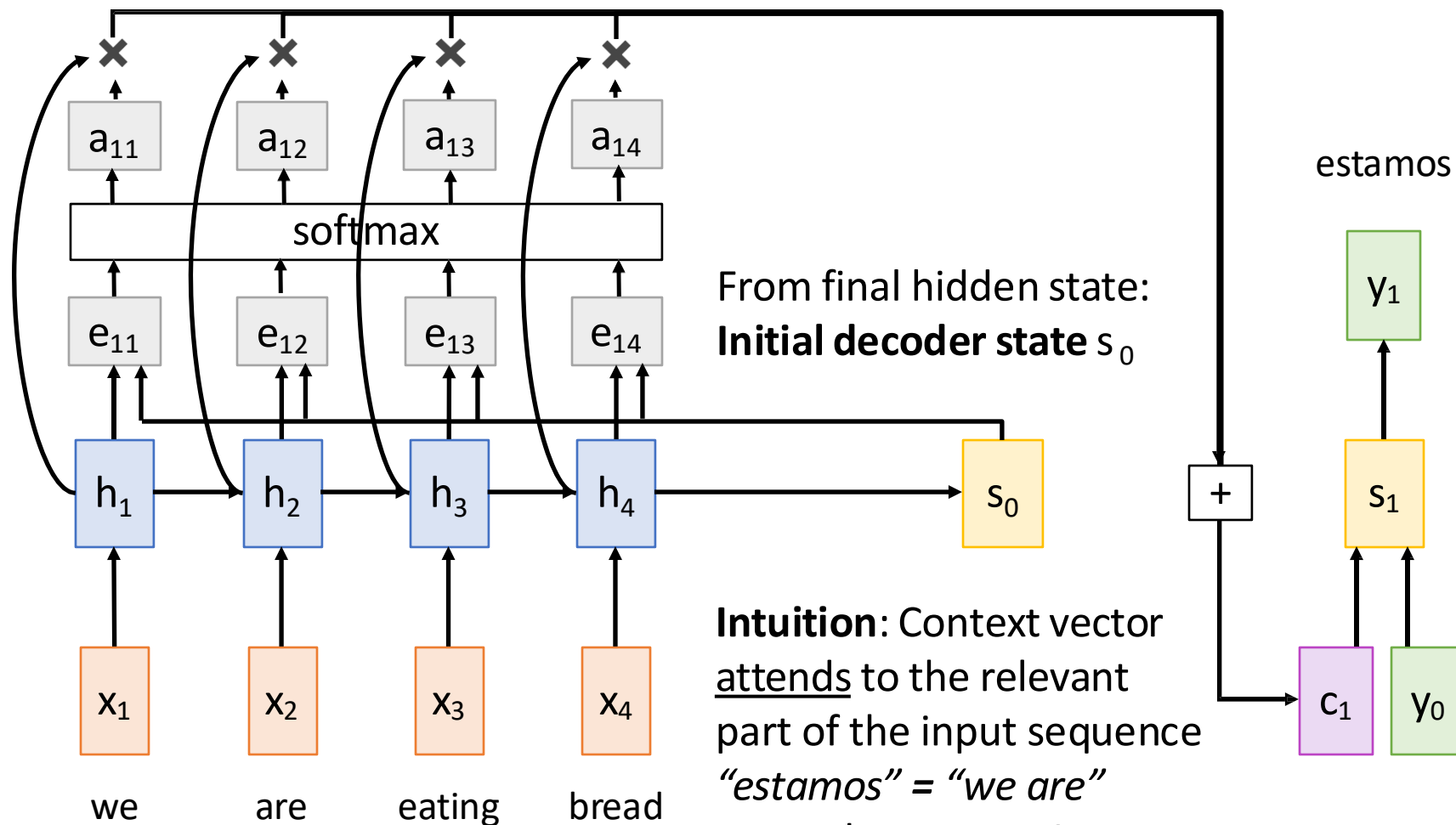
Normalize alignment scores
to get **attention weights**
 $0 < a_{t,i} < 1 \quad \sum_i a_{t,i} = 1$

Compute context vector as linear
combination of hidden states
 $c_t = \sum_i a_{t,i} h_i$

Use context vector in
decoder: $s_t = g_U(y_{t-1}, s_{t-1}, c_t)$

**This is all differentiable! Do not
supervise attention weights –
backprop through everything**

Sequence-to-Sequence with RNNs and Attention



From final hidden state:
Initial decoder state s_0

Intuition: Context vector attends to the relevant part of the input sequence
"estamos" = "we are"
so maybe $a_{11}=a_{12}=0.45$,
 $a_{13}=a_{14}=0.05$

Compute (scalar) **alignment scores**
 $e_{t,i} = f_{\text{att}}(s_{t-1}, h_i)$ (f_{att} is an MLP)

Normalize alignment scores to get **attention weights**
 $0 < a_{t,i} < 1 \quad \sum_i a_{t,i} = 1$

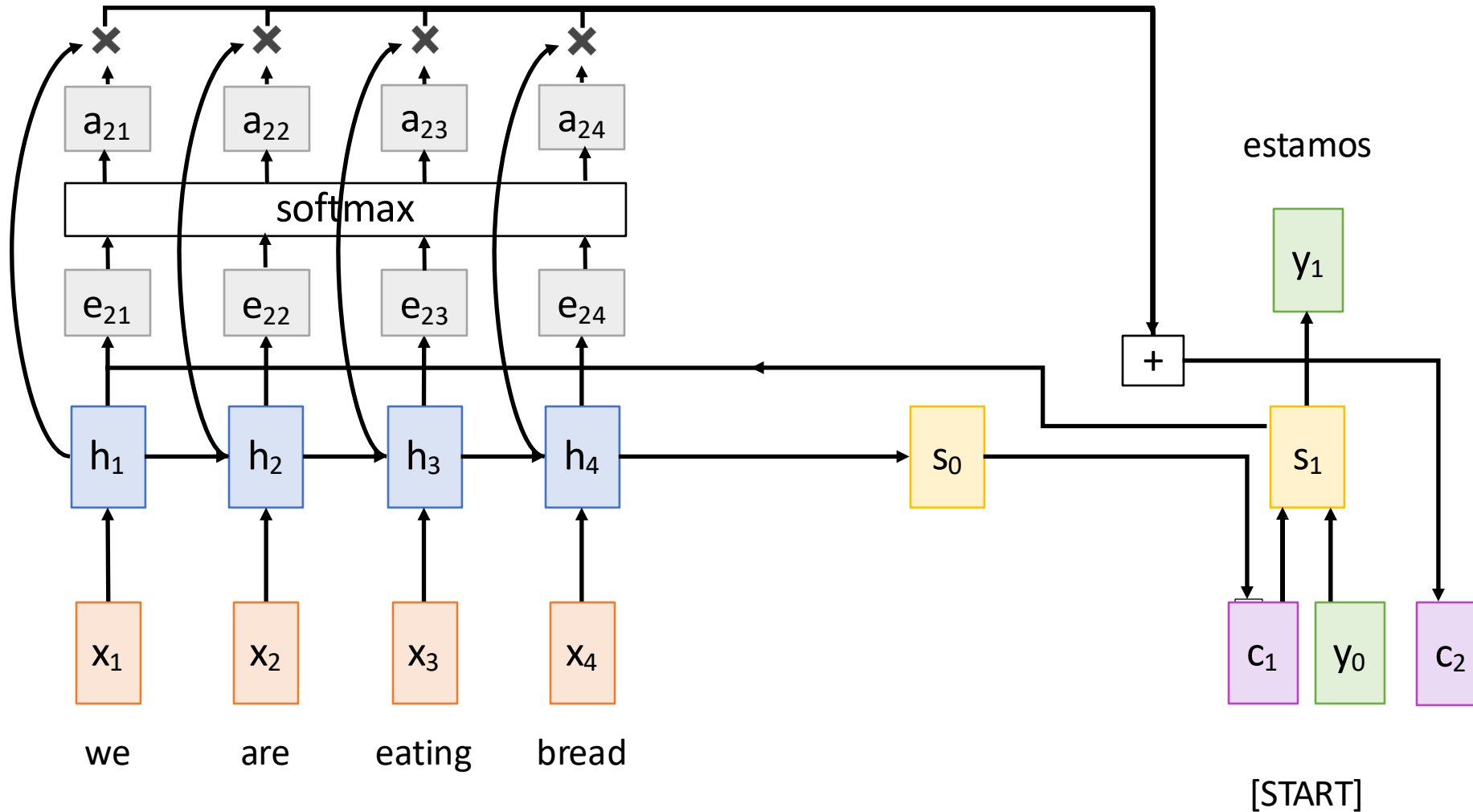
Compute context vector as linear combination of hidden states
 $c_t = \sum_i a_{t,i} h_i$

Use context vector in decoder: $s_t = g_U(y_{t-1}, s_{t-1}, c_t)$

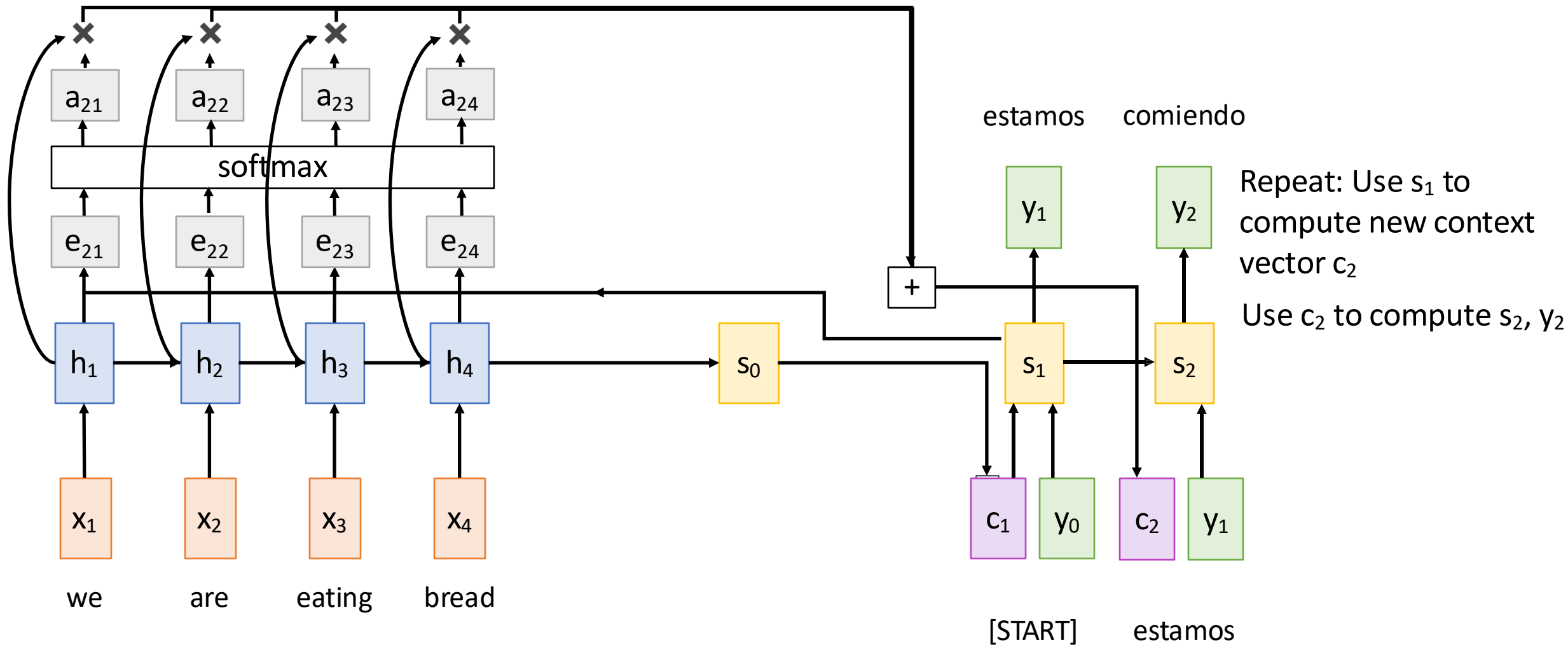
This is all differentiable! Do not supervise attention weights – backprop through everything

Sequence-to-Sequence with RNNs

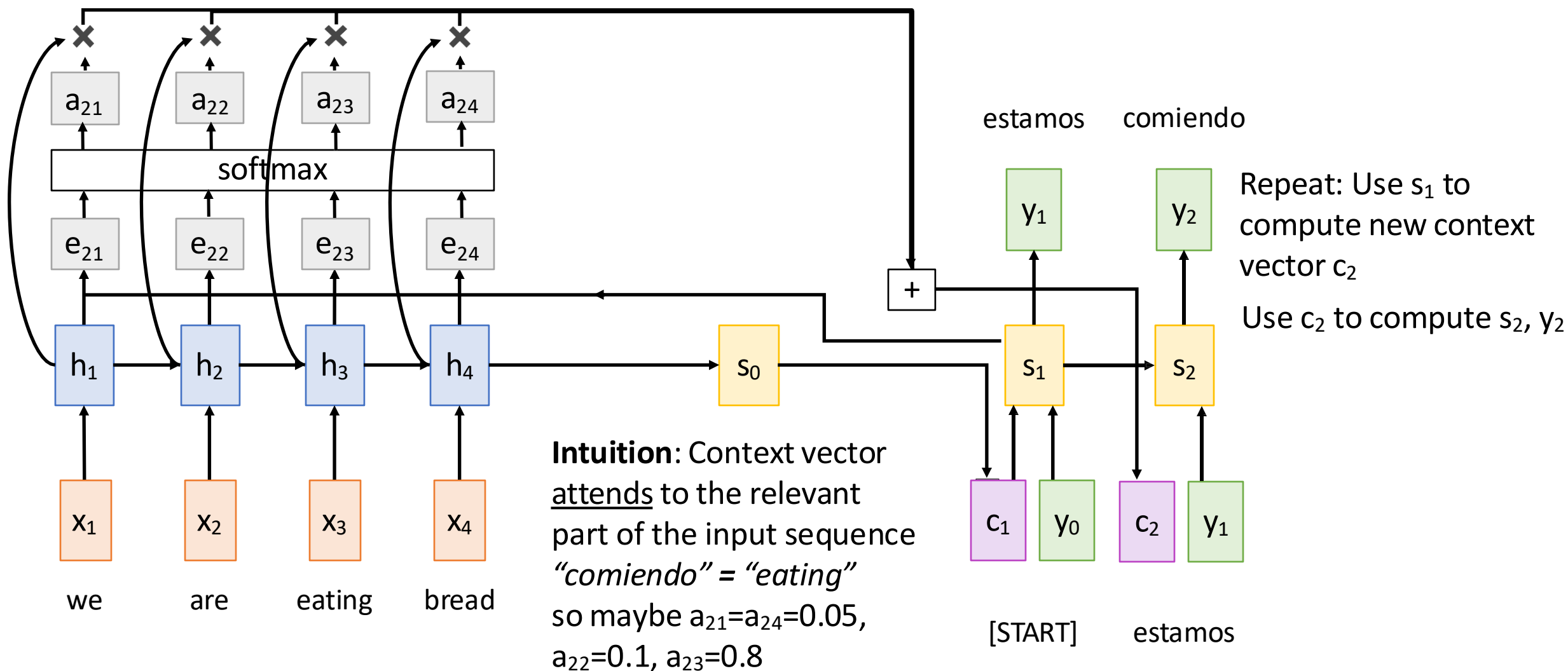
Repeat: Use s_1 to compute new context vector c_2



Sequence-to-Sequence with RNNs and Attention



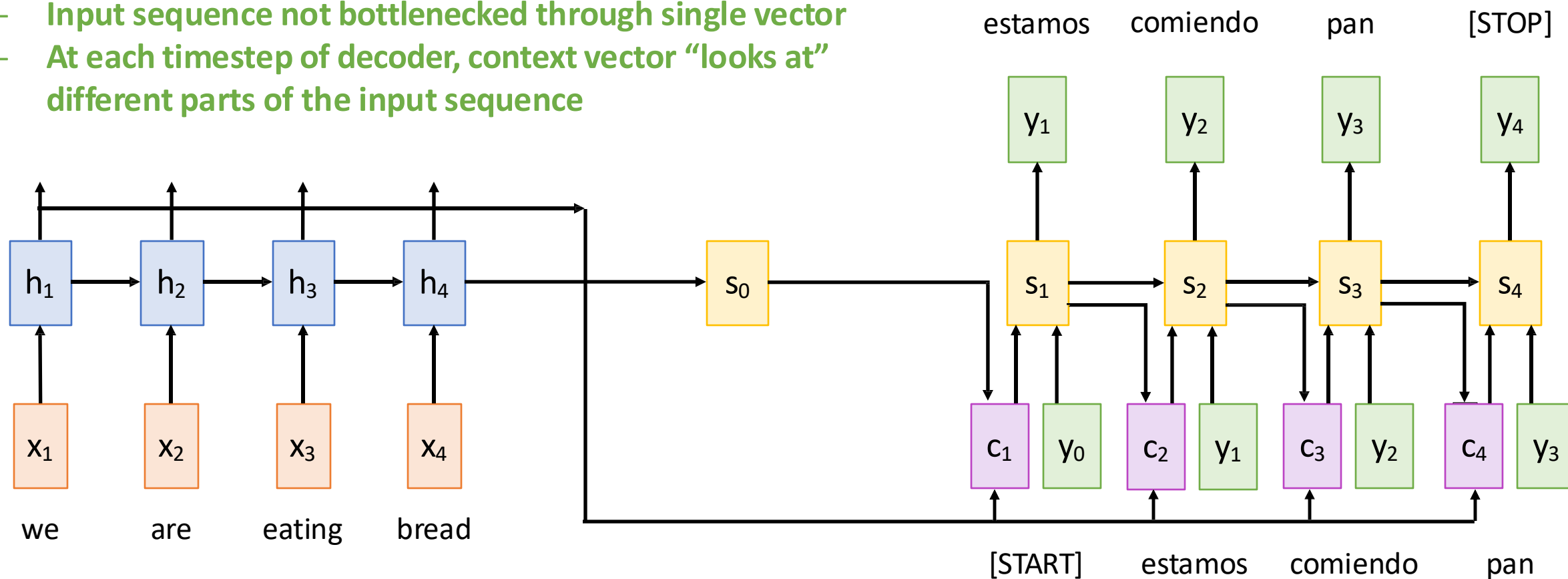
Sequence-to-Sequence with RNNs and Attention



Sequence-to-Sequence with RNNs and Attention

Use a different context vector in each timestep of decoder

- Input sequence not bottlenecked through single vector
- At each timestep of decoder, context vector “looks at” different parts of the input sequence



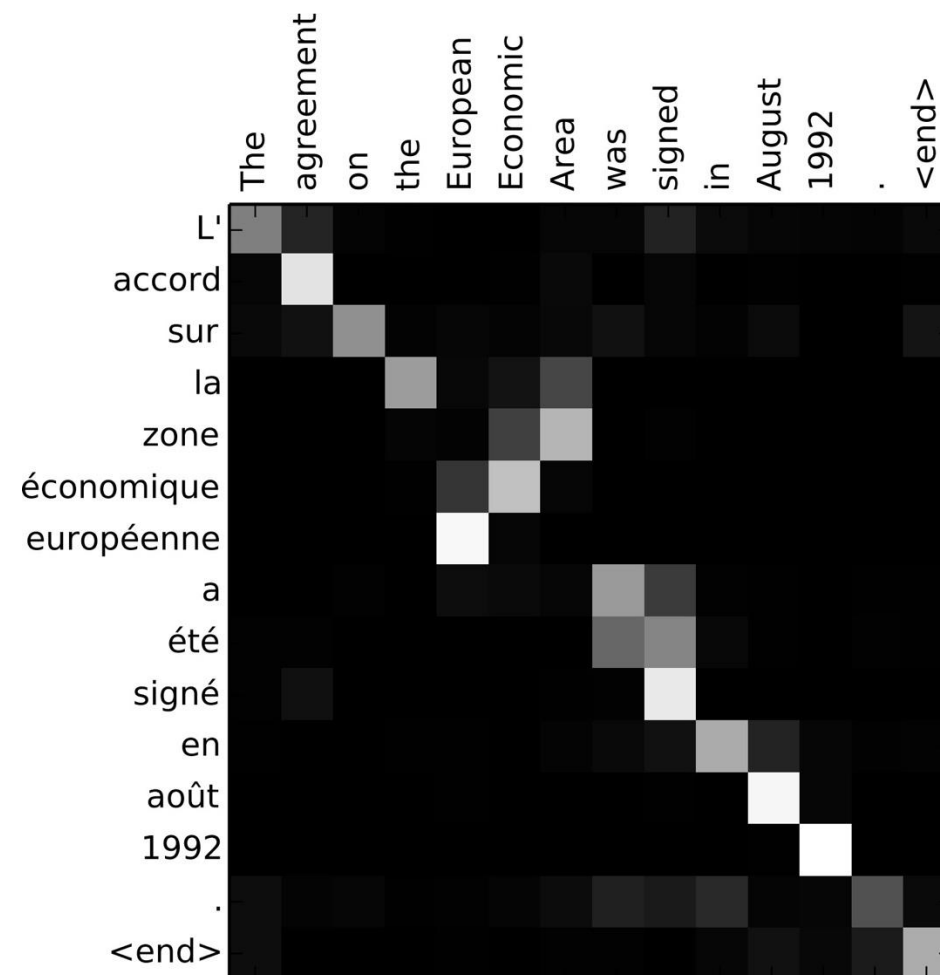
Sequence-to-Sequence with RNNs and Attention

Example: English to French translation

Input: “The agreement on the European Economic Area was signed in August 1992.”

Output: “L’accord sur la zone économique européenne a été signé en août 1992.”

Visualize attention weights $a_{t,i}$



Sequence-to-Sequence with RNNs and Attention

Example: English to French translation

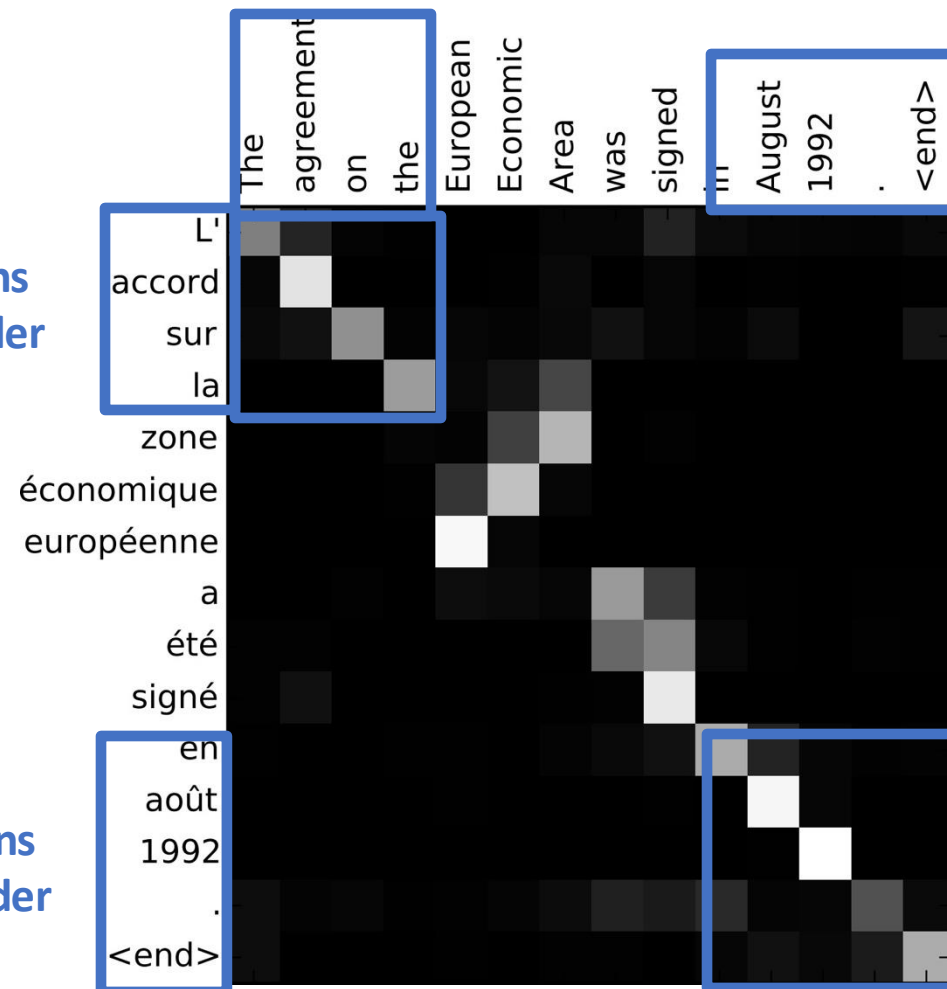
Input: “**The agreement on the** European Economic Area was signed **in August 1992.**”

Output: “**L’accord sur la** zone économique européenne a été signé **en août 1992.**”

Diagonal attention means words correspond in order

Diagonal attention means words correspond in order

Visualize attention weights $a_{t,i}$



Sequence-to-Sequence with RNNs and Attention

Example: English to French translation

Input: “The agreement on the European Economic Area was signed in August 1992.”

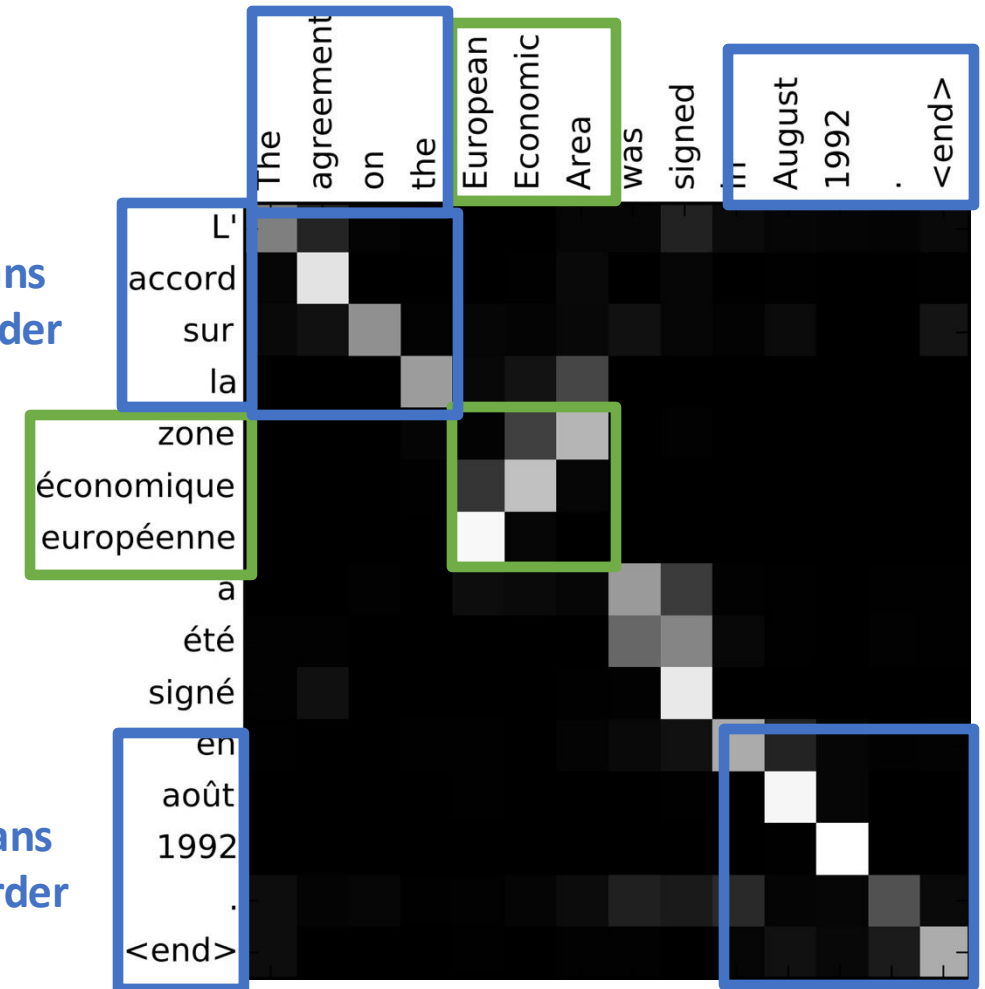
Output: “L'accord sur la zone économique européenne a été signé en août 1992.”

Visualize attention weights $a_{t,i}$

Diagonal attention means words correspond in order

Attention figures out different word orders

Diagonal attention means words correspond in order



Sequence-to-Sequence with RNNs and Attention

Example: English to French translation

Input: “The agreement on the European Economic Area was signed in August 1992.”

Output: “L'accord sur la zone économique européenne a été signé en août 1992.”

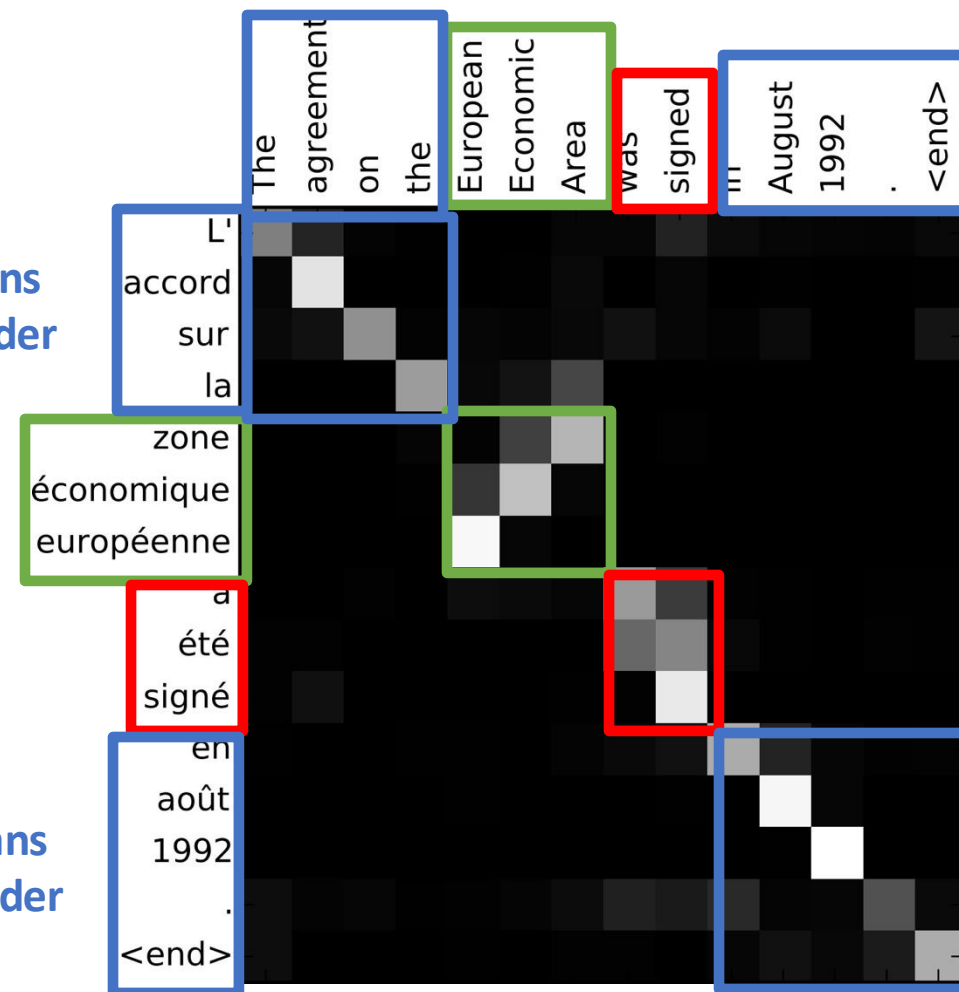
Visualize attention weights $a_{t,i}$

Diagonal attention means words correspond in order

Attention figures out different word orders

Verb conjugation

Diagonal attention means words correspond in order



Sequence-to-Sequence with RNNs and Attention

The decoder doesn't use the fact that h_i form an ordered sequence – it just treats them as an unordered set $\{h_i\}$

Can use similar architecture given any set of input hidden vectors $\{h_i\}$!

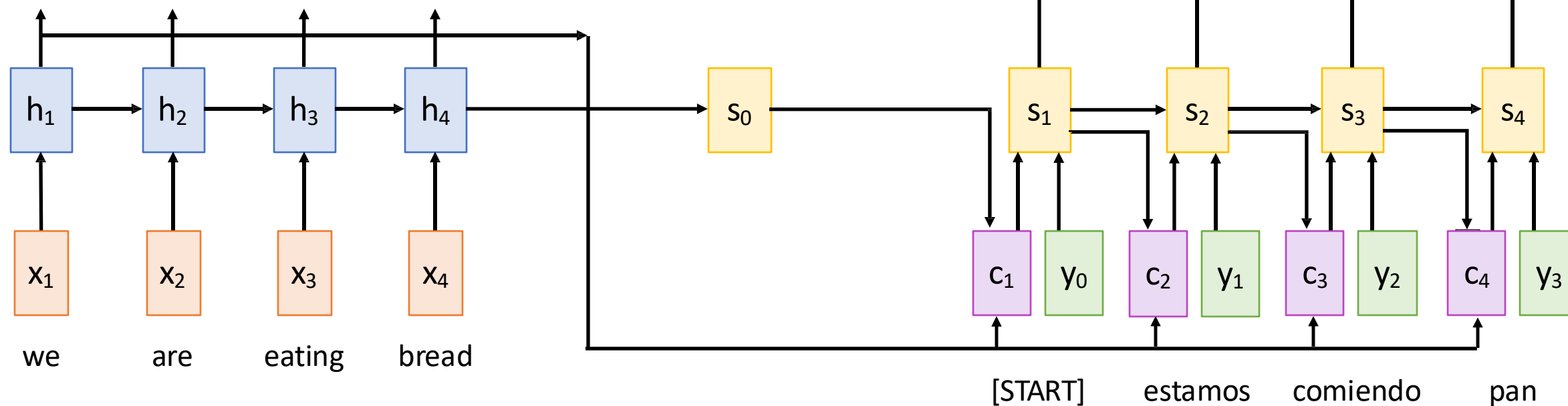
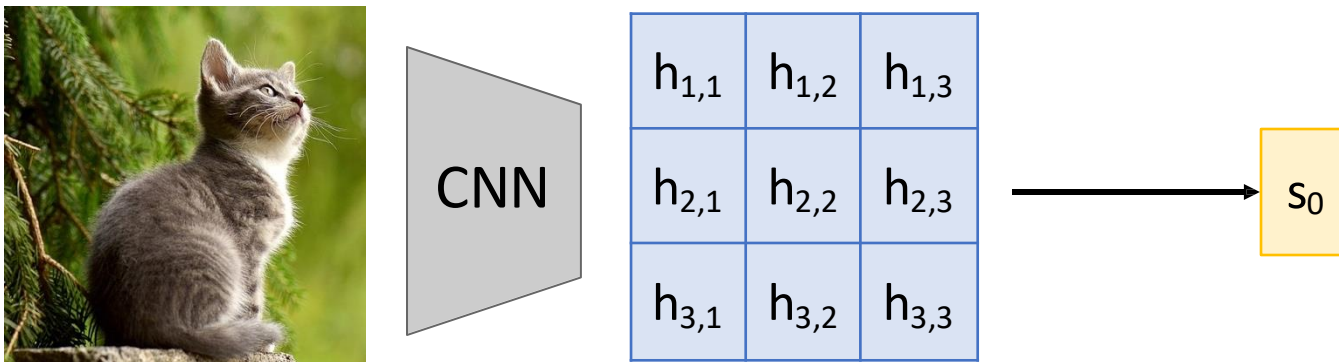


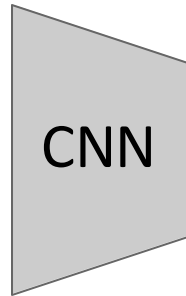
Image Captioning with RNNs and Attention



Use a CNN to compute a
grid of features for an image

Image Captioning with RNNs and Attention

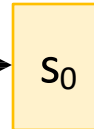
$$e_{t,i,j} = f_{\text{att}}(s_{t-1}, h_{i,j})$$



$h_{1,1}$	$h_{1,2}$	$h_{1,3}$
$h_{2,1}$	$h_{2,2}$	$h_{2,3}$
$h_{3,1}$	$h_{3,2}$	$h_{3,3}$

Alignment scores

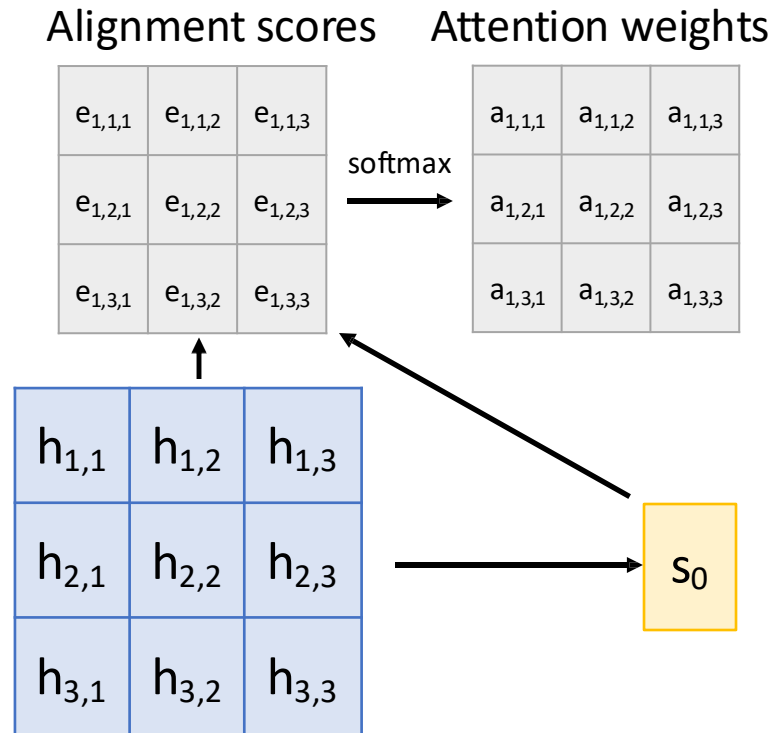
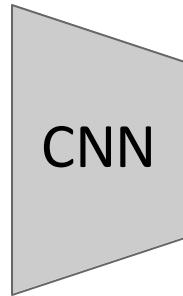
$e_{1,1,1}$	$e_{1,1,2}$	$e_{1,1,3}$
$e_{1,2,1}$	$e_{1,2,2}$	$e_{1,2,3}$
$e_{1,3,1}$	$e_{1,3,2}$	$e_{1,3,3}$



Use a CNN to compute a
grid of features for an image

Image Captioning with RNNs and Attention

$$e_{t,i,j} = f_{\text{att}}(s_{t-1}, h_{i,j})$$
$$a_{t,:} = \text{softmax}(e_{t,:})$$



Use a CNN to compute a
grid of features for an image

Image Captioning with RNNs and Attention

$$e_{t,i,j} = f_{\text{att}}(s_{t-1}, h_{i,j})$$
$$a_{t,:} = \text{softmax}(e_{t,:})$$
$$c_t = \sum_{i,j} a_{t,i,j} h_{i,j}$$

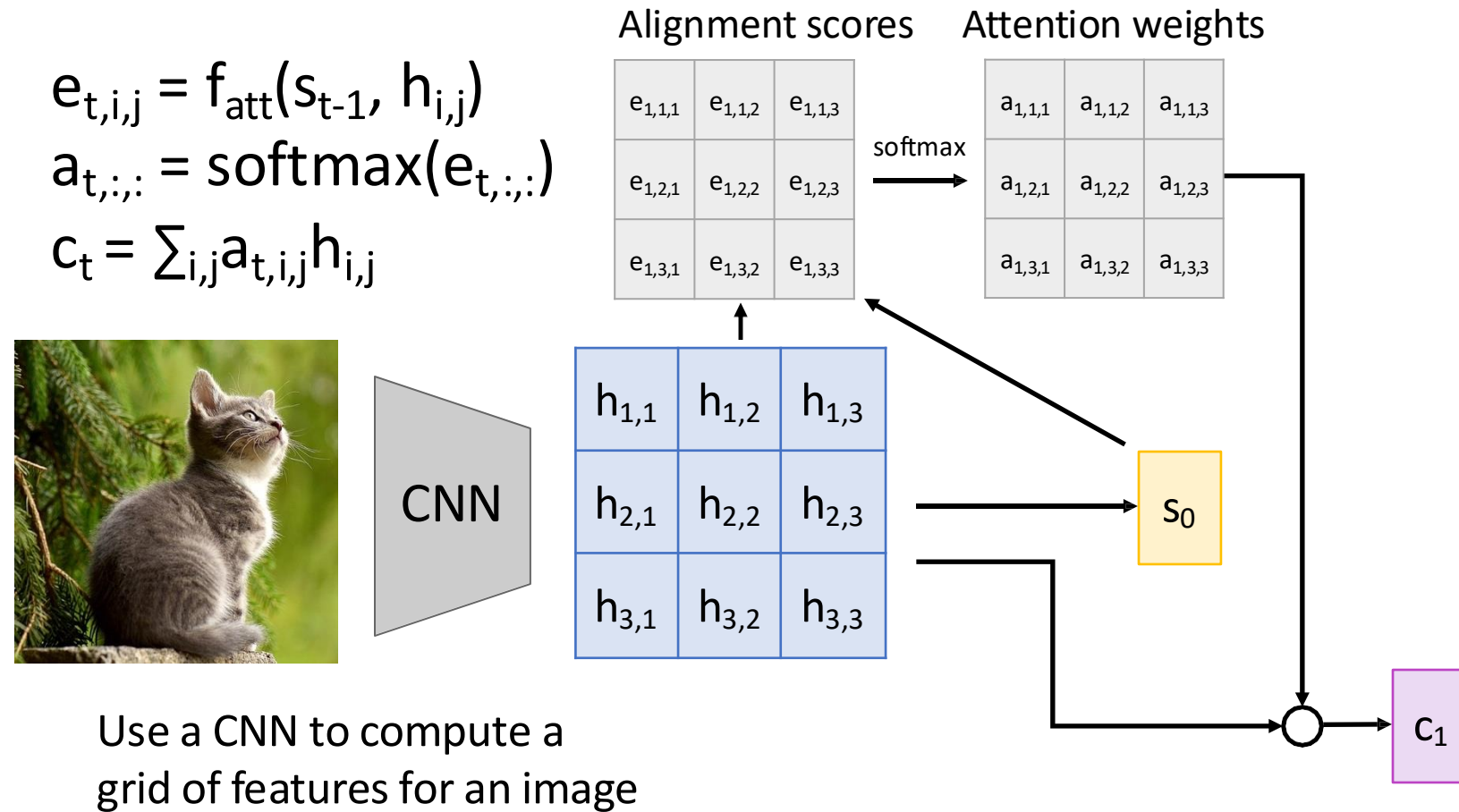
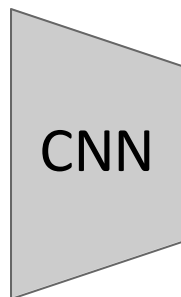


Image Captioning with RNNs and Attention

$$e_{t,i,j} = f_{\text{att}}(s_{t-1}, h_{i,j})$$
$$a_{t,:} = \text{softmax}(e_{t,:})$$
$$c_t = \sum_{i,j} a_{t,i,j} h_{i,j}$$



Use a CNN to compute a
grid of features for an image

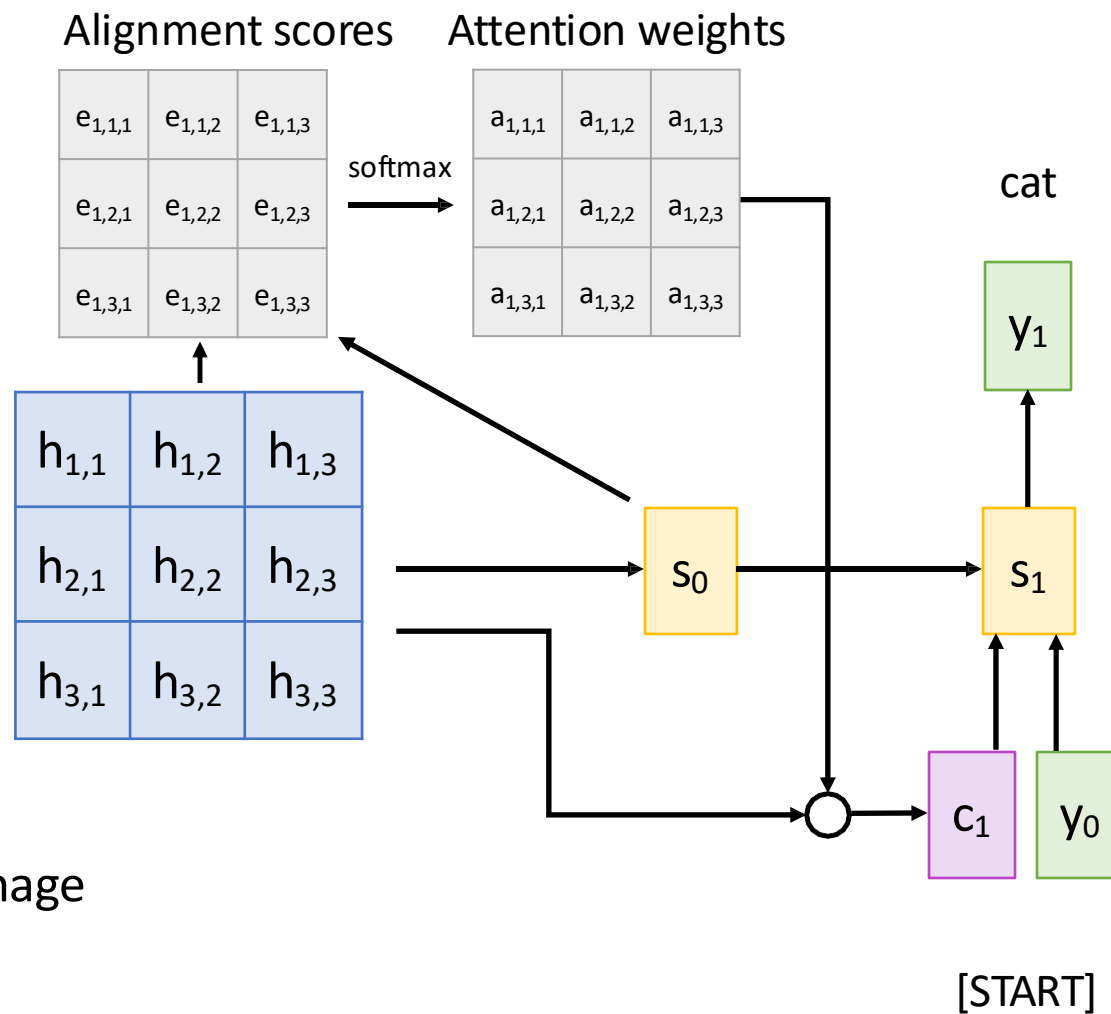


Image Captioning with RNNs and Attention

$$e_{t,i,j} = f_{\text{att}}(s_{t-1}, h_{i,j})$$
$$a_{t,:} = \text{softmax}(e_{t,:})$$
$$c_t = \sum_{i,j} a_{t,i,j} h_{i,j}$$

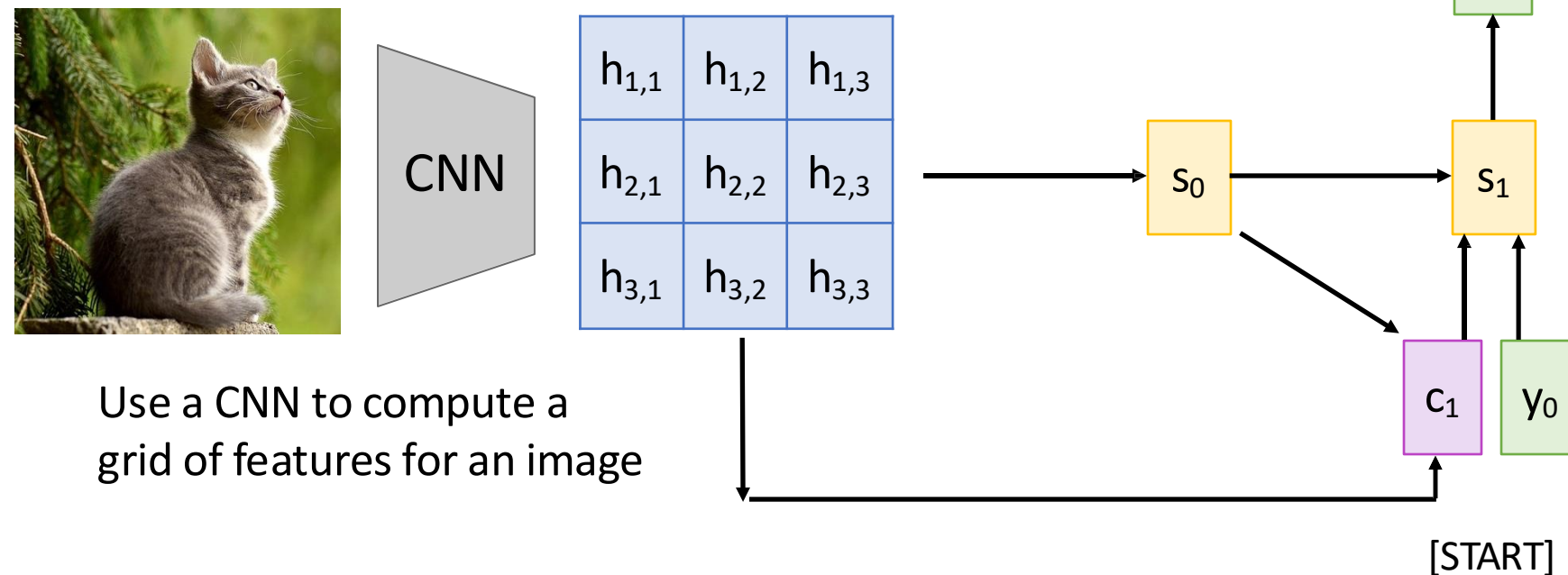
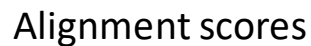


Image Captioning with RNNs and Attention

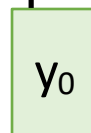
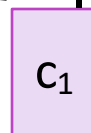
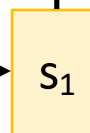
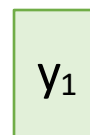
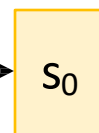
$$\begin{aligned} e_{t,i,j} &= f_{\text{att}}(s_{t-1}, h_{i,j}) \\ a_{t,:,:} &= \text{softmax}(e_{t,:,:}) \\ c_t &= \sum_{i,j} a_{t,i,j} h_{i,j} \end{aligned}$$



$e_{2,1,1}$	$e_{2,1,2}$	$e_{2,1,3}$
$e_{2,2,1}$	$e_{2,2,2}$	$e_{2,2,3}$
$e_{2,3,1}$	$e_{2,3,2}$	$e_{2,3,3}$



$h_{1,1}$	$h_{1,2}$	$h_{1,3}$
$h_{2,1}$	$h_{2,2}$	$h_{2,3}$
$h_{3,1}$	$h_{3,2}$	$h_{3,3}$



[START]

Use a CNN to compute a grid of features for an image



CNN

Image Captioning with RNNs and Attention

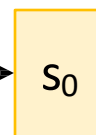
$$e_{t,i,j} = f_{\text{att}}(s_{t-1}, h_{i,j})$$
$$a_{t,:} = \text{softmax}(e_{t,:})$$
$$c_t = \sum_{i,j} a_{t,i,j} h_{i,j}$$

Alignment scores Attention weights

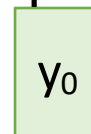
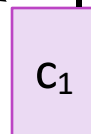
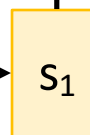
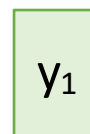
$e_{2,1,1}$	$e_{2,1,2}$	$e_{2,1,3}$	$\xrightarrow{\text{softmax}}$	$a_{2,1,1}$	$a_{2,1,2}$	$a_{2,1,3}$
$e_{2,2,1}$	$e_{2,2,2}$	$e_{2,2,3}$		$a_{2,2,1}$	$a_{2,2,2}$	$a_{2,2,3}$
$e_{2,3,1}$	$e_{2,3,2}$	$e_{2,3,3}$		$a_{2,3,1}$	$a_{2,3,2}$	$a_{2,3,3}$



$h_{1,1}$	$h_{1,2}$	$h_{1,3}$
$h_{2,1}$	$h_{2,2}$	$h_{2,3}$
$h_{3,1}$	$h_{3,2}$	$h_{3,3}$



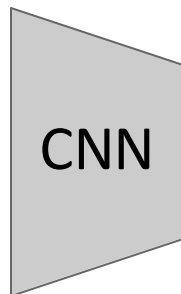
cat



[START]



$\xrightarrow{\text{softmax}}$



Use a CNN to compute a grid of features for an image

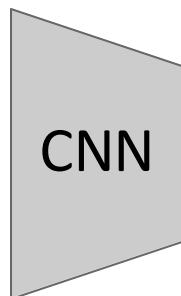
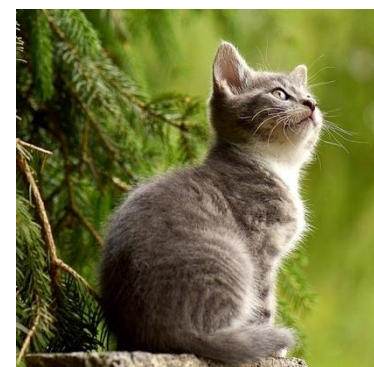


Image Captioning with RNNs and Attention

$$e_{t,i,j} = f_{\text{att}}(s_{t-1}, h_{i,j})$$

$$a_{t,:} = \text{softmax}(e_{t,:})$$

$$c_t = \sum_{i,j} a_{t,i,j} h_{i,j}$$



Use a CNN to compute a grid of features for an image

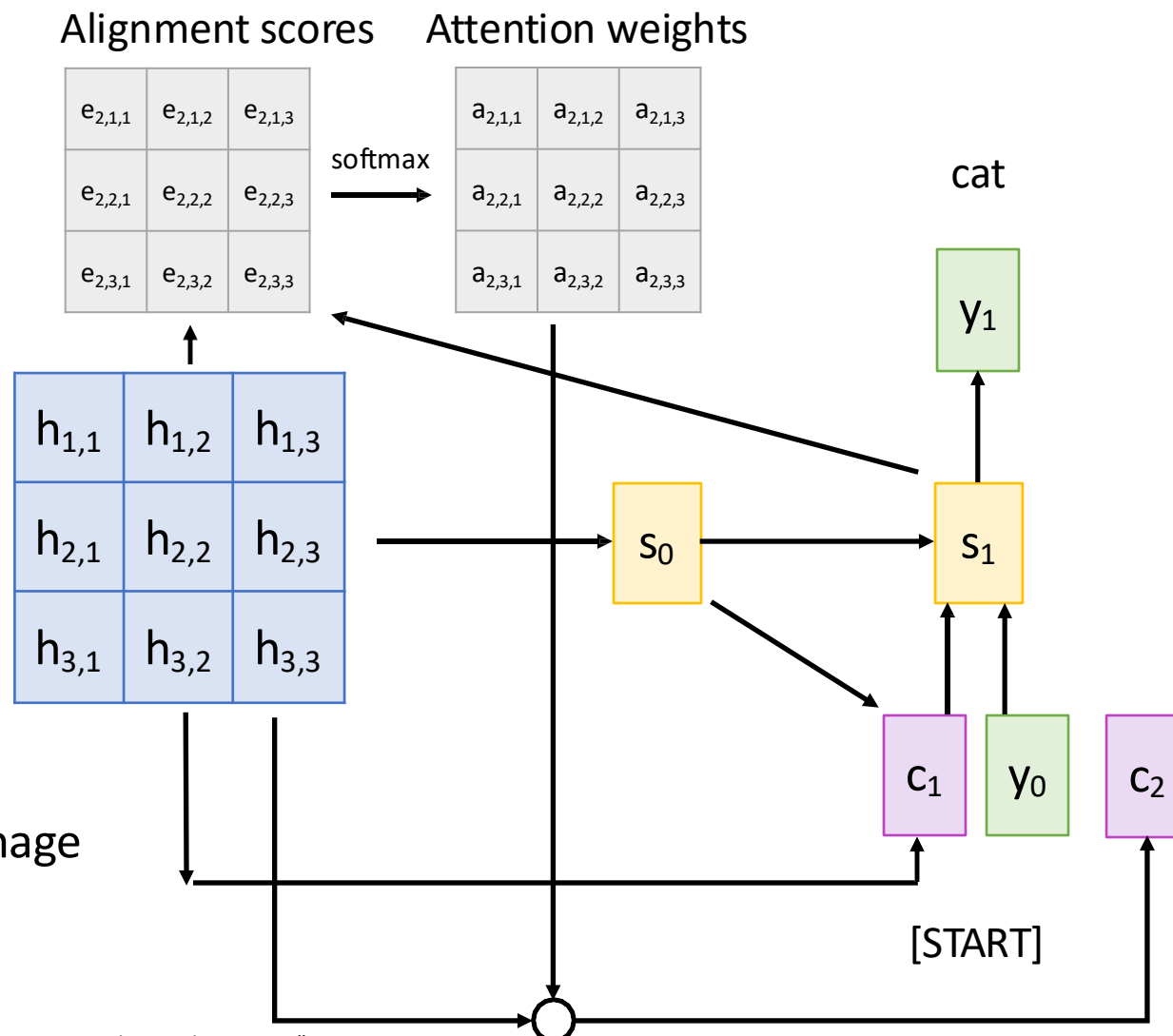


Image Captioning with RNNs and Attention

$$e_{t,i,j} = f_{\text{att}}(s_{t-1}, h_{i,j})$$

$$a_{t,:} = \text{softmax}(e_{t,:})$$

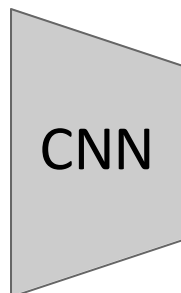
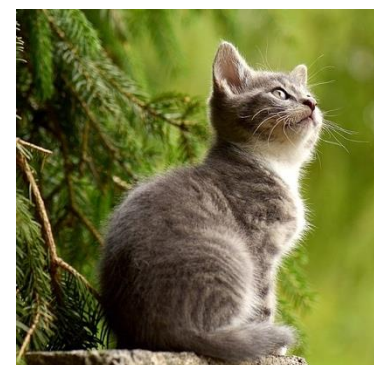
$$c_t = \sum_{i,j} a_{t,i,j} h_{i,j}$$

Alignment scores Attention weights

$e_{2,1,1}$	$e_{2,1,2}$	$e_{2,1,3}$
$e_{2,2,1}$	$e_{2,2,2}$	$e_{2,2,3}$
$e_{2,3,1}$	$e_{2,3,2}$	$e_{2,3,3}$

softmax

$a_{2,1,1}$	$a_{2,1,2}$	$a_{2,1,3}$
$a_{2,2,1}$	$a_{2,2,2}$	$a_{2,2,3}$
$a_{2,3,1}$	$a_{2,3,2}$	$a_{2,3,3}$



Use a CNN to compute a grid of features for an image

$h_{1,1}$	$h_{1,2}$	$h_{1,3}$
$h_{2,1}$	$h_{2,2}$	$h_{2,3}$
$h_{3,1}$	$h_{3,2}$	$h_{3,3}$

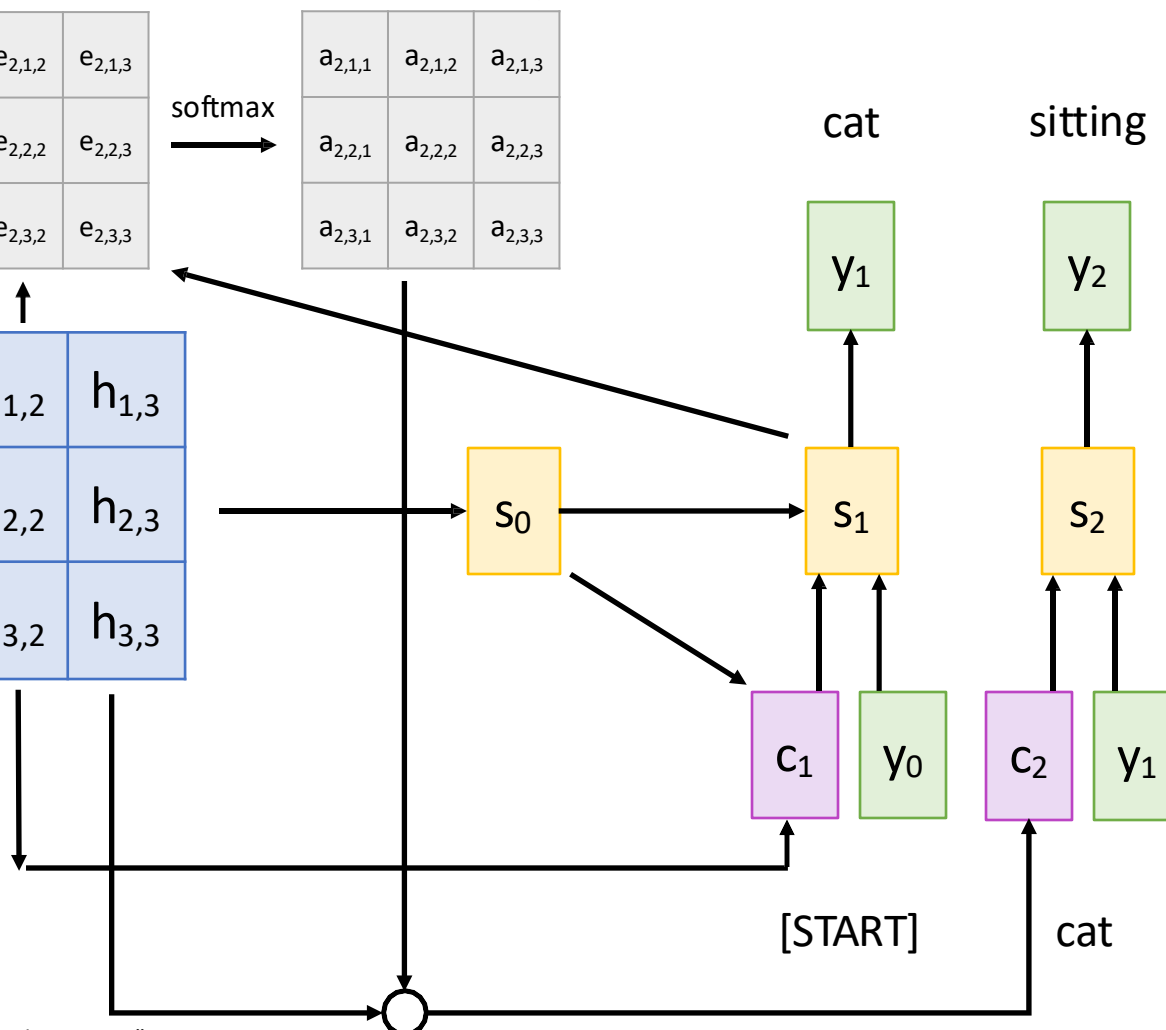


Image Captioning with RNNs and Attention

$$e_{t,i,j} = f_{\text{att}}(s_{t-1}, h_{i,j})$$
$$a_{t,:} = \text{softmax}(e_{t,:})$$
$$c_t = \sum_{i,j} a_{t,i,j} h_{i,j}$$

Each timestep of decoder
uses a different context
vector that looks at different
parts of the input image

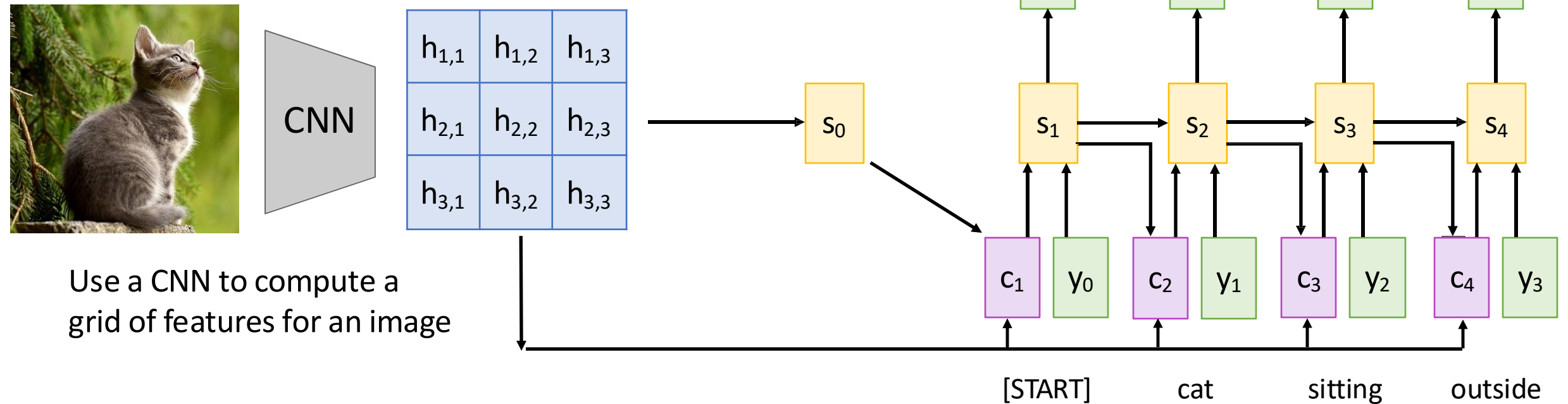


Image Captioning with RNNs and Attention

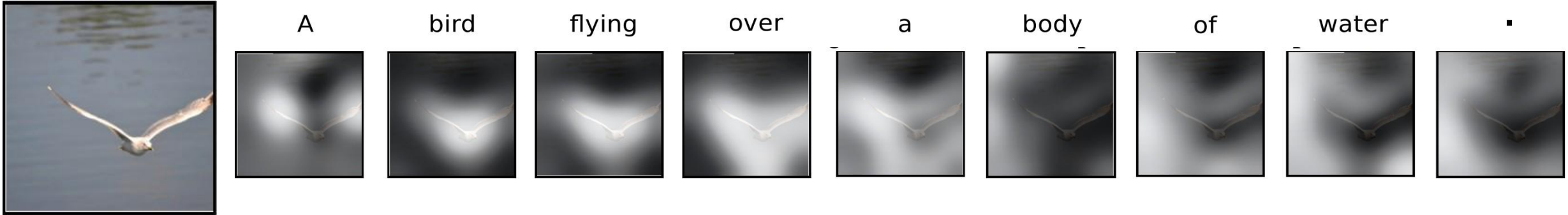


Image Captioning with RNNs and Attention



A dog is standing on a hardwood floor.



A stop sign is on a road with a mountain in the background.



A group of people sitting on a boat in the water.



A giraffe standing in a forest with trees in the background.

X, Attend, and Y

“Show, attend, and tell” (*Xu et al, ICML 2015*)

Look at image, attend to image regions, produce question

“Ask, attend, and answer” (*Xu and Saenko, ECCV 2016*)

“Show, ask, attend, and answer” (*Kazemi and Elqursh, 2017*)

Read text of question, attend to image regions, produce answer

“Listen, attend, and spell” (*Chan et al, ICASSP 2016*)

Process raw audio, attend to audio regions while producing text

“Listen, attend, and walk” (*Mei et al, AAAI 2016*)

Process text, attend to text regions, output navigation commands

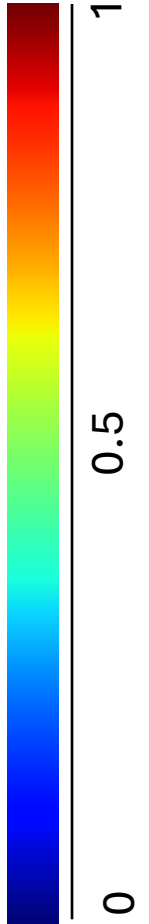
Co-Attention for Visual Question Answering



What **color** is the plastic **container** inside the cage?



What **animal** is in the cage?



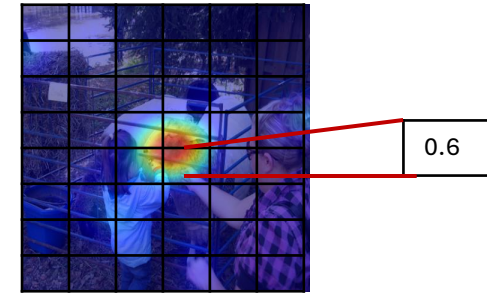
VQA elements

0.8
What animal is in the cage?

Elements : words

$$(Q_1, \dots, Q_{15}) \in \mathbb{R}^{15 \times d_Q}$$

(Embedding with RNN)



Elements: image regions

$$(I_1, \dots, I_{49}) \in \mathbb{R}^{49 \times d_I}$$

(Extracted from ResNet)

VQA Co-Attention

- Given an image I and a question Q
- We infer the amount of attention that should be given to each image-region and a word by computing attention belief b :

$$g_I(;) = \text{softmax}(b(I, Q, ;)), \quad g_Q(;) = \text{softmax}(b(I, Q, ;))$$

- We calculate the attend representation:

$$a_Q = \sum_{i=1..15} g_Q(i) \cdot Q_j \quad ; \quad a_I = \sum_{j=1..49} g_I(i) \cdot I_i$$

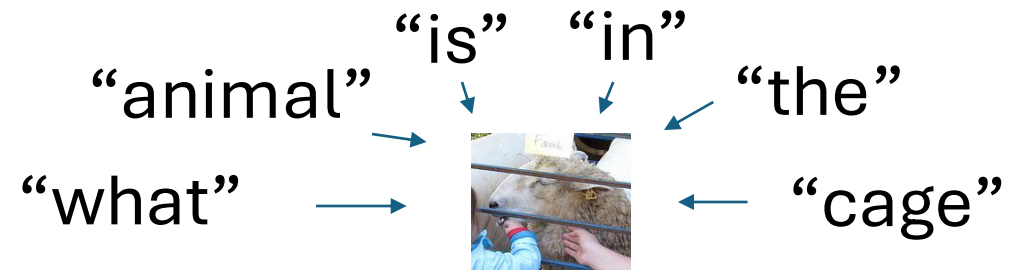
Factorize the attention belief

$$b_I(I, Q, i) \propto \exp(w_I \psi_I(i) + w_{I,Q} \mu_{Q \rightarrow I}(i))$$

Unary ($\psi_I(i)$): region information

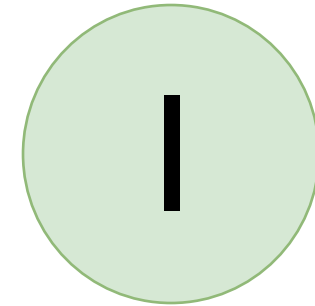
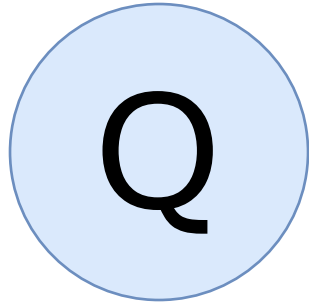


question-to-Image ($\mu_{Q \rightarrow I}(i)$):
interactions of regions with words



Factor Graph Attention

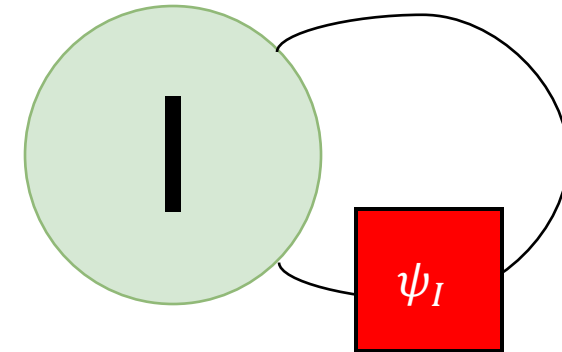
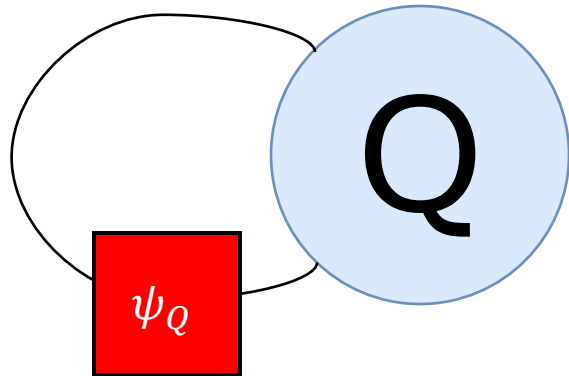
- Each modality is treated as a node



What animal is in the cage?



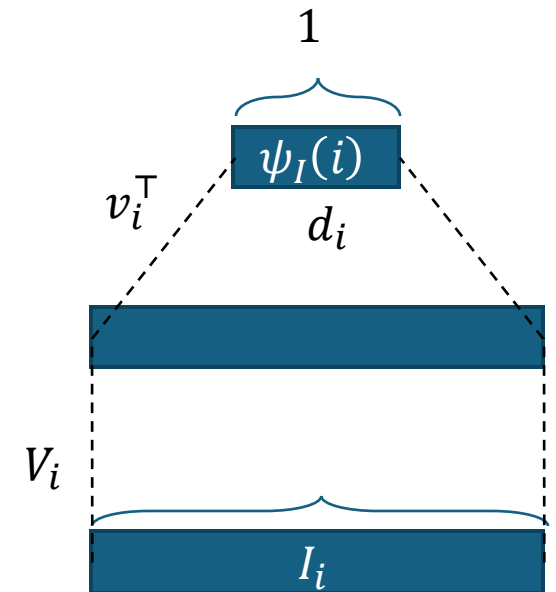
Unary factor



For each image region:

$$\psi_I(i) = v_i^\top \text{relu}(V_i I_i)$$

Where weights $v_i^\top \in \mathbb{R}^{d_I \times 1}$ and $V_i \in \mathbb{R}^{d_I \times d_I}$



$$b_I(I, Q, i) \propto \exp(w_I \psi_I(i) + w_{I,Q} \mu_{Q \rightarrow I}(i))$$

Unary attention visualize

- Entity information reacts to strong features across the dataset
- Can reveal dataset **bias**



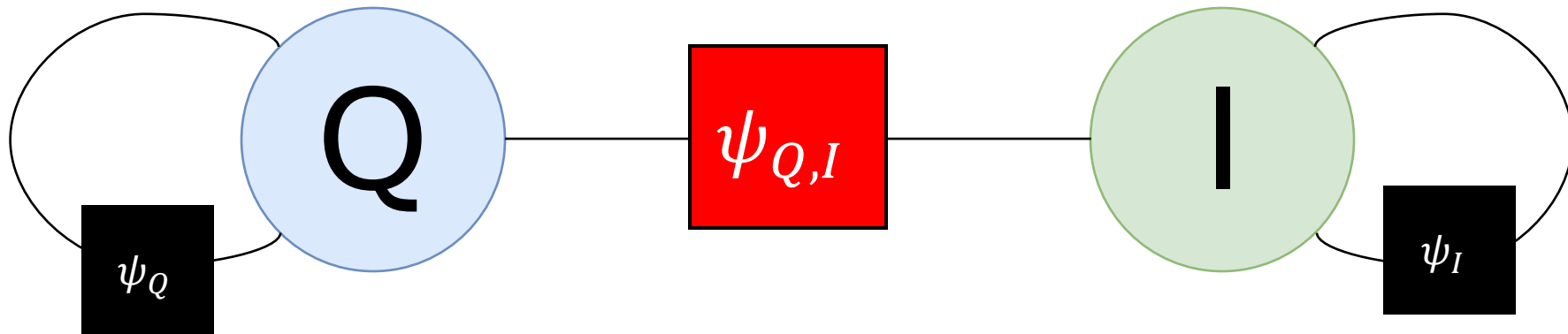
What animal is in the cage?



What color is the plastic container inside the cage?

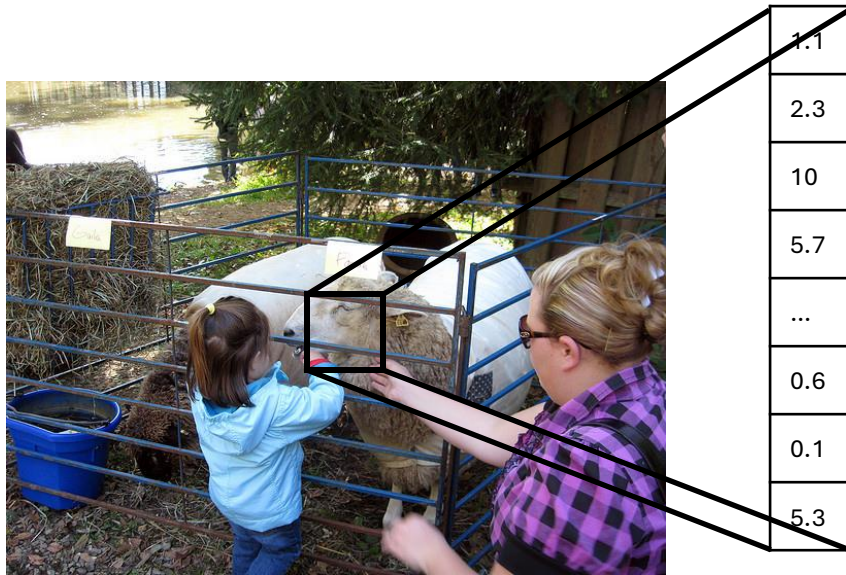
$$b_I(I, Q, i) \propto \exp(w_I \psi_I(i) + w_{I,Q} \mu_{Q \rightarrow I}(i))$$

Pairwise Interaction factor



$$b_I(I, Q, i) \propto \exp(w_I \psi_I(i) + w_{I,Q} \mu_{Q \rightarrow I}(i))$$

Interactions via dot-product



•

3.2	5.1	9	6	...	0.1	0.4	7.8
-----	-----	---	---	-----	-----	-----	-----

 = 6.5

What animal is in the cage?

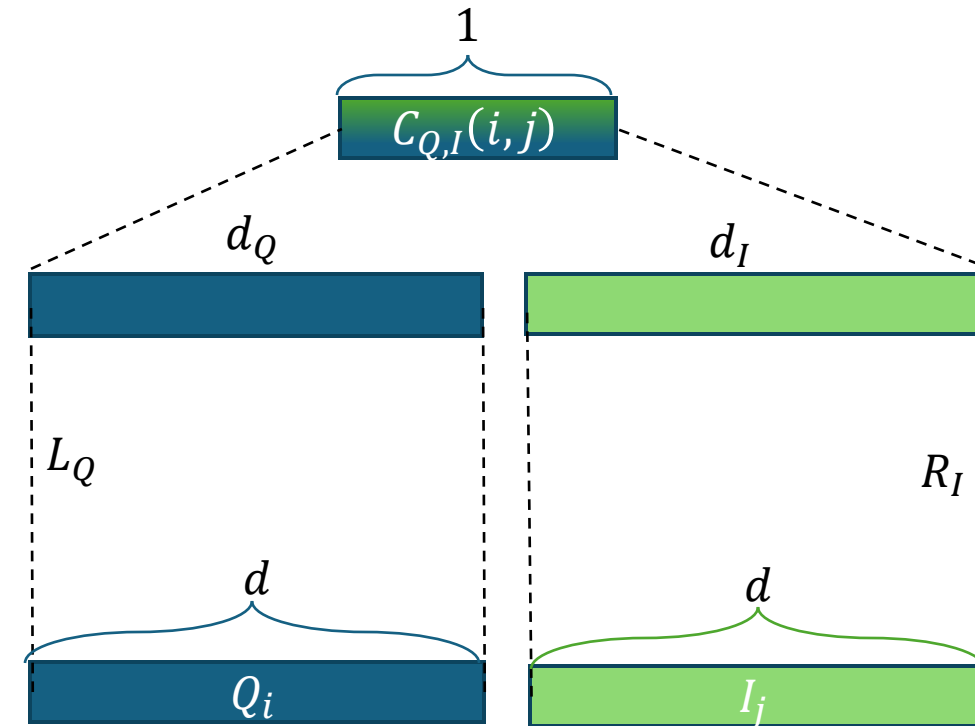
$$b_I(I, Q, i) \propto \exp(w_I \psi_I(i) + w_{I,Q} \mu_{Q \rightarrow I}(i))$$

The interaction is learned

- For each two modalities

$$C_{Q,I}(i,j) = \left(\frac{L_Q Q_i}{\|L_Q Q_i\|} \cdot \frac{R_I I_j}{\|R_I I_j\|} \right)$$

- Where weights $L_q \in \mathbb{R}^{d_Q \times d}$ and $R_i \in \mathbb{R}^{d_I \times d}$



$$b_I(I, Q, i) \propto \exp(w_I \psi_I(i) + w_{I,Q} \mu_{Q \rightarrow I}(i))$$

Pairwise interaction matrix



What animal
is in the cage?

$$C_{Q,I}(i,j) = \left(\frac{L_Q Q_i}{\|L_Q Q_i\|} \cdot \frac{R_I I_j}{\|R_I I_j\|} \right)$$

			
What	1.2	2.3	1.7
animal	-0.4	6.5	4.2
is	1.3	1.2	0.6

$$b_I(I, Q, i) \propto \exp(w_I \psi_I(i) + w_{I,Q} \mu_{Q \rightarrow I}(i))$$

Message from question to image

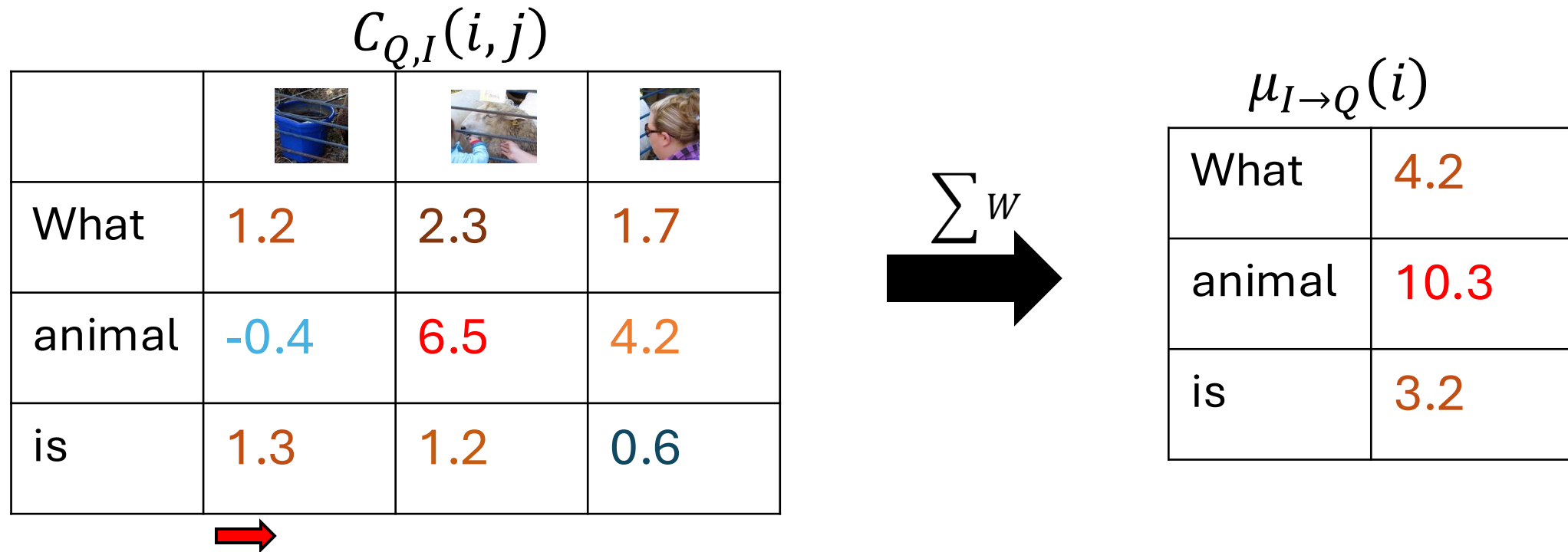
- Messages via weighted marginalization: $\mu_{Q \rightarrow I}(i) = \sum_{Q_j} W_{I,i,j} C_{Q,I}(i,j)$

	$C_{Q,I}(i,j)$				$\mu_{Q \rightarrow I}(i)$	
						2.1
What	1.2	2.3	1.7	$\sum^w$ 		9
animal	-0.4	6.5	4.2			6.5
is	1.3	1.2	0.6			

$$b_I(I, Q, i) \propto \exp(w_I \psi_I(i) + w_{I,Q} \mu_{Q \rightarrow I}(i))$$

Message from image to question

- Messages via weighted marginalization: $\mu_{Q \rightarrow I}(i) = \sum_{I_j} W_{I,i,j} C_{Q,I}(j, i)$



$$b_I(I, Q, i) \propto \exp(w_I \psi_I(i) + w_{I,Q} \mu_{Q \rightarrow I}(i))$$

Message's visualization

- Messages reacts to highly word-correlated regions



What **animal** is in the cage?

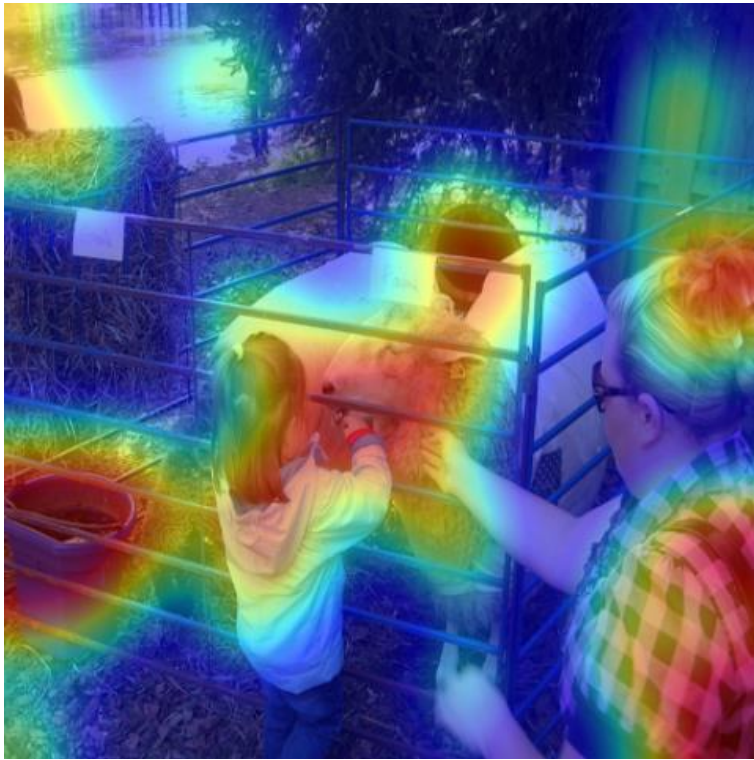


What **color** is the **plastic container** inside the cage?

$$b_I(I, Q, i) \propto \exp(w_I \psi_I(i) + w_{I,Q} \mu_{Q \rightarrow I}(i))$$

Unary potential, pairwise messages and belief

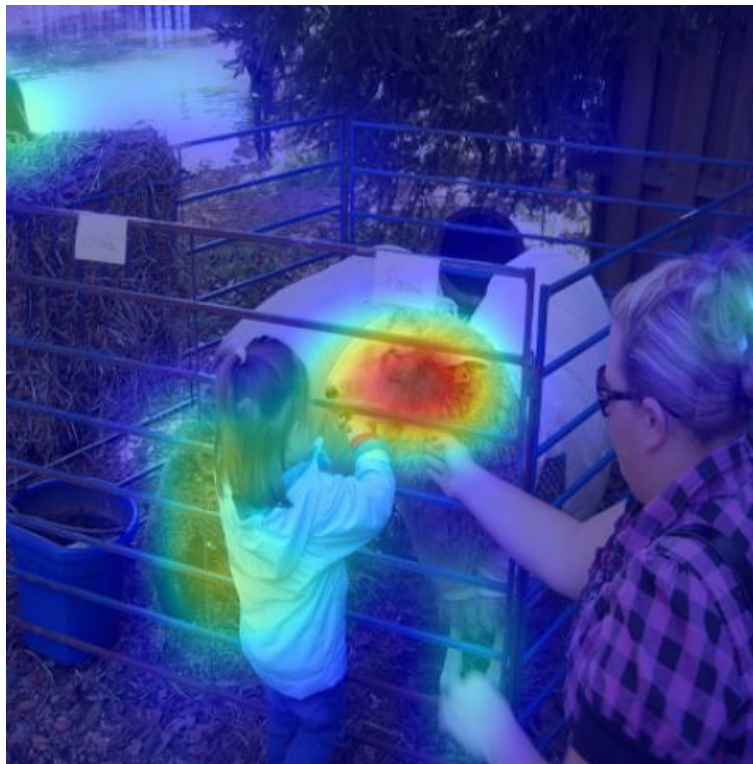
$$\psi_I(i)$$



What **animal** is in the cage?

$$\psi_Q(i)$$

$$\mu_{Q \rightarrow I}(i, j)$$



What **animal** is in the cage?

$$\mu_{I \rightarrow Q}(i, j)$$

$$b_I(i)$$

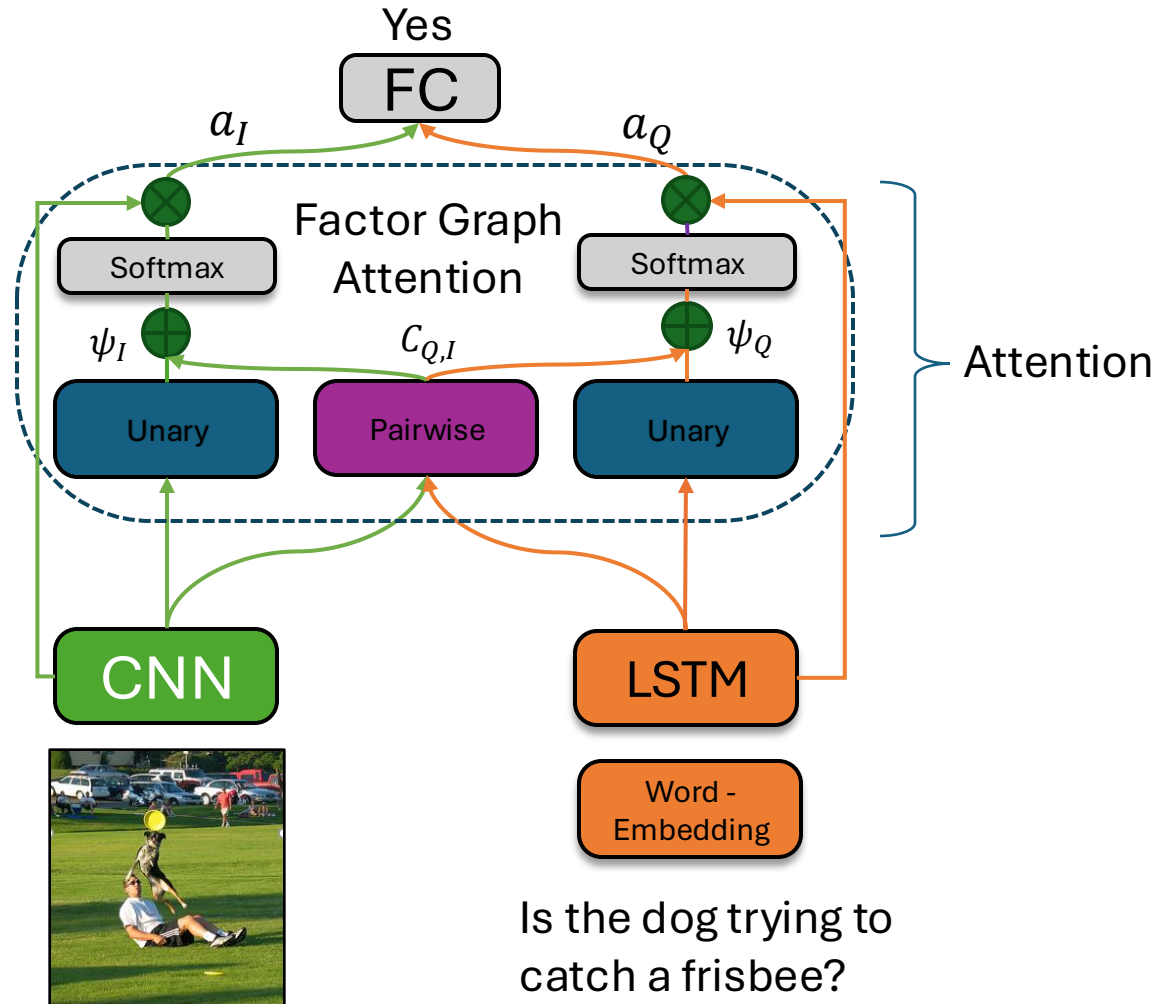


What **animal** is in the cage?

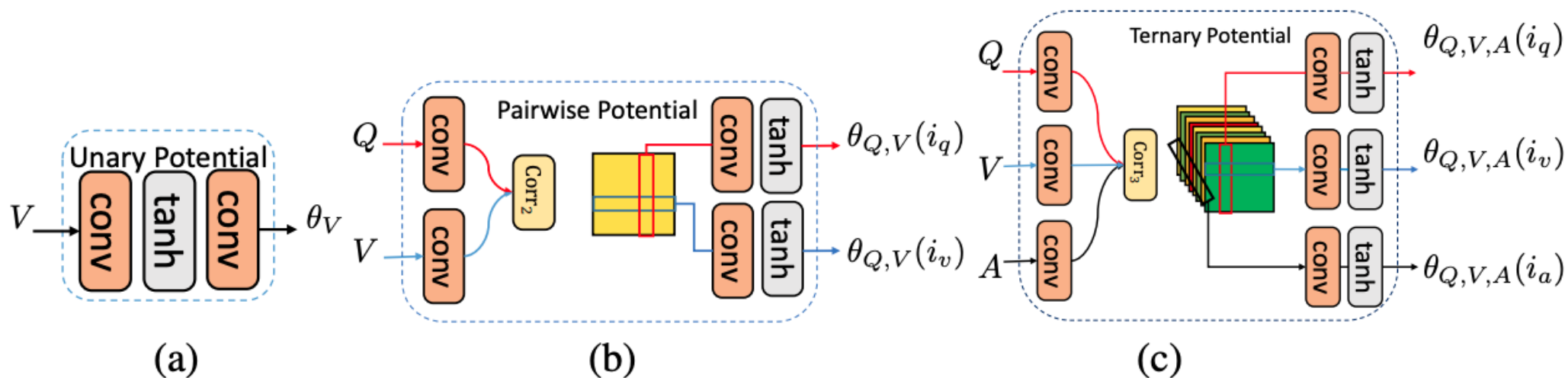
$$b_Q(j)$$

Strong VQA baseline

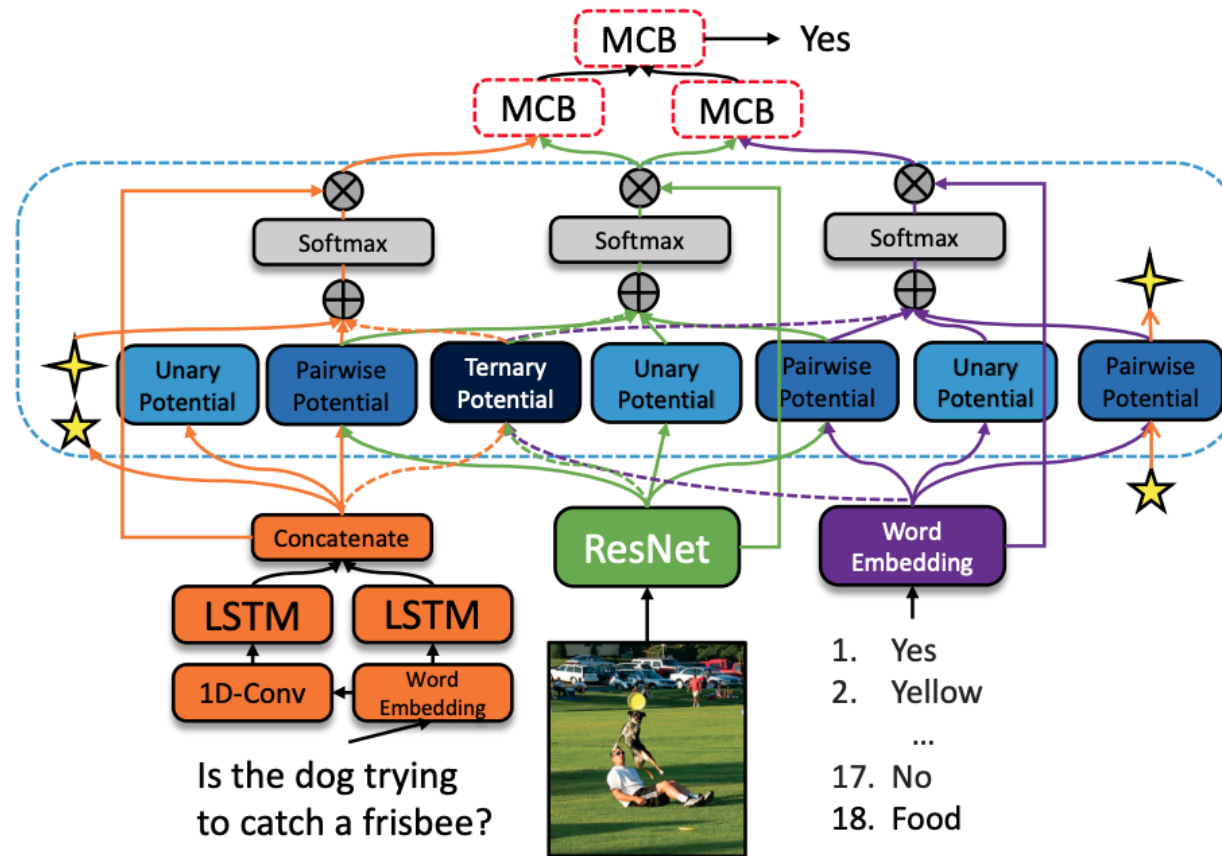
Note:
This is a
classification
networks



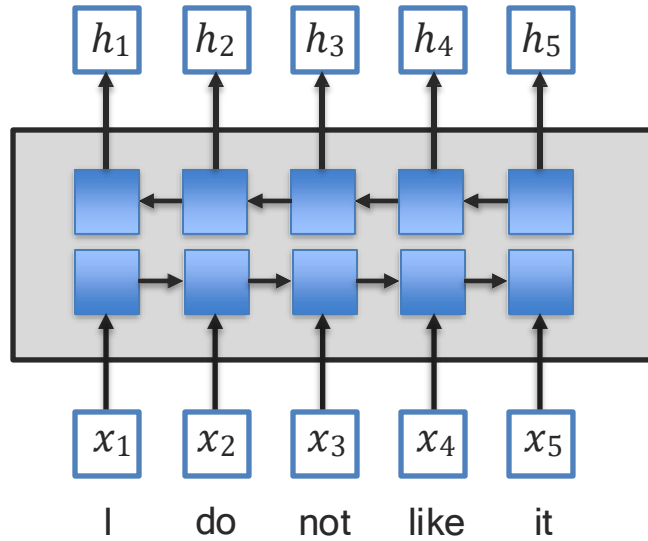
High-order attention belief



State-of-the-art MC-VQA

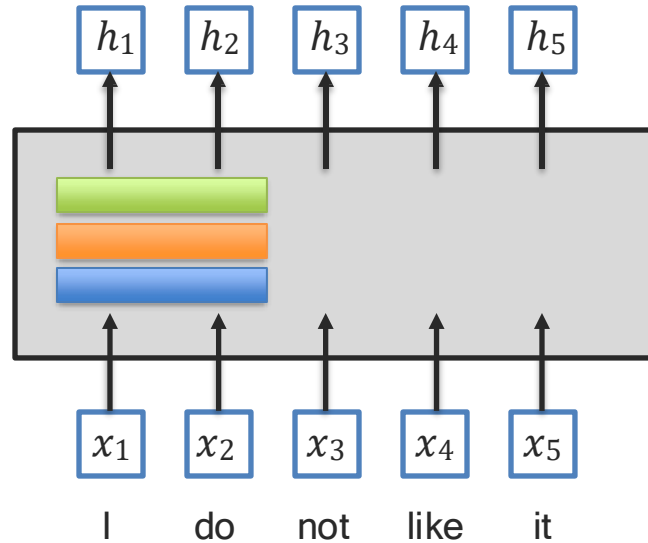


Option 1: Bi-directional LSTM



But harder to parallelize...

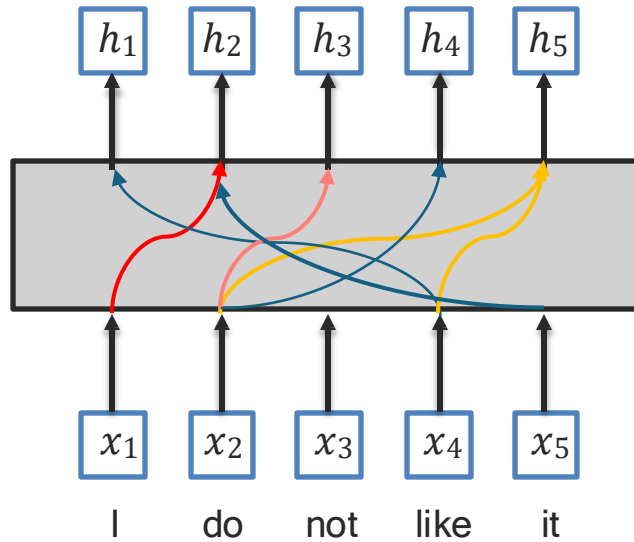
Option 2: Convolutions



Can be parallelized!

But modeling long-range dependencies
require multiple layers

Option 3: Self-attention



Can be parallelized!

Long-range dependencies

Dynamic attention weights

Attention Layer

Inputs:

Query vectors: $\mathbf{Q}$ (Shape: $N_Q \times D_Q$)

Input vectors: $\mathbf{X}$ (Shape: $N_X \times D_X$)

Key matrix: $\mathbf{W}_K$ (Shape: $D_X \times D_Q$)

Value matrix: $\mathbf{W}_V$ (Shape: $D_X \times D_V$)

Computation:

Key vectors: $\mathbf{K} = \mathbf{XW}_K$ (Shape: $N_X \times D_Q$)

Value Vectors: $\mathbf{V} = \mathbf{XW}_V$ (Shape: $N_X \times D_V$)

Similarities: $\mathbf{E} = \mathbf{QK}^T$ (Shape: $N_Q \times N_X$) $E_{i,j} = \mathbf{Q}_i \cdot \mathbf{K}_j / \sqrt{D_Q}$

Attention weights: $\mathbf{A} = \text{softmax}(\mathbf{E}, \text{dim}=1)$ (Shape: $N_Q \times N_X$)

Output vectors: $\mathbf{Y} = \mathbf{AV}$ (Shape: $N_Q \times D_V$) $Y_i = \sum_j A_{i,j} \mathbf{V}_j$

X_1

X_2

X_3

Q_1

Q_2

Q_3

Q_4

Attention Layer

Inputs:

Query vectors: $\mathbf{Q}$ (Shape: $N_Q \times D_Q$)

Input vectors: $\mathbf{X}$ (Shape: $N_X \times D_X$)

Key matrix: $\mathbf{W}_K$ (Shape: $D_X \times D_Q$)

Value matrix: $\mathbf{W}_V$ (Shape: $D_X \times D_V$)

Computation:

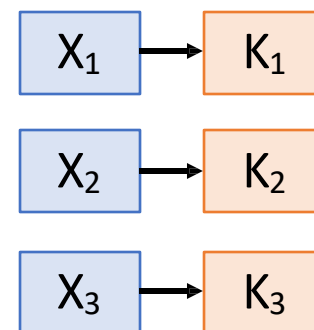
Key vectors: $\mathbf{K} = \mathbf{XW}_K$ (Shape: $N_X \times D_Q$)

Value Vectors: $\mathbf{V} = \mathbf{XW}_V$ (Shape: $N_X \times D_V$)

Similarities: $\mathbf{E} = \mathbf{QK}^T$ (Shape: $N_Q \times N_X$) $E_{i,j} = \mathbf{Q}_i \cdot \mathbf{K}_j / \sqrt{D_Q}$

Attention weights: $\mathbf{A} = \text{softmax}(\mathbf{E}, \text{dim}=1)$ (Shape: $N_Q \times N_X$)

Output vectors: $\mathbf{Y} = \mathbf{AV}$ (Shape: $N_Q \times D_V$) $Y_i = \sum_j A_{i,j} \mathbf{V}_j$



Q_1

Q_2

Q_3

Q_4

Attention Layer

Inputs:

Query vectors: $\mathbf{Q}$ (Shape: $N_Q \times D_Q$)

Input vectors: $\mathbf{X}$ (Shape: $N_X \times D_X$)

Key matrix: $\mathbf{W}_K$ (Shape: $D_X \times D_Q$)

Value matrix: $\mathbf{W}_V$ (Shape: $D_X \times D_V$)

Computation:

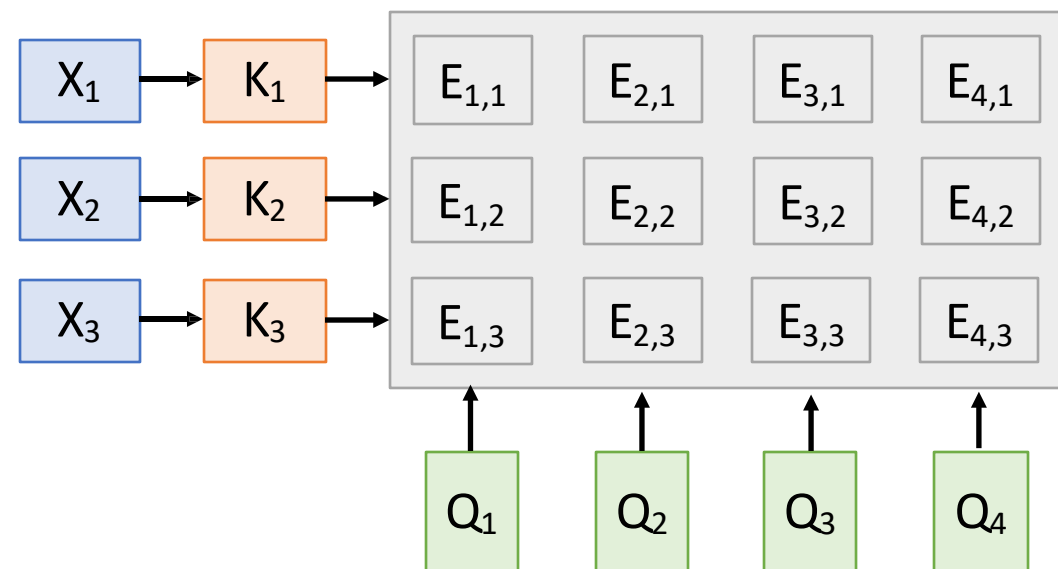
Key vectors: $\mathbf{K} = \mathbf{XW}_K$ (Shape: $N_X \times D_Q$)

Value Vectors: $\mathbf{V} = \mathbf{XW}_V$ (Shape: $N_X \times D_V$)

Similarities: $\mathbf{E} = \mathbf{QK}^T$ (Shape: $N_Q \times N_X$) $E_{i,j} = \mathbf{Q}_i \cdot \mathbf{K}_j / \text{sqrt}(D_Q)$

Attention weights: $\mathbf{A} = \text{softmax}(\mathbf{E}, \text{dim}=1)$ (Shape: $N_Q \times N_X$)

Output vectors: $\mathbf{Y} = \mathbf{AV}$ (Shape: $N_Q \times D_V$) $Y_i = \sum_j A_{i,j} \mathbf{V}_j$



Attention Layer

Inputs:

Query vectors: $\mathbf{Q}$ (Shape: $N_Q \times D_Q$)

Input vectors: $\mathbf{X}$ (Shape: $N_X \times D_X$)

Key matrix: $\mathbf{W}_K$ (Shape: $D_X \times D_Q$)

Value matrix: $\mathbf{W}_V$ (Shape: $D_X \times D_V$)

Computation:

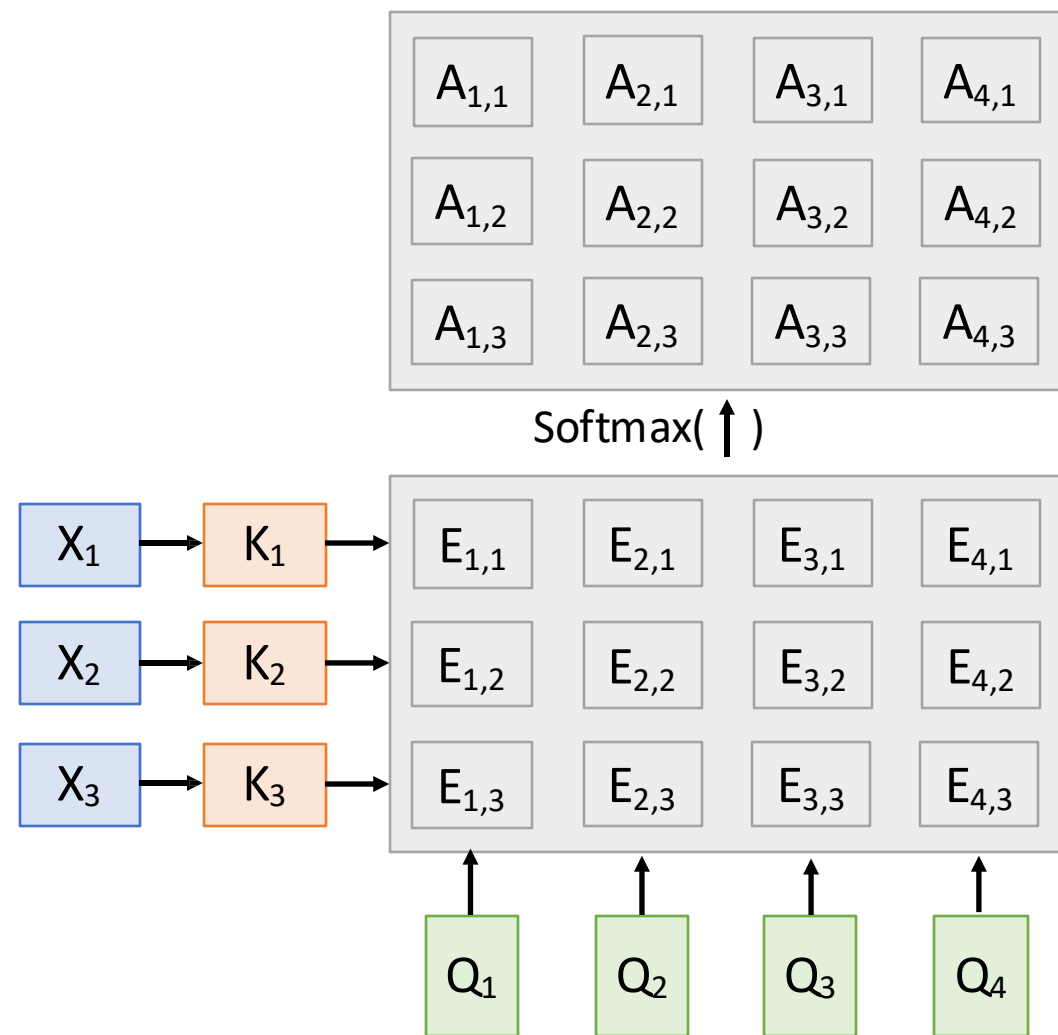
Key vectors: $\mathbf{K} = \mathbf{XW}_K$ (Shape: $N_X \times D_Q$)

Value Vectors: $\mathbf{V} = \mathbf{XW}_V$ (Shape: $N_X \times D_V$)

Similarities: $\mathbf{E} = \mathbf{QK}^T$ (Shape: $N_Q \times N_X$) $E_{i,j} = \mathbf{Q}_i \cdot \mathbf{K}_j / \text{sqrt}(D_Q)$

Attention weights: $\mathbf{A} = \text{softmax}(\mathbf{E}, \text{dim}=1)$ (Shape: $N_Q \times N_X$)

Output vectors: $\mathbf{Y} = \mathbf{AV}$ (Shape: $N_Q \times D_V$) $Y_i = \sum_j A_{i,j} \mathbf{V}_j$



Attention Layer

Inputs:

Query vectors: $\mathbf{Q}$ (Shape: $N_Q \times D_Q$)

Input vectors: $\mathbf{X}$ (Shape: $N_X \times D_X$)

Key matrix: $\mathbf{W}_K$ (Shape: $D_X \times D_Q$)

Value matrix: $\mathbf{W}_V$ (Shape: $D_X \times D_V$)

Computation:

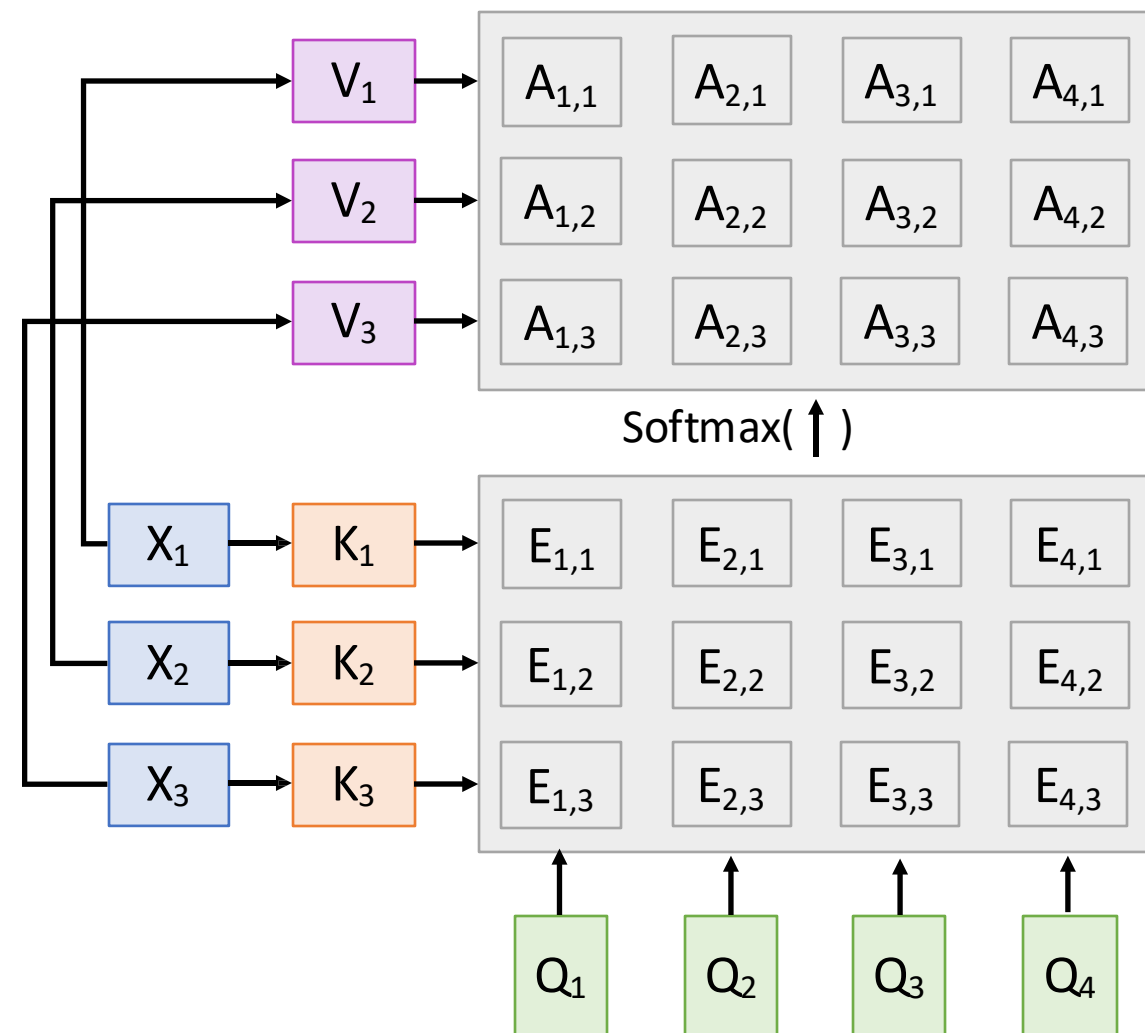
Key vectors: $\mathbf{K} = \mathbf{XW}_K$ (Shape: $N_X \times D_Q$)

Value Vectors: $\mathbf{V} = \mathbf{XW}_V$ (Shape: $N_X \times D_V$)

Similarities: $\mathbf{E} = \mathbf{QK}^T$ (Shape: $N_Q \times N_X$) $E_{i,j} = \mathbf{Q}_i \cdot \mathbf{K}_j / \text{sqrt}(D_Q)$

Attention weights: $\mathbf{A} = \text{softmax}(\mathbf{E}, \text{dim}=1)$ (Shape: $N_Q \times N_X$)

Output vectors: $\mathbf{Y} = \mathbf{AV}$ (Shape: $N_Q \times D_V$) $Y_i = \sum_j A_{i,j} \mathbf{V}_j$



Attention Layer

Inputs:

Query vectors: $\mathbf{Q}$ (Shape: $N_Q \times D_Q$)

Input vectors: $\mathbf{X}$ (Shape: $N_X \times D_X$)

Key matrix: $\mathbf{W}_K$ (Shape: $D_X \times D_Q$)

Value matrix: $\mathbf{W}_V$ (Shape: $D_X \times D_V$)

Computation:

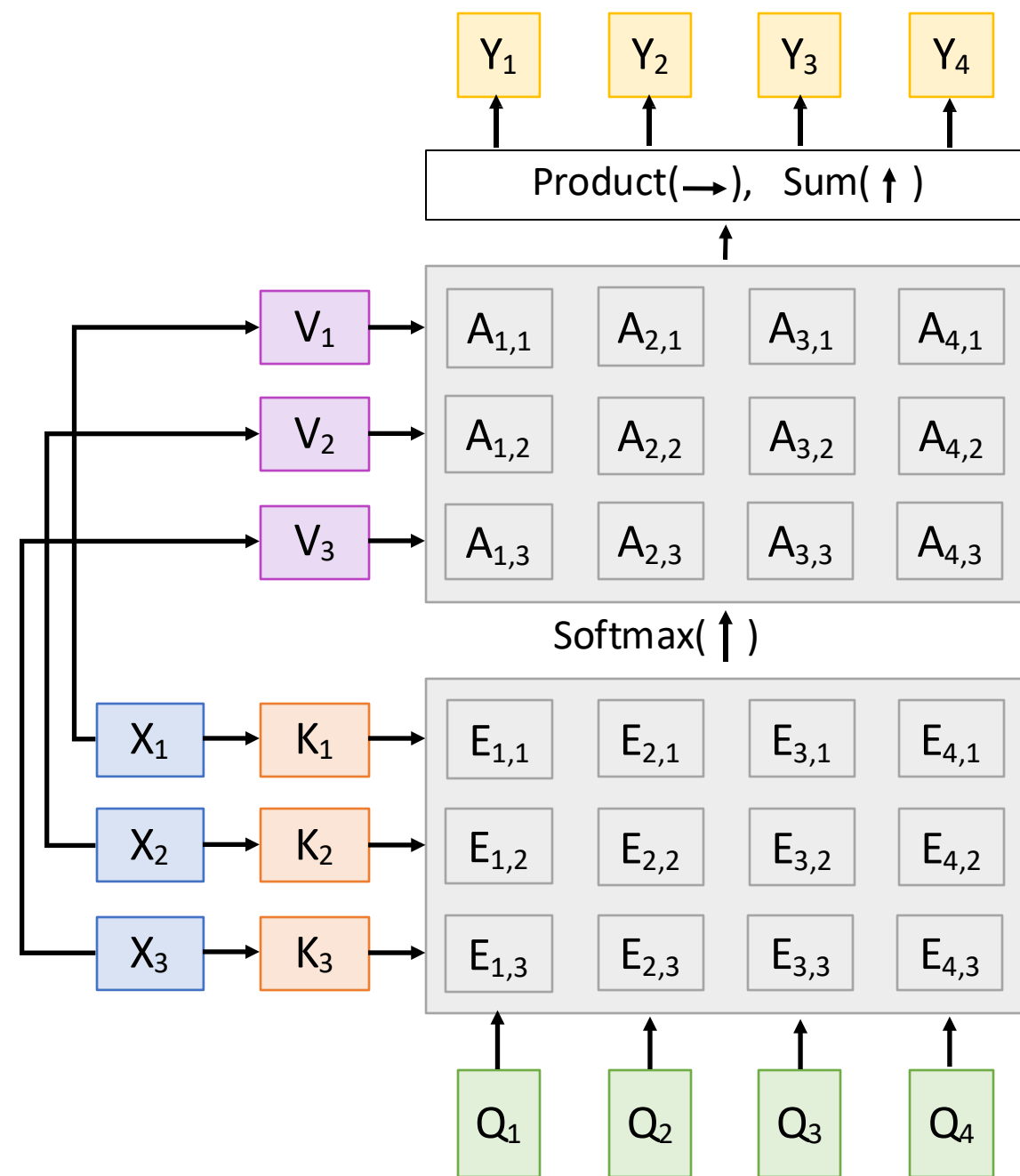
Key vectors: $\mathbf{K} = \mathbf{XW}_K$ (Shape: $N_X \times D_Q$)

Value Vectors: $\mathbf{V} = \mathbf{XW}_V$ (Shape: $N_X \times D_V$)

Similarities: $\mathbf{E} = \mathbf{QK}^T$ (Shape: $N_Q \times N_X$) $E_{i,j} = \mathbf{Q}_i \cdot \mathbf{K}_j / \text{sqrt}(D_Q)$

Attention weights: $\mathbf{A} = \text{softmax}(\mathbf{E}, \text{dim}=1)$ (Shape: $N_Q \times N_X$)

Output vectors: $\mathbf{Y} = \mathbf{AV}$ (Shape: $N_Q \times D_V$) $Y_i = \sum_j A_{i,j} \mathbf{V}_j$



Self-Attention Layer

One **query** per **input vector**

- Inputs:
- Input vectors: X (Shape: $N_x \times D_x$) Key matrix: W_K (Shape: $D_x \times D_Q$) Value matrix: W_V (Shape: $D_x \times D_V$) Query matrix: W_Q (Shape: $D_x \times D_Q$)
- Computation:
- Query vectors: $Q = XW_Q$
- Key vectors: $K = XW_K$ (Shape: $N_x \times D_Q$)
- Value Vectors: $V = XW_V$ (Shape: $N_x \times D_V$)
- Similarities: $E = QK^T$ (Shape: $N_x \times N_x$) $E_{i,j} = Q_i \cdot K_j / \text{sqrt}(D_Q)$
Attention weights: $A = \text{softmax}(E, \text{dim}=1)$ (Shape: $N_x \times N_x$) Output vectors: $Y = AV$ (Shape: $N_x \times D_V$) $Y_i = \sum_j A_{i,j} V_j$

X_1

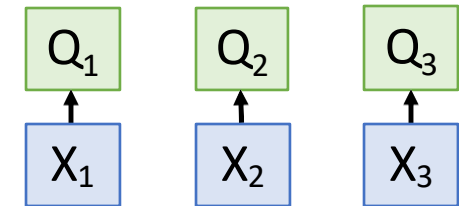
X_2

X_3

Self-Attention Layer

One **query** per **input vector**

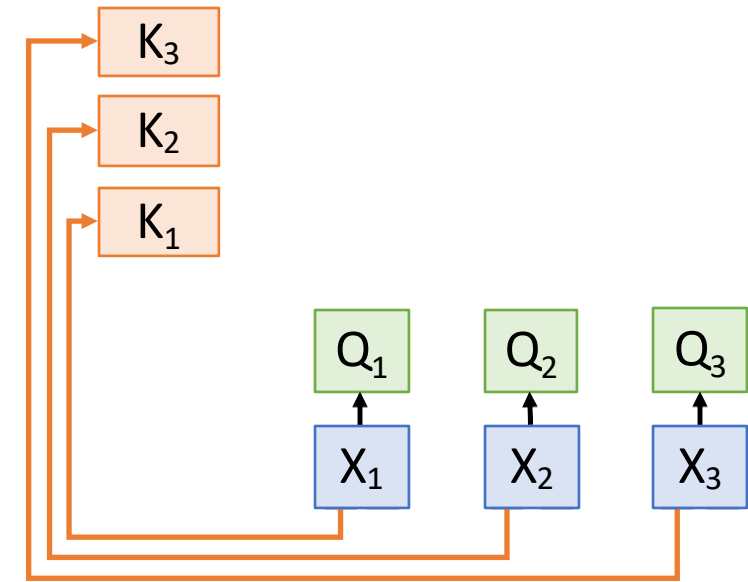
- Inputs:
- Input vectors: X (Shape: $N_x \times D_x$) Key matrix: W_K (Shape: $D_x \times D_Q$) Value matrix: W_V (Shape: $D_x \times D_V$) Query matrix: W_Q (Shape: $D_x \times D_Q$)
- Computation:
- Query vectors: $Q = XW_Q$
- Key vectors: $K = XW_K$ (Shape: $N_x \times D_Q$)
- Value Vectors: $V = XW_V$ (Shape: $N_x \times D_V$)
- Similarities: $E = QK^T$ (Shape: $N_x \times N_x$) $E_{i,j} = Q_i \cdot K_j / \text{sqrt}(D_Q)$
Attention weights: $A = \text{softmax}(E, \text{dim}=1)$ (Shape: $N_x \times N_x$) Output vectors: $Y = AV$ (Shape: $N_x \times D_V$) $Y_i = \sum_j A_{i,j} V_j$



Self-Attention Layer

One **query** per **input vector**

- Inputs:
- Input vectors: X (Shape: $N_x \times D_x$) Key matrix: W_K (Shape: $D_x \times D_Q$) Value matrix: W_V (Shape: $D_x \times D_V$) Query matrix: W_Q (Shape: $D_x \times D_Q$)
- Computation:
- Query vectors: $Q = XW_Q$
- Key vectors: $K = XW_K$ (Shape: $N_x \times D_Q$)
- Value Vectors: $V = XW_V$ (Shape: $N_x \times D_V$)
- Similarities: $E = QK^T$ (Shape: $N_x \times N_x$) $E_{i,j} = Q_i \cdot K_j / \text{sqrt}(D_Q)$
Attention weights: $A = \text{softmax}(E, \text{dim}=1)$ (Shape: $N_x \times N_x$) Output vectors: $Y = AV$ (Shape: $N_x \times D_V$) $Y_i = \sum_j A_{i,j} V_j$



Self-Attention Layer

One **query** per **input vector**

- Inputs:

- Input vectors: X (Shape: $N_x \times D_x$) Key matrix: W_K (Shape: $D_x \times D_Q$) Value matrix: W_V (Shape: $D_x \times D_V$) Query matrix: W_Q (Shape: $D_x \times D_Q$)

- Computation:

- Query vectors: $Q = XW_Q$

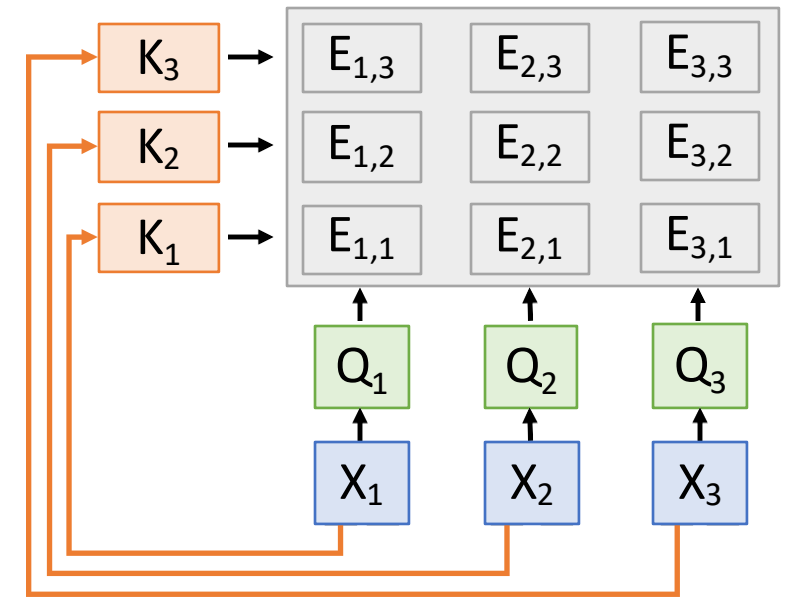
- Key vectors: $K = XW_K$ (Shape: $N_x \times D_Q$)

- Value Vectors: $V = XW_V$ (Shape: $N_x \times D_V$)

- Similarities: $E = QK^T$ (Shape: $N_x \times N_x$) $E_{i,j} = Q_i \cdot K_j / \text{sqrt}(D_Q)$

Attention weights: $A = \text{softmax}(E, \text{dim}=1)$ (Shape: $N_x \times N_x$) Output vectors:

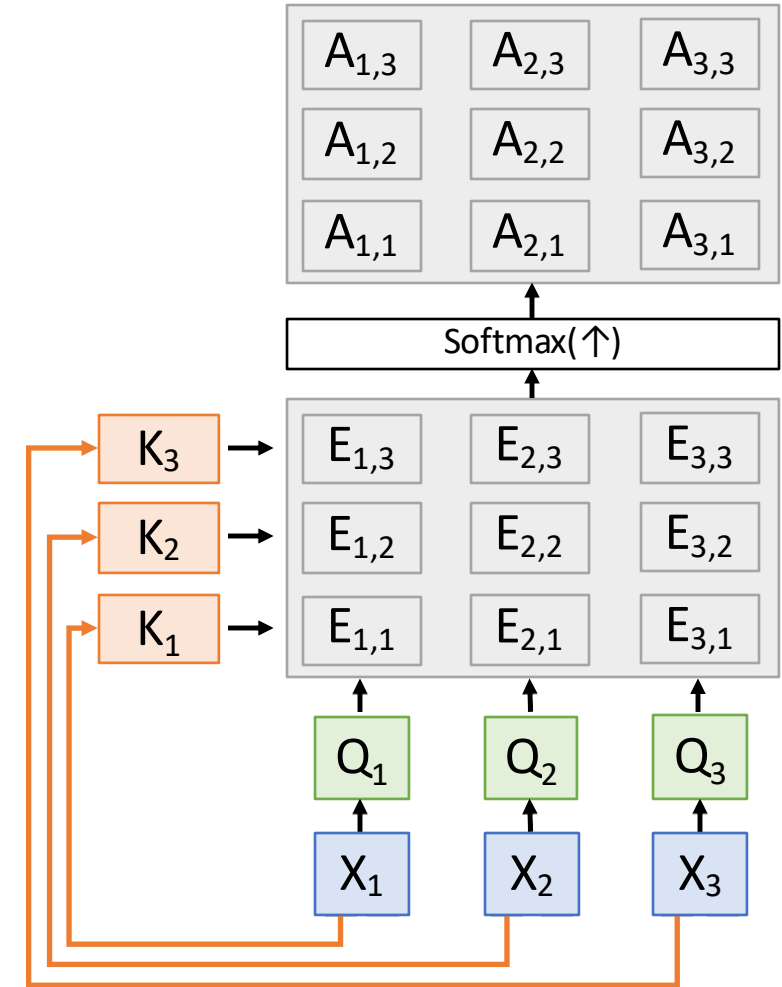
$Y = AV$ (Shape: $N_x \times D_V$) $Y_i = \sum_j A_{i,j} V_j$



Self-Attention Layer

One **query** per **input vector**

- Inputs:
- Input vectors: X (Shape: $N_x \times D_x$) Key matrix: W_K (Shape: $D_x \times D_Q$) Value matrix: W_V (Shape: $D_x \times D_V$) Query matrix: W_Q (Shape: $D_x \times D_Q$)
- Computation:
- Query vectors: $Q = XW_Q$
- Key vectors: $K = XW_K$ (Shape: $N_x \times D_Q$)
- Value Vectors: $V = XW_V$ (Shape: $N_x \times D_V$)
- Similarities: $E = QK^T$ (Shape: $N_x \times N_x$) $E_{i,j} = Q_i \cdot K_j / \text{sqrt}(D_Q)$
Attention weights: $A = \text{softmax}(E, \text{dim}=1)$ (Shape: $N_x \times N_x$) Output vectors: $Y = AV$ (Shape: $N_x \times D_V$) $Y_i = \sum_j A_{i,j} V_j$



Self-Attention Layer

One **query** per **input vector**

- Inputs:

- Input vectors: X (Shape: $N_x \times D_x$) Key matrix: W_K (Shape: $D_x \times D_Q$) Value matrix: W_V (Shape: $D_x \times D_V$) Query matrix: W_Q (Shape: $D_x \times D_Q$)

- Computation:

- Query vectors: $Q = XW_Q$

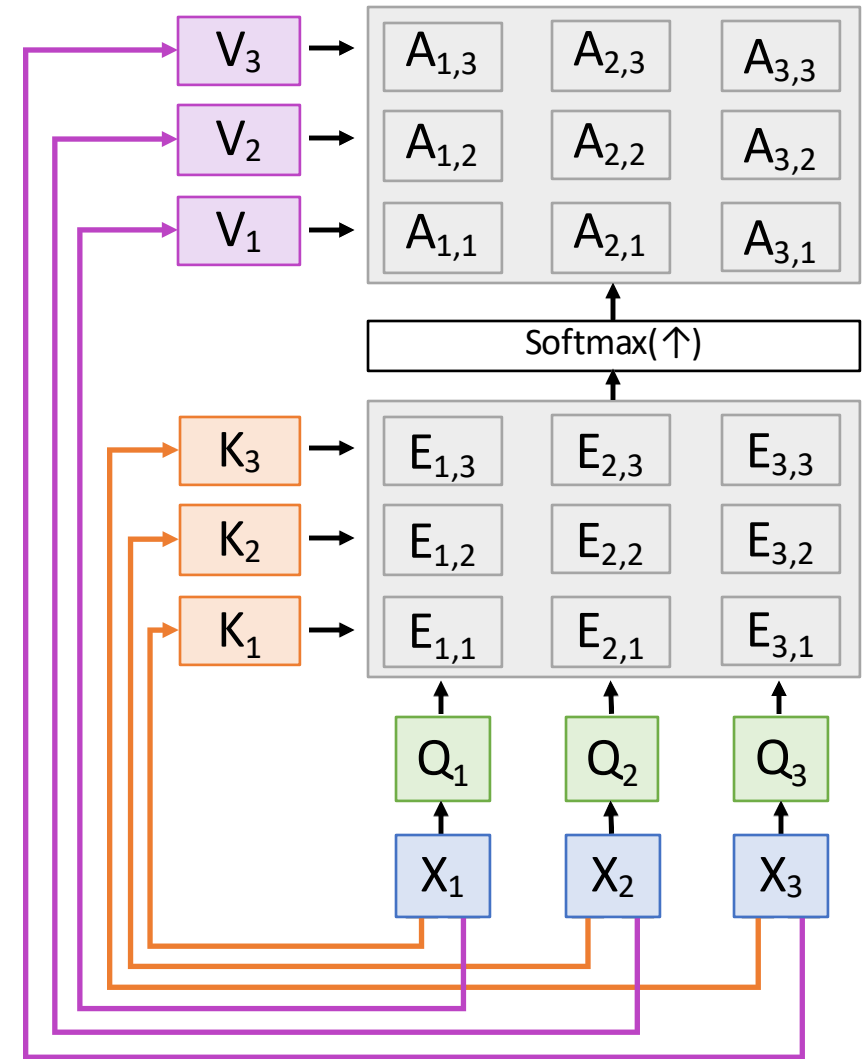
- Key vectors: $K = XW_K$ (Shape: $N_x \times D_Q$)

- Value Vectors: $V = XW_V$ (Shape: $N_x \times D_V$)

- Similarities: $E = QK^T$ (Shape: $N_x \times N_x$) $E_{i,j} = Q_i \cdot K_j / \text{sqrt}(D_Q)$

Attention weights: $A = \text{softmax}(E, \text{dim}=1)$ (Shape: $N_x \times N_x$) Output vectors:

$Y = AV$ (Shape: $N_x \times D_V$) $Y_i = \sum_j A_{i,j} V_j$



Self-Attention Layer

One **query** per **input vector**

- Inputs:

- Input vectors: X (Shape: $N_x \times D_x$) Key matrix: W_K (Shape: $D_x \times D_Q$) Value matrix: W_V (Shape: $D_x \times D_V$) Query matrix: W_Q (Shape: $D_x \times D_Q$)

- Computation:

- Query vectors: $Q = XW_Q$

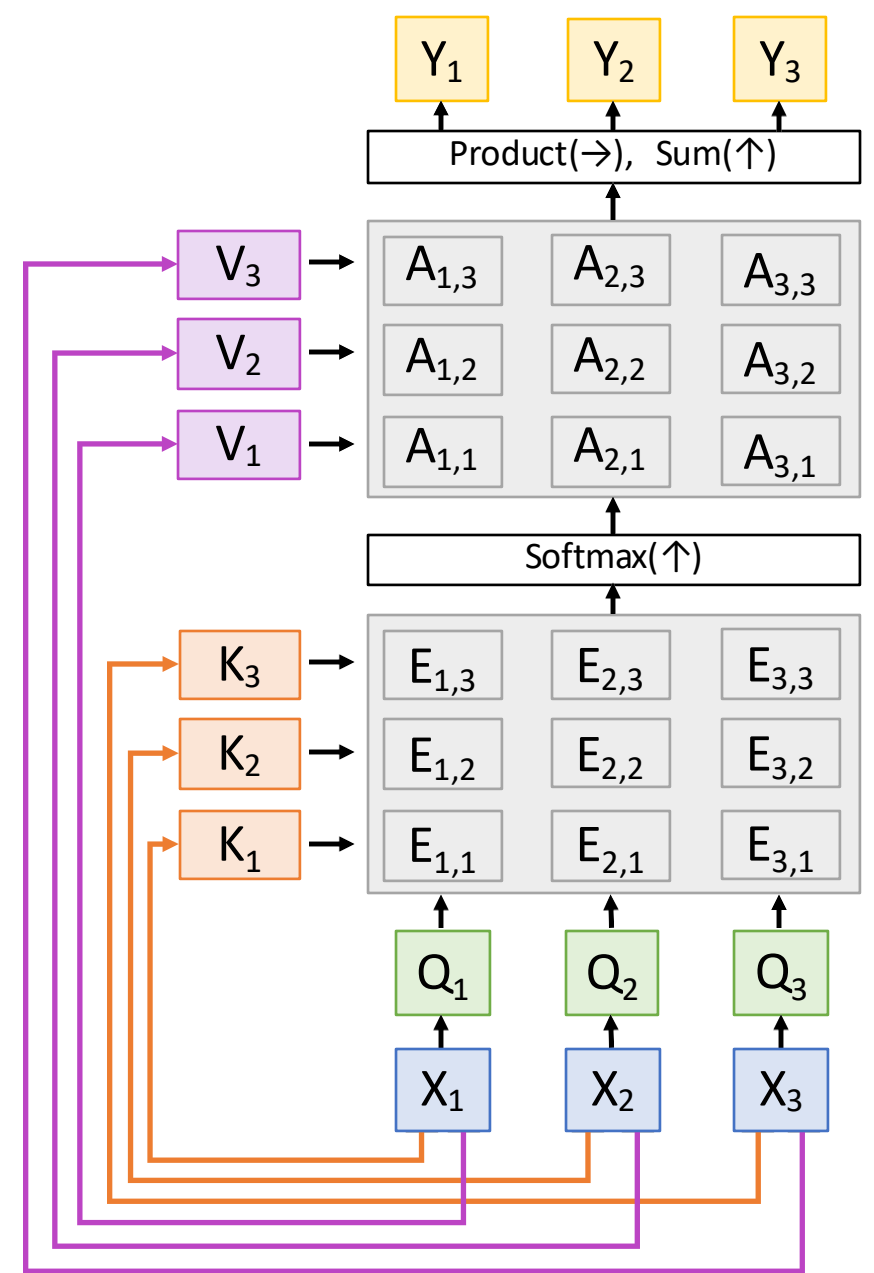
- Key vectors: $K = XW_K$ (Shape: $N_x \times D_Q$)

- Value Vectors: $V = XW_V$ (Shape: $N_x \times D_V$)

- Similarities: $E = QK^T$ (Shape: $N_x \times N_x$) $E_{i,j} = Q_i \cdot K_j / \text{sqrt}(D_Q)$

Attention weights: $A = \text{softmax}(E, \text{dim}=1)$ (Shape: $N_x \times N_x$) Output vectors:

$Y = AV$ (Shape: $N_x \times D_V$) $Y_i = \sum_j A_{i,j} V_j$



Self-Attention Layer

Inputs:

Input vectors: $\mathbf{X}$ (Shape: $N_X \times D_X$)

Key matrix: $\mathbf{W}_K$ (Shape: $D_X \times D_Q$)

Value matrix: $\mathbf{W}_V$ (Shape: $D_X \times D_V$)

Query matrix: $\mathbf{W}_Q$ (Shape: $D_X \times D_Q$)

Computation:

Query vectors: $\mathbf{Q} = \mathbf{XW}_Q$

Key vectors: $\mathbf{K} = \mathbf{XW}_K$ (Shape: $N_X \times D_Q$)

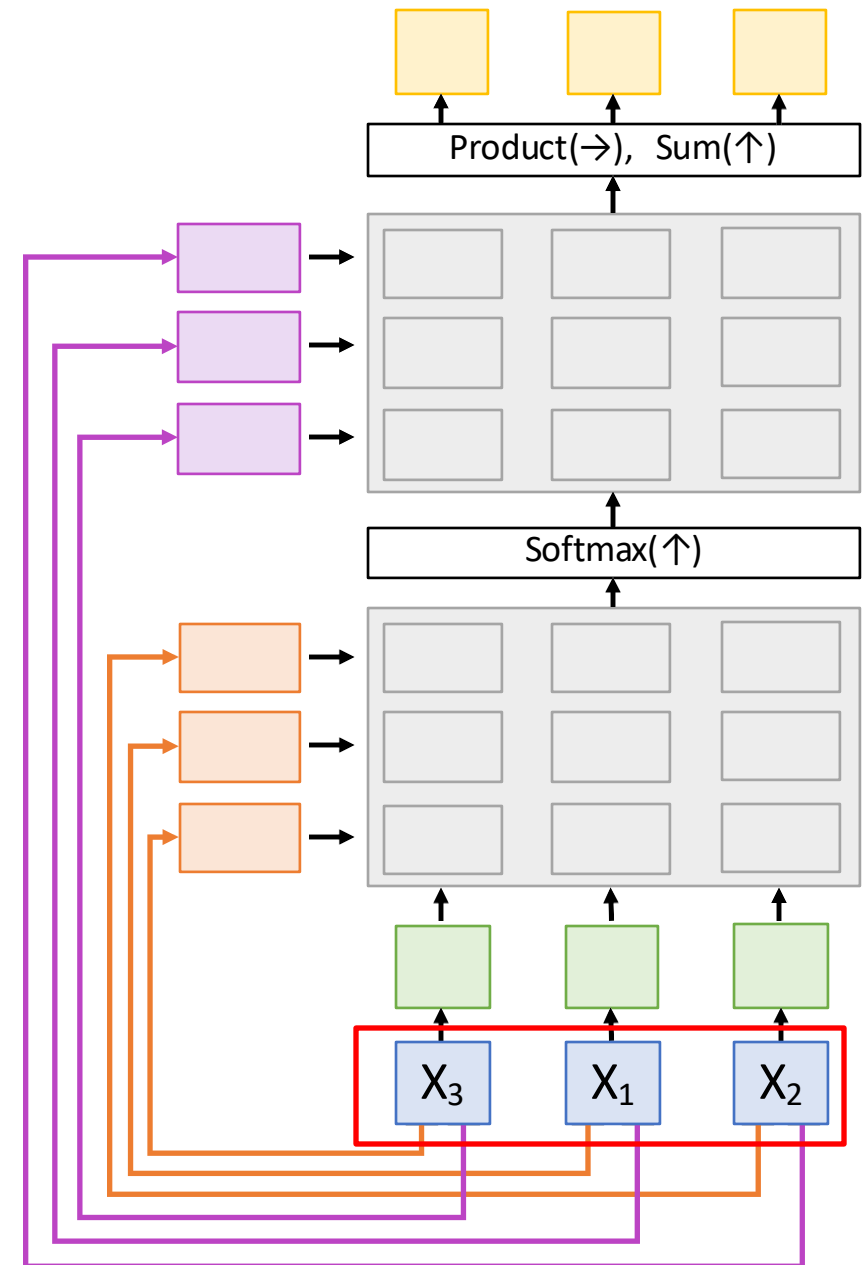
Value Vectors: $\mathbf{V} = \mathbf{XW}_V$ (Shape: $N_X \times D_V$)

Similarities: $\mathbf{E} = \mathbf{QK}^T$ (Shape: $N_X \times N_X$) $E_{i,j} = \mathbf{Q}_i \cdot \mathbf{K}_j / \text{sqrt}(D_Q)$

Attention weights: $\mathbf{A} = \text{softmax}(\mathbf{E}, \text{dim}=1)$ (Shape: $N_X \times N_X$)

Output vectors: $\mathbf{Y} = \mathbf{AV}$ (Shape: $N_X \times D_V$) $Y_i = \sum_j A_{i,j} \mathbf{V}_j$

Consider **permuting**
the input vectors:



Self-Attention Layer

Inputs:

Input vectors: $\mathbf{X}$ (Shape: $N_X \times D_X$)

Key matrix: $\mathbf{W}_K$ (Shape: $D_X \times D_Q$)

Value matrix: $\mathbf{W}_V$ (Shape: $D_X \times D_V$)

Query matrix: $\mathbf{W}_Q$ (Shape: $D_X \times D_Q$)

Computation:

Query vectors: $\mathbf{Q} = \mathbf{XW}_Q$

Key vectors: $\mathbf{K} = \mathbf{XW}_K$ (Shape: $N_X \times D_Q$)

Value Vectors: $\mathbf{V} = \mathbf{XW}_V$ (Shape: $N_X \times D_V$)

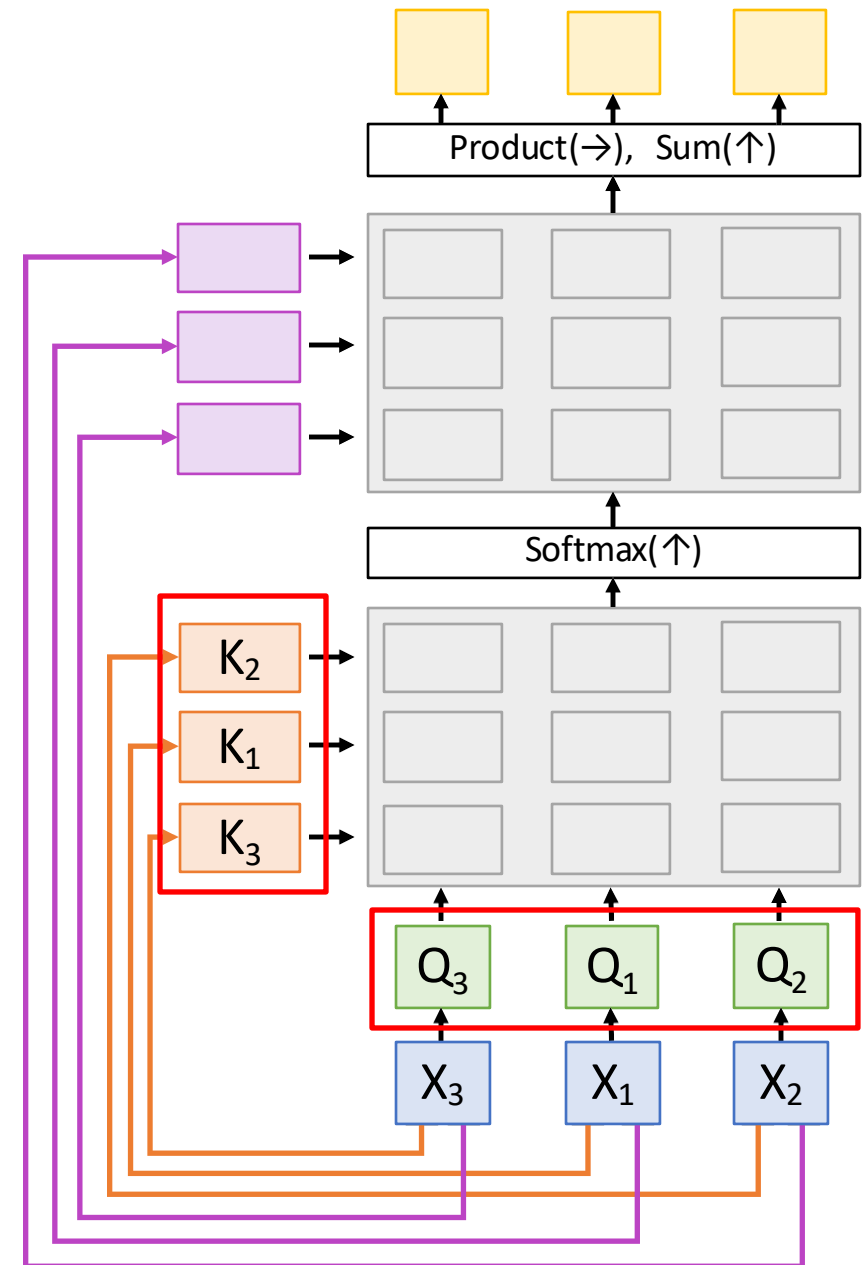
Similarities: $\mathbf{E} = \mathbf{QK}^T$ (Shape: $N_X \times N_X$) $E_{i,j} = \mathbf{Q}_i \cdot \mathbf{K}_j / \text{sqrt}(D_Q)$

Attention weights: $\mathbf{A} = \text{softmax}(\mathbf{E}, \text{dim}=1)$ (Shape: $N_X \times N_X$)

Output vectors: $\mathbf{Y} = \mathbf{AV}$ (Shape: $N_X \times D_V$) $Y_i = \sum_j A_{i,j} \mathbf{V}_j$

Consider **permuting**
the input vectors:

Queries and Keys will be
the same, but permuted



Self-Attention Layer

Inputs:

Input vectors: $\mathbf{X}$ (Shape: $N_X \times D_X$)

Key matrix: $\mathbf{W}_K$ (Shape: $D_X \times D_Q$)

Value matrix: $\mathbf{W}_V$ (Shape: $D_X \times D_V$)

Query matrix: $\mathbf{W}_Q$ (Shape: $D_X \times D_Q$)

Computation:

Query vectors: $\mathbf{Q} = \mathbf{XW}_Q$

Key vectors: $\mathbf{K} = \mathbf{XW}_K$ (Shape: $N_X \times D_Q$)

Value Vectors: $\mathbf{V} = \mathbf{XW}_V$ (Shape: $N_X \times D_V$)

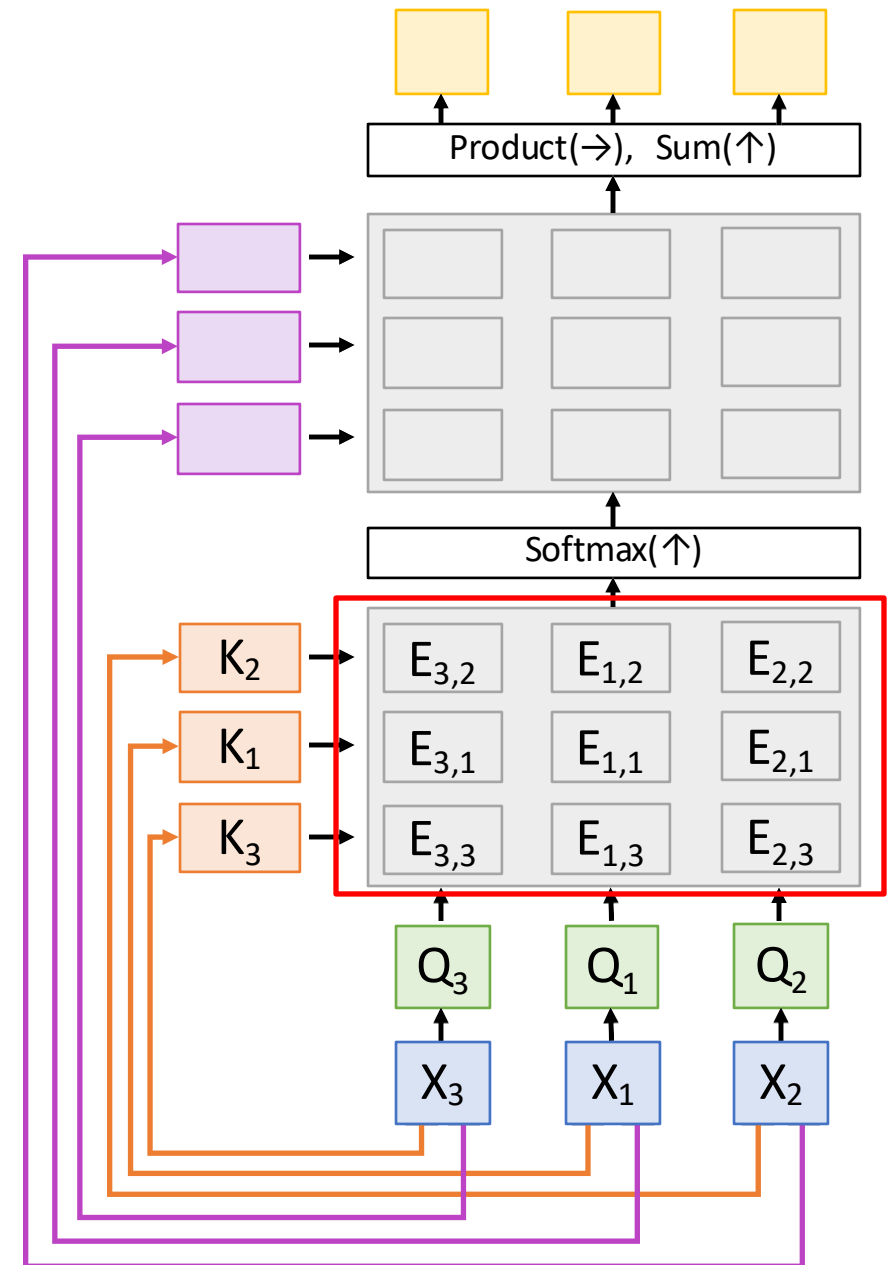
Similarities: $\mathbf{E} = \mathbf{QK}^T$ (Shape: $N_X \times N_X$) $E_{i,j} = \mathbf{Q}_i \cdot \mathbf{K}_j / \text{sqrt}(D_Q)$

Attention weights: $\mathbf{A} = \text{softmax}(\mathbf{E}, \text{dim}=1)$ (Shape: $N_X \times N_X$)

Output vectors: $\mathbf{Y} = \mathbf{AV}$ (Shape: $N_X \times D_V$) $Y_i = \sum_j A_{i,j} \mathbf{V}_j$

Consider **permuting**
the input vectors:

Similarities will be the
same, but permuted



Self-Attention Layer

Inputs:

Input vectors: $\mathbf{X}$ (Shape: $N_X \times D_X$)

Key matrix: $\mathbf{W}_K$ (Shape: $D_X \times D_Q$)

Value matrix: $\mathbf{W}_V$ (Shape: $D_X \times D_V$)

Query matrix: $\mathbf{W}_Q$ (Shape: $D_X \times D_Q$)

Computation:

Query vectors: $\mathbf{Q} = \mathbf{XW}_Q$

Key vectors: $\mathbf{K} = \mathbf{XW}_K$ (Shape: $N_X \times D_Q$)

Value Vectors: $\mathbf{V} = \mathbf{XW}_V$ (Shape: $N_X \times D_V$)

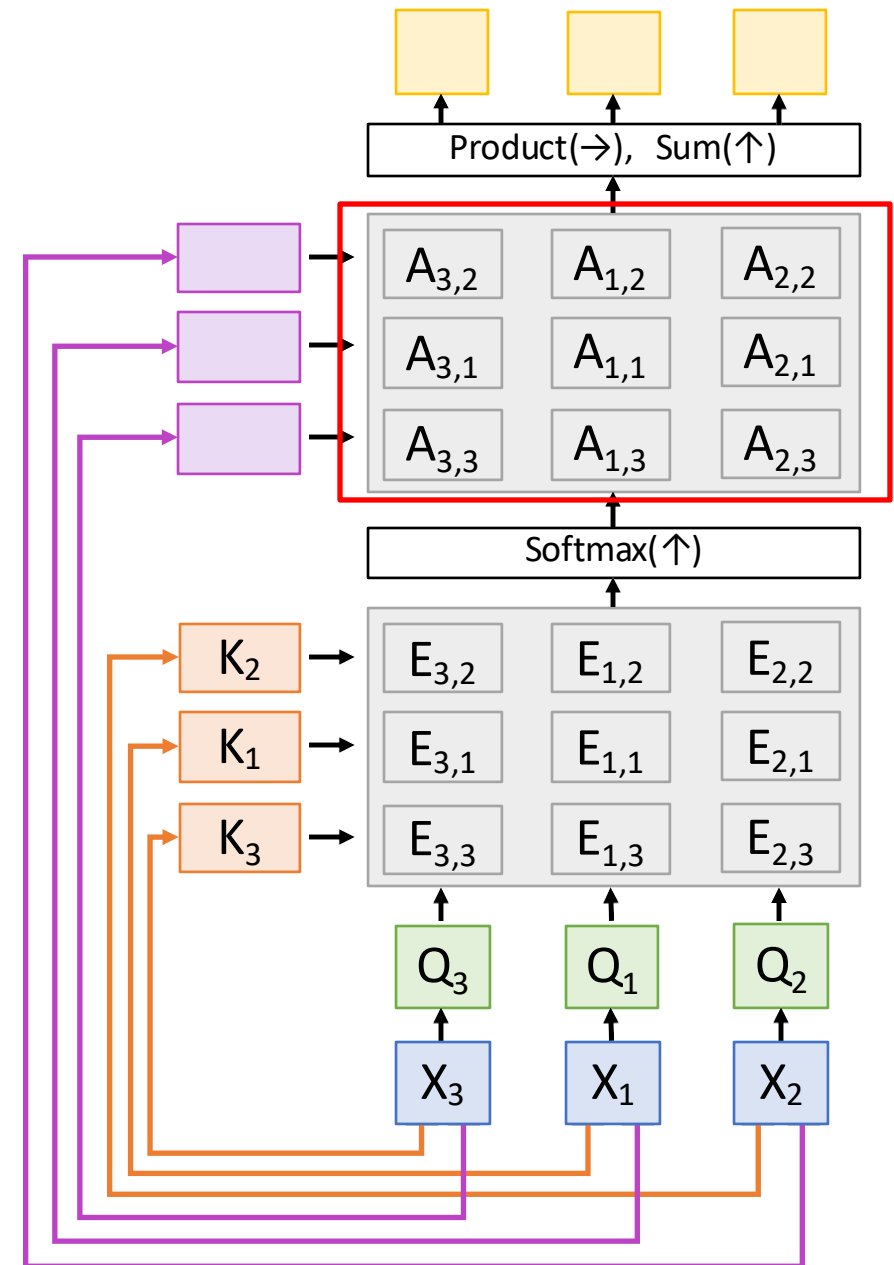
Similarities: $\mathbf{E} = \mathbf{QK}^T$ (Shape: $N_X \times N_X$) $E_{i,j} = \mathbf{Q}_i \cdot \mathbf{K}_j / \text{sqrt}(D_Q)$

Attention weights: $\mathbf{A} = \text{softmax}(\mathbf{E}, \text{dim}=1)$ (Shape: $N_X \times N_X$)

Output vectors: $\mathbf{Y} = \mathbf{AV}$ (Shape: $N_X \times D_V$) $Y_i = \sum_j A_{i,j} \mathbf{V}_j$

Consider **permuting**
the input vectors:

Attention weights will be
the same, but permuted



Self-Attention Layer

Inputs:

Input vectors: $\mathbf{X}$ (Shape: $N_X \times D_X$)

Key matrix: $\mathbf{W}_K$ (Shape: $D_X \times D_Q$)

Value matrix: $\mathbf{W}_V$ (Shape: $D_X \times D_V$)

Query matrix: $\mathbf{W}_Q$ (Shape: $D_X \times D_Q$)

Computation:

Query vectors: $\mathbf{Q} = \mathbf{XW}_Q$

Key vectors: $\mathbf{K} = \mathbf{XW}_K$ (Shape: $N_X \times D_Q$)

Value Vectors: $\mathbf{V} = \mathbf{XW}_V$ (Shape: $N_X \times D_V$)

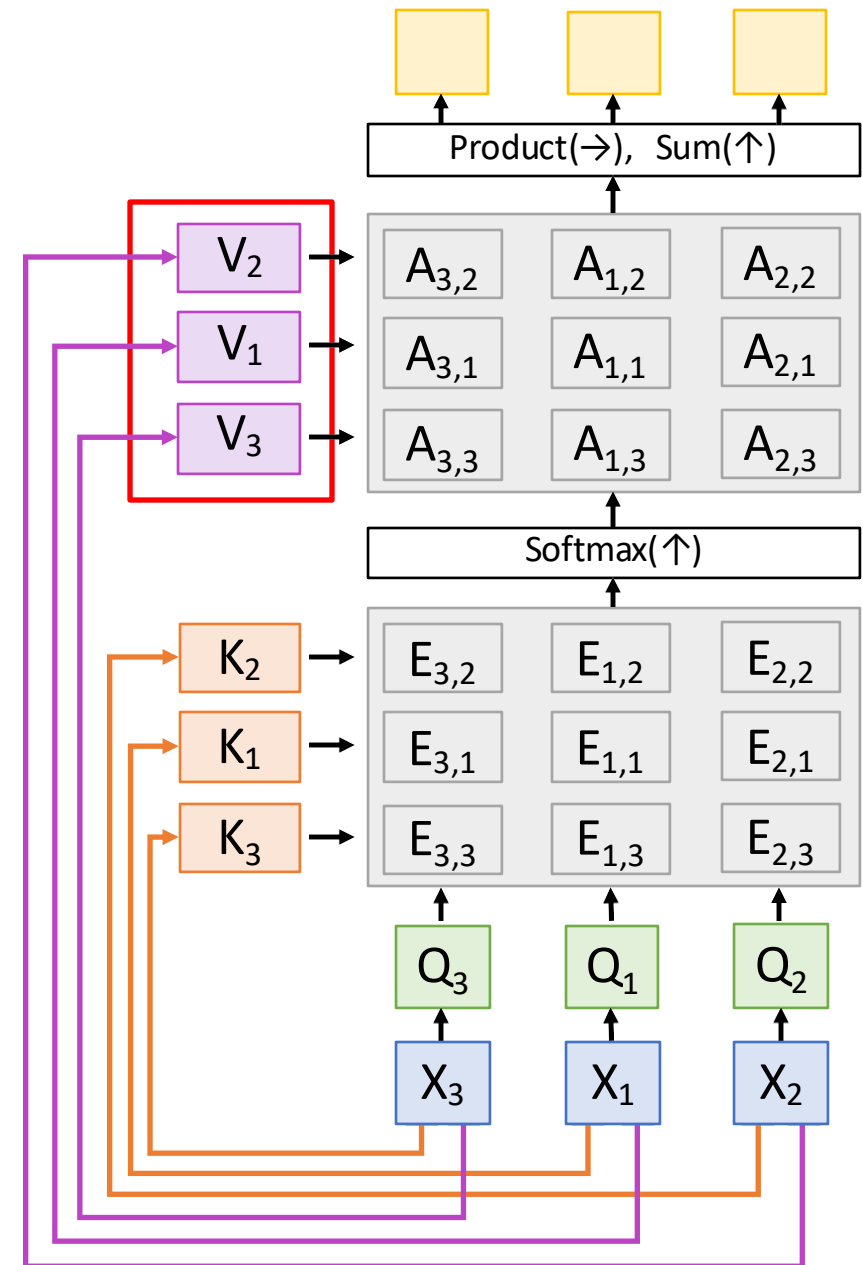
Similarities: $\mathbf{E} = \mathbf{QK}^T$ (Shape: $N_X \times N_X$) $E_{i,j} = \mathbf{Q}_i \cdot \mathbf{K}_j / \text{sqrt}(D_Q)$

Attention weights: $\mathbf{A} = \text{softmax}(\mathbf{E}, \text{dim}=1)$ (Shape: $N_X \times N_X$)

Output vectors: $\mathbf{Y} = \mathbf{AV}$ (Shape: $N_X \times D_V$) $Y_i = \sum_j A_{i,j} \mathbf{V}_j$

Consider **permuting**
the input vectors:

Values will be the
same, but permuted



Self-Attention Layer

Inputs:

Input vectors: $\mathbf{X}$ (Shape: $N_X \times D_X$)

Key matrix: $\mathbf{W}_K$ (Shape: $D_X \times D_Q$)

Value matrix: $\mathbf{W}_V$ (Shape: $D_X \times D_V$)

Query matrix: $\mathbf{W}_Q$ (Shape: $D_X \times D_Q$)

Computation:

Query vectors: $\mathbf{Q} = \mathbf{XW}_Q$

Key vectors: $\mathbf{K} = \mathbf{XW}_K$ (Shape: $N_X \times D_Q$)

Value Vectors: $\mathbf{V} = \mathbf{XW}_V$ (Shape: $N_X \times D_V$)

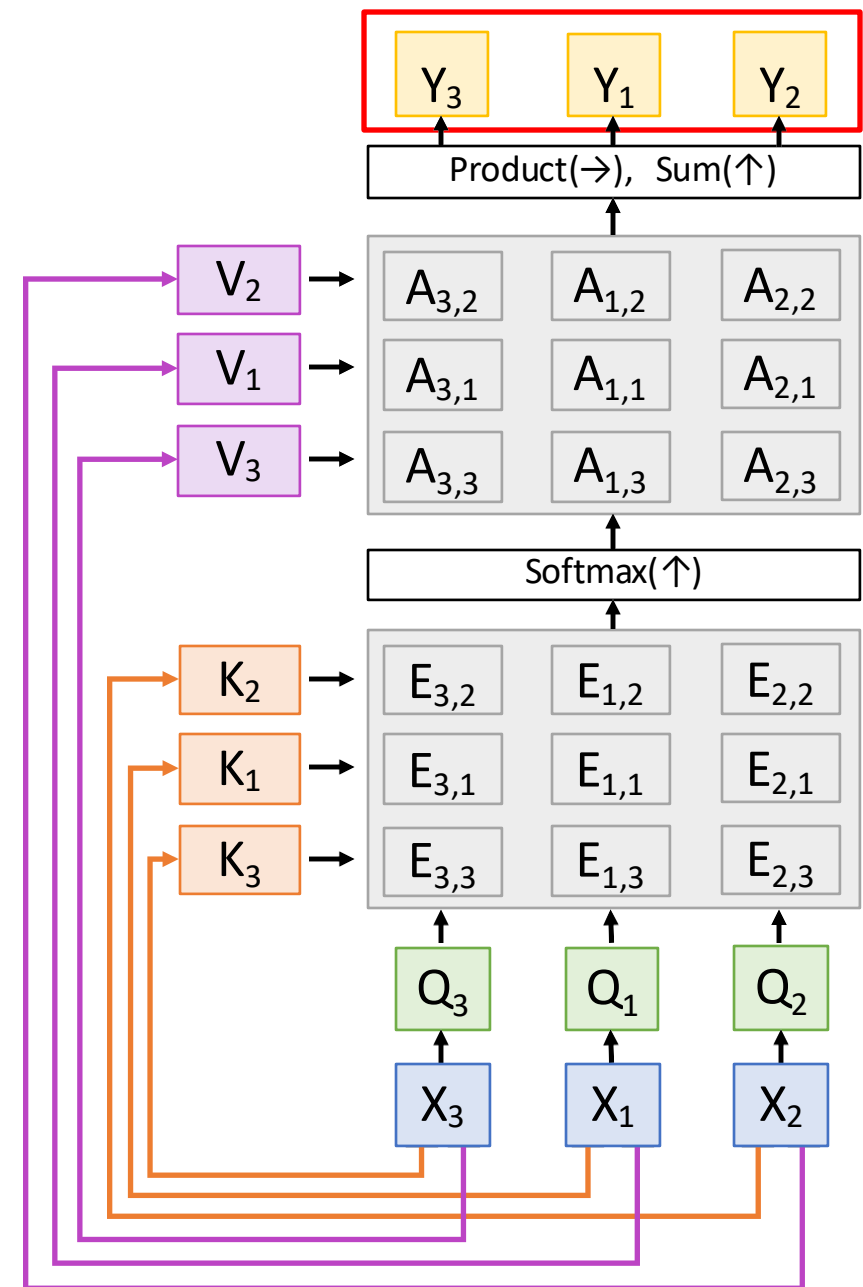
Similarities: $\mathbf{E} = \mathbf{QK}^T$ (Shape: $N_X \times N_X$) $E_{i,j} = \mathbf{Q}_i \cdot \mathbf{K}_j / \text{sqrt}(D_Q)$

Attention weights: $\mathbf{A} = \text{softmax}(\mathbf{E}, \text{dim}=1)$ (Shape: $N_X \times N_X$)

Output vectors: $\mathbf{Y} = \mathbf{AV}$ (Shape: $N_X \times D_V$) $Y_i = \sum_j A_{i,j} \mathbf{V}_j$

Consider **permuting**
the input vectors:

Outputs will be the
same, but **permuted**



Self-Attention Layer

Inputs:

Input vectors: $\mathbf{X}$ (Shape: $N_X \times D_X$)

Key matrix: $\mathbf{W}_K$ (Shape: $D_X \times D_Q$)

Value matrix: $\mathbf{W}_V$ (Shape: $D_X \times D_V$)

Query matrix: $\mathbf{W}_Q$ (Shape: $D_X \times D_Q$)

Computation:

Query vectors: $\mathbf{Q} = \mathbf{XW}_Q$

Key vectors: $\mathbf{K} = \mathbf{XW}_K$ (Shape: $N_X \times D_Q$)

Value Vectors: $\mathbf{V} = \mathbf{XW}_V$ (Shape: $N_X \times D_V$)

Similarities: $\mathbf{E} = \mathbf{QK}^T$ (Shape: $N_X \times N_X$) $E_{i,j} = \mathbf{Q}_i \cdot \mathbf{K}_j / \text{sqrt}(D_Q)$

Attention weights: $\mathbf{A} = \text{softmax}(\mathbf{E}, \text{dim}=1)$ (Shape: $N_X \times N_X$)

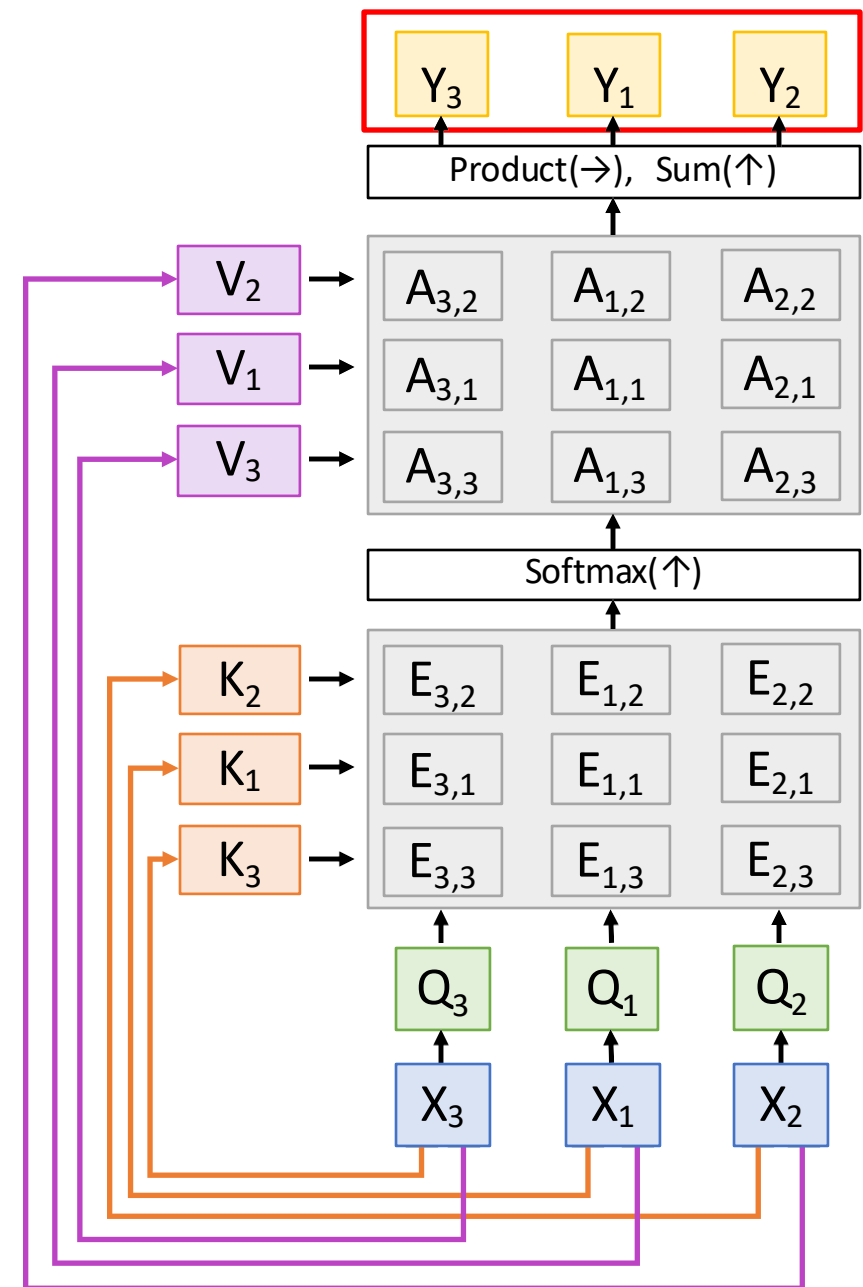
Output vectors: $\mathbf{Y} = \mathbf{AV}$ (Shape: $N_X \times D_V$) $Y_i = \sum_j A_{i,j} \mathbf{V}_j$

Consider **permuting**
the input vectors:

Outputs will be the
same, but **permuted**

Self-attention layer is
Permutation Equivariant
 $f(s(x)) = s(f(x))$

Self-Attention layer works
on **sets** of vectors



Self-Attention Layer

Inputs:

Input vectors: $\mathbf{X}$ (Shape: $N_X \times D_X$)

Key matrix: $\mathbf{W}_K$ (Shape: $D_X \times D_Q$)

Value matrix: $\mathbf{W}_V$ (Shape: $D_X \times D_V$)

Query matrix: $\mathbf{W}_Q$ (Shape: $D_X \times D_Q$)

Computation:

Query vectors: $\mathbf{Q} = \mathbf{XW}_Q$

Key vectors: $\mathbf{K} = \mathbf{XW}_K$ (Shape: $N_X \times D_Q$)

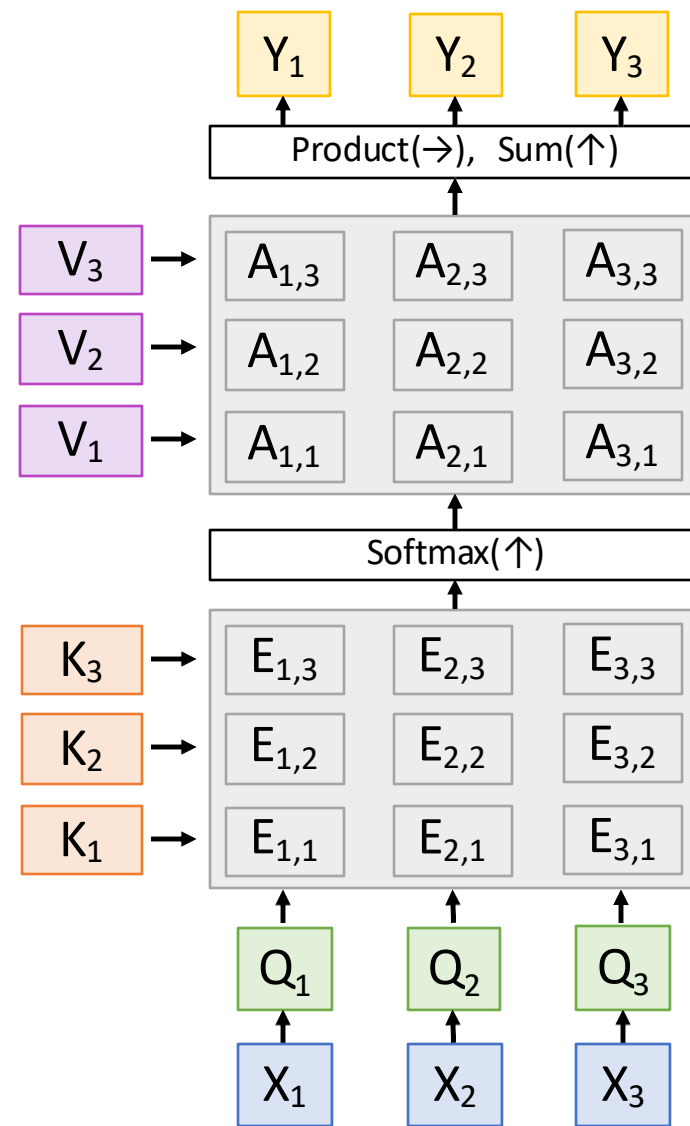
Value Vectors: $\mathbf{V} = \mathbf{XW}_V$ (Shape: $N_X \times D_V$)

Similarities: $\mathbf{E} = \mathbf{QK}^T$ (Shape: $N_X \times N_X$) $E_{i,j} = \mathbf{Q}_i \cdot \mathbf{K}_j / \text{sqrt}(D_Q)$

Attention weights: $\mathbf{A} = \text{softmax}(\mathbf{E}, \text{dim}=1)$ (Shape: $N_X \times N_X$)

Output vectors: $\mathbf{Y} = \mathbf{AV}$ (Shape: $N_X \times D_V$) $Y_i = \sum_j A_{i,j} \mathbf{V}_j$

Self attention doesn't
"know" the order of the
vectors it is processing!



Self-Attention Layer

Inputs:

Input vectors: $\mathbf{X}$ (Shape: $N_X \times D_X$)

Key matrix: $\mathbf{W}_K$ (Shape: $D_X \times D_Q$)

Value matrix: $\mathbf{W}_V$ (Shape: $D_X \times D_V$)

Query matrix: $\mathbf{W}_Q$ (Shape: $D_X \times D_Q$)

Computation:

Query vectors: $\mathbf{Q} = \mathbf{XW}_Q$

Key vectors: $\mathbf{K} = \mathbf{XW}_K$ (Shape: $N_X \times D_Q$)

Value Vectors: $\mathbf{V} = \mathbf{XW}_V$ (Shape: $N_X \times D_V$)

Similarities: $\mathbf{E} = \mathbf{QK}^T$ (Shape: $N_X \times N_X$) $E_{i,j} = \mathbf{Q}_i \cdot \mathbf{K}_j / \text{sqrt}(D_Q)$

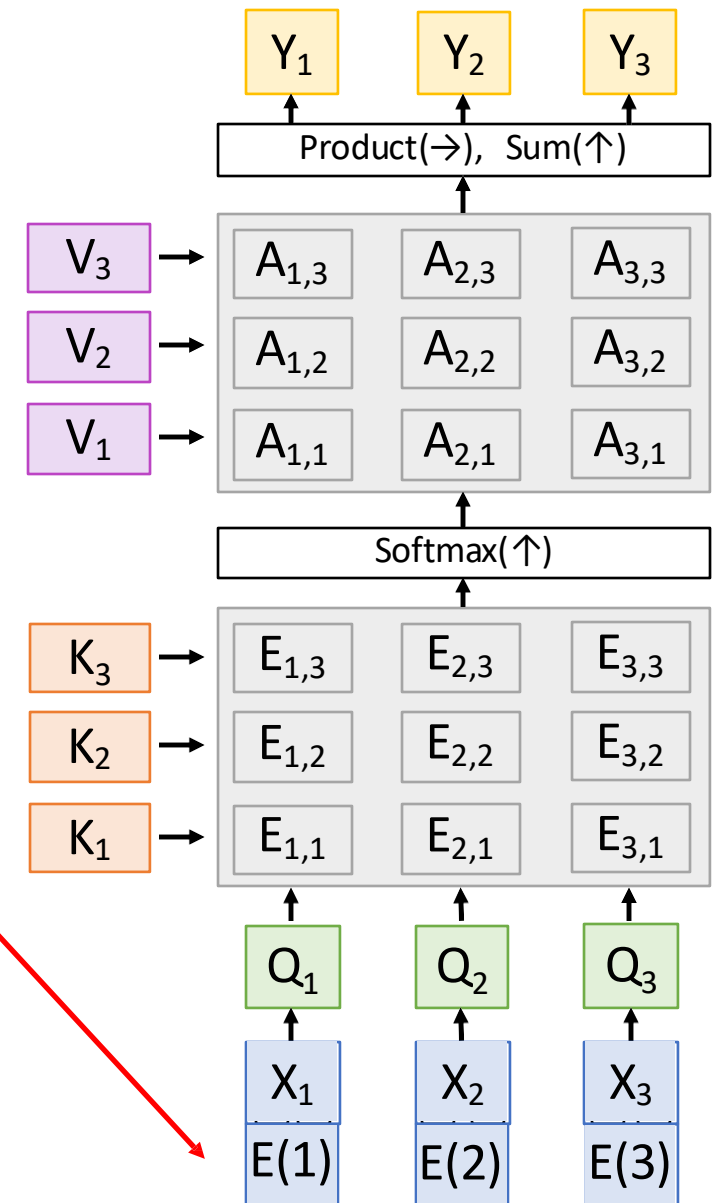
Attention weights: $\mathbf{A} = \text{softmax}(\mathbf{E}, \text{dim}=1)$ (Shape: $N_X \times N_X$)

Output vectors: $\mathbf{Y} = \mathbf{AV}$ (Shape: $N_X \times D_V$) $Y_i = \sum_j A_{i,j} \mathbf{V}_j$

Self attention doesn't
"know" the order of the
vectors it is processing!

In order to make
processing position-
aware, concatenate input
with **positional encoding**

E can be learned lookup
table, or fixed function



Multihead Self-Attention Layer

Use H independent
“Attention Heads” in parallel

Inputs:

Input vectors: $\mathbf{X}$ (Shape: $N_x \times D_x$)

Key matrix: $\mathbf{W}_K$ (Shape: $D_x \times D_K$)

Value matrix: $\mathbf{W}_V$ (Shape: $D_x \times D_V$)

Query matrix: $\mathbf{W}_Q$ (Shape: $D_x \times D_Q$)

Computation:

Query vectors: $\mathbf{Q} = \mathbf{XW}_Q$

Key vectors: $\mathbf{K} = \mathbf{XW}_K$ (Shape: $N_x \times D_K$)

Value Vectors: $\mathbf{V} = \mathbf{XW}_V$ (Shape: $N_x \times D_V$)

Similarities: $\mathbf{E} = \mathbf{QK}^T$ (Shape: $N_x \times N_x$) $E_{i,j} = \mathbf{Q}_i \cdot \mathbf{K}_j / \text{sqrt}(D_Q)$

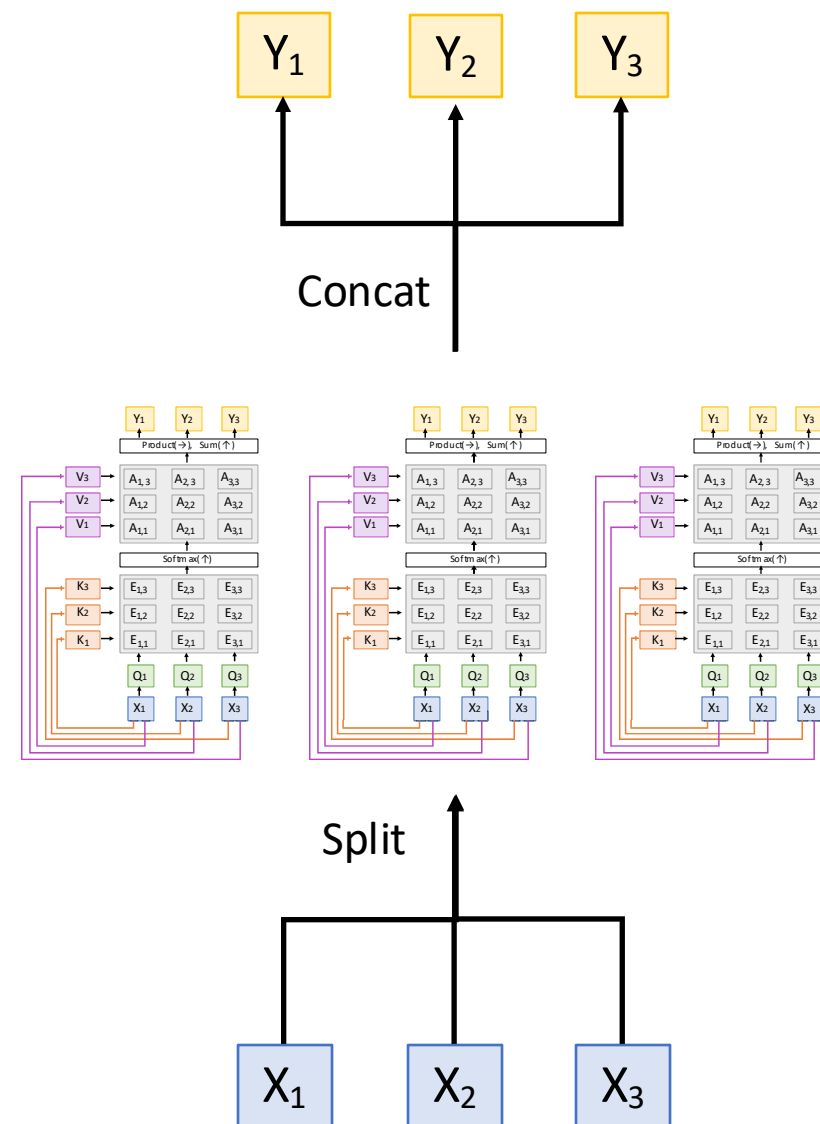
Attention weights: $\mathbf{A} = \text{softmax}(\mathbf{E}, \text{dim}=1)$ (Shape: $N_x \times N_x$)

Output vectors: $\mathbf{Y} = \mathbf{AV}$ (Shape: $N_x \times D_V$) $Y_i = \sum_j A_{i,j} \mathbf{V}_j$

Hyperparameters:

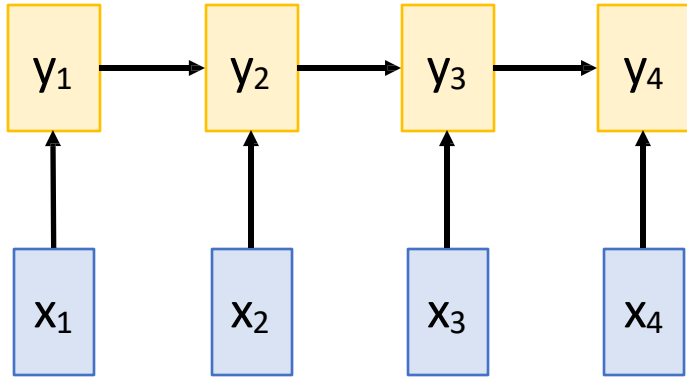
Query dimension D_Q

Number of heads H



Summary: Three Ways of Processing Sequences

Recurrent Neural Network



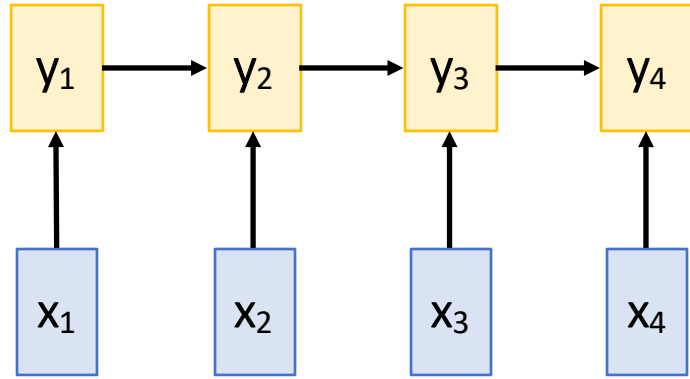
Works on **Ordered Sequences**

(+) **Good at long sequences:** After one RNN layer, h_T "sees" the whole sequence

(-) **Not parallelizable:** need to compute hidden states sequentially

Summary: Three Ways of Processing Sequences

Recurrent Neural Network

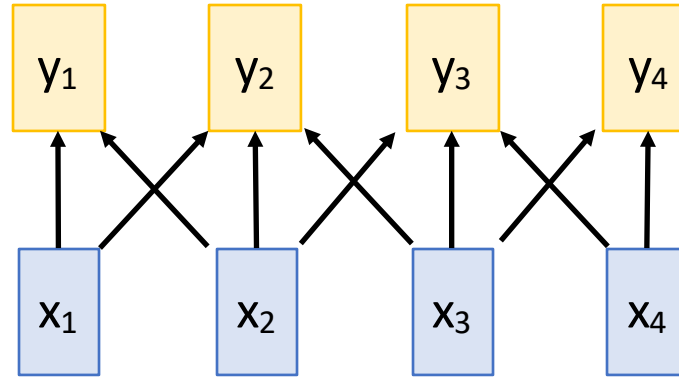


Works on **Ordered Sequences**

(+) **Good at long sequences:** After one RNN layer, h_T "sees" the whole sequence

(-) **Not parallelizable:** need to compute hidden states sequentially

1D Convolution



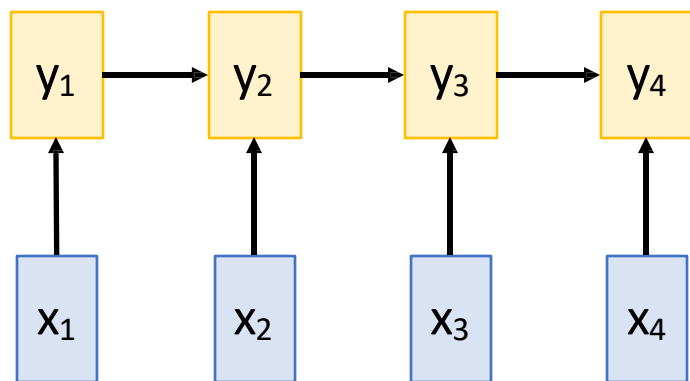
Works on **Multidimensional Grids**

(-) **Bad at long sequences:** Need to stack many conv layers for outputs to "see" the whole sequence

(+) **Highly parallel:** Each output can be computed in parallel

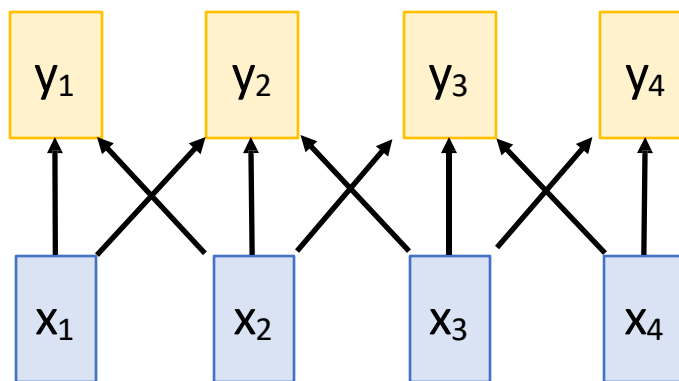
Summary: Three Ways of Processing Sequences

Recurrent Neural Network



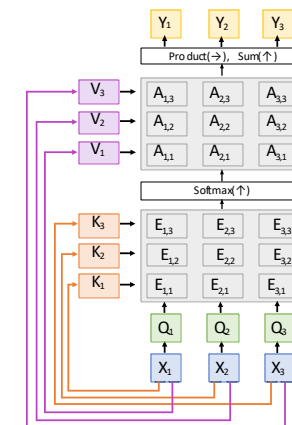
Works on **Ordered Sequences**
(+) **Good at long sequences:** After one RNN layer, h_T "sees" the whole sequence
(-) **Not parallelizable:** need to compute hidden states sequentially

1D Convolution



Works on **Multidimensional Grids**
(-) **Bad at long sequences:** Need to stack many conv layers for outputs to "see" the whole sequence
(+) **Highly parallel:** Each output can be computed in parallel

Self-Attention



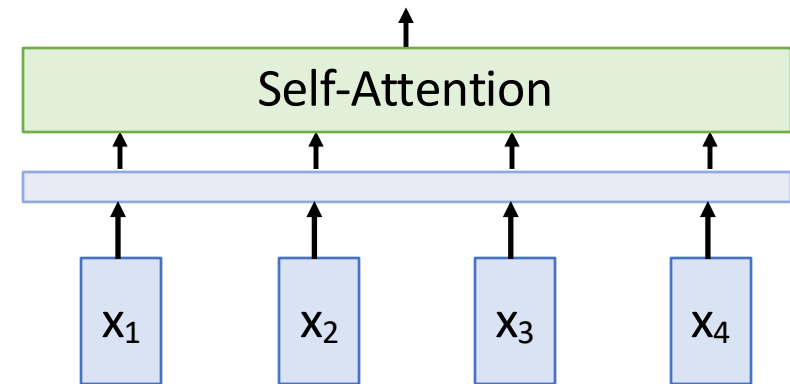
Works on **Sets of Vectors**
(-) **Good at long sequences:** after one self-attention layer, each output "sees" all inputs!
(+) **Highly parallel:** Each output can be computed in parallel
(-) **Very memory intensive**

The Transformer



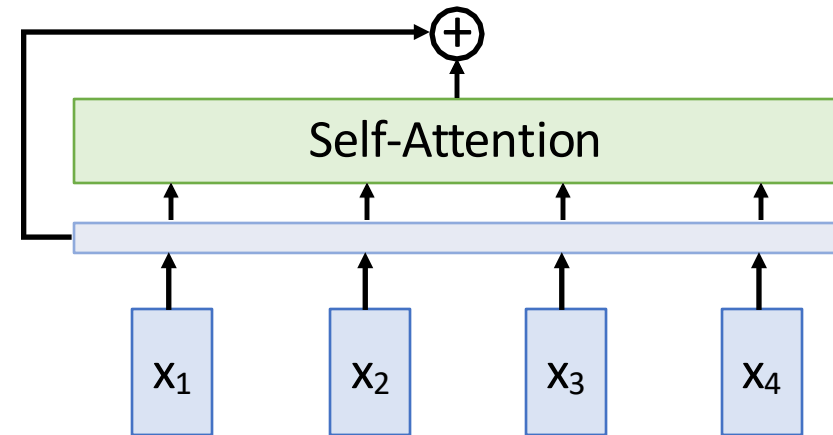
The Transformer

All vectors interact
with each other



The Transformer

Residual connection
All vectors interact
with each other



The Transformer

Recall **Layer Normalization**:

Given $h_1, \dots, h_N$ (Shape: D)

scale: γ (Shape: D)

shift: β (Shape: D)

$\mu_i = (1/D) \sum_j h_{i,j}$ (scalar)

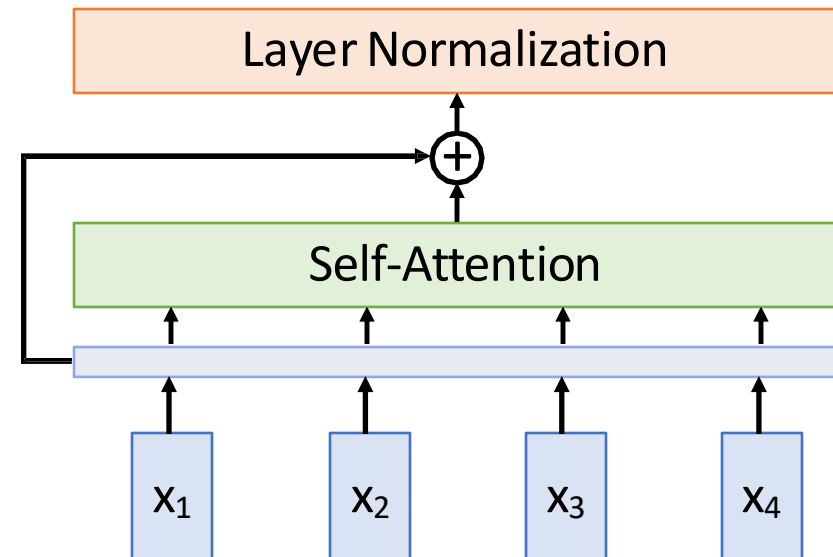
$\sigma_i = (\sum_j (h_{i,j} - \mu_i)^2)^{1/2}$ (scalar)

$z_i = (h_i - \mu_i) / \sigma_i$

$y_i = \gamma * z_i + \beta$

Ba et al, 2016

Residual connection
All vectors interact
with each other



The Transformer

Recall **Layer Normalization**:

Given $h_1, \dots, h_N$ (Shape: D)

scale: γ (Shape: D)

shift: β (Shape: D)

$\mu_i = (1/D) \sum_j h_{i,j}$ (scalar)

$\sigma_i = (\sum_j (h_{i,j} - \mu_i)^2)^{1/2}$ (scalar)

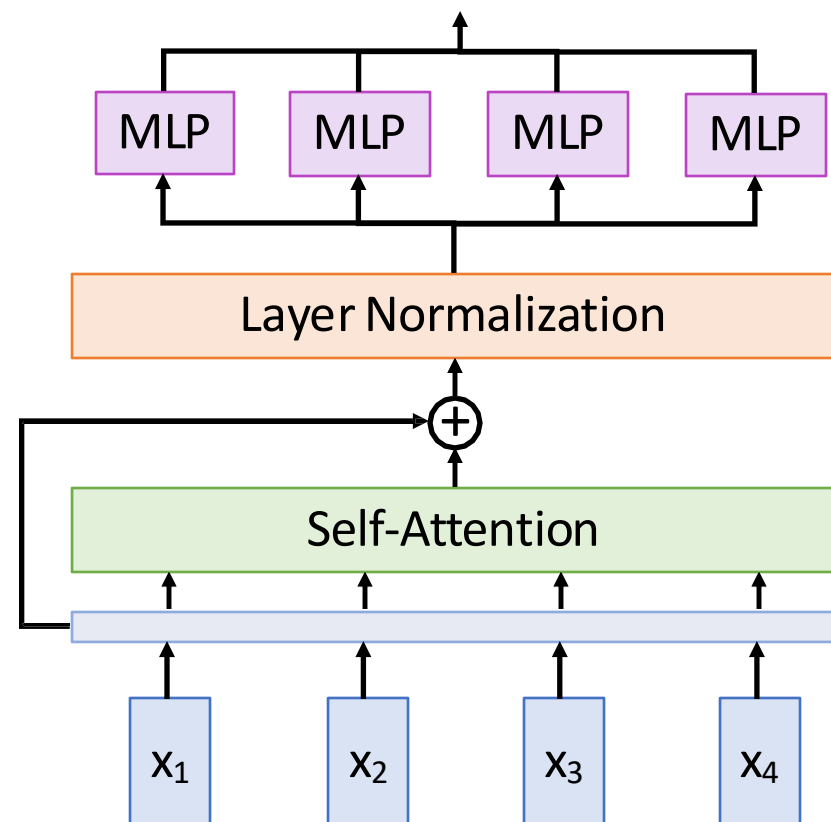
$z_i = (h_i - \mu_i) / \sigma_i$

$y_i = \gamma * z_i + \beta$

Ba et al, 2016

MLP independently
on each vector

Residual connection
All vectors interact
with each other



The Transformer

Recall **Layer Normalization**:

Given $h_1, \dots, h_N$ (Shape: D)

scale: γ (Shape: D)

shift: β (Shape: D)

$\mu_i = (1/D) \sum_j h_{i,j}$ (scalar)

$\sigma_i = (\sum_j (h_{i,j} - \mu_i)^2)^{1/2}$ (scalar)

$z_i = (h_i - \mu_i) / \sigma_i$

$y_i = \gamma * z_i + \beta$

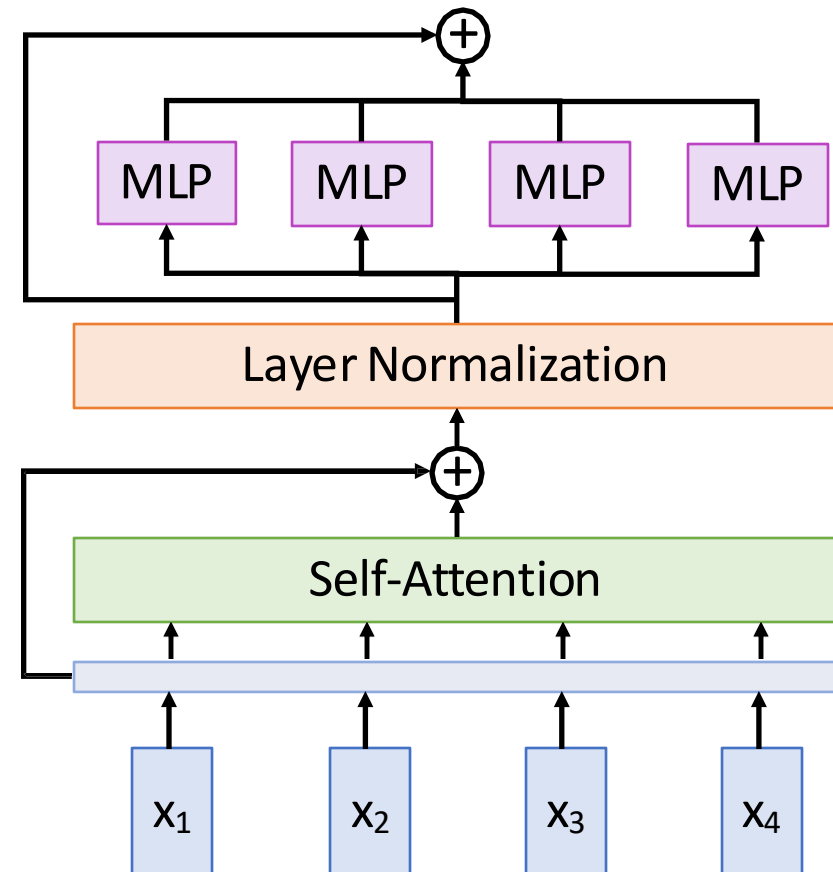
Ba et al, 2016

Residual connection

MLP independently
on each vector

Residual connection

All vectors interact
with each other



The Transformer

Recall **Layer Normalization**:

Given $h_1, \dots, h_N$ (Shape: D)

scale: γ (Shape: D)

shift: β (Shape: D)

$\mu_i = (1/D) \sum_j h_{i,j}$ (scalar)

$\sigma_i = (\sum_j (h_{i,j} - \mu_i)^2)^{1/2}$ (scalar)

$z_i = (h_i - \mu_i) / \sigma_i$

$y_i = \gamma * z_i + \beta$

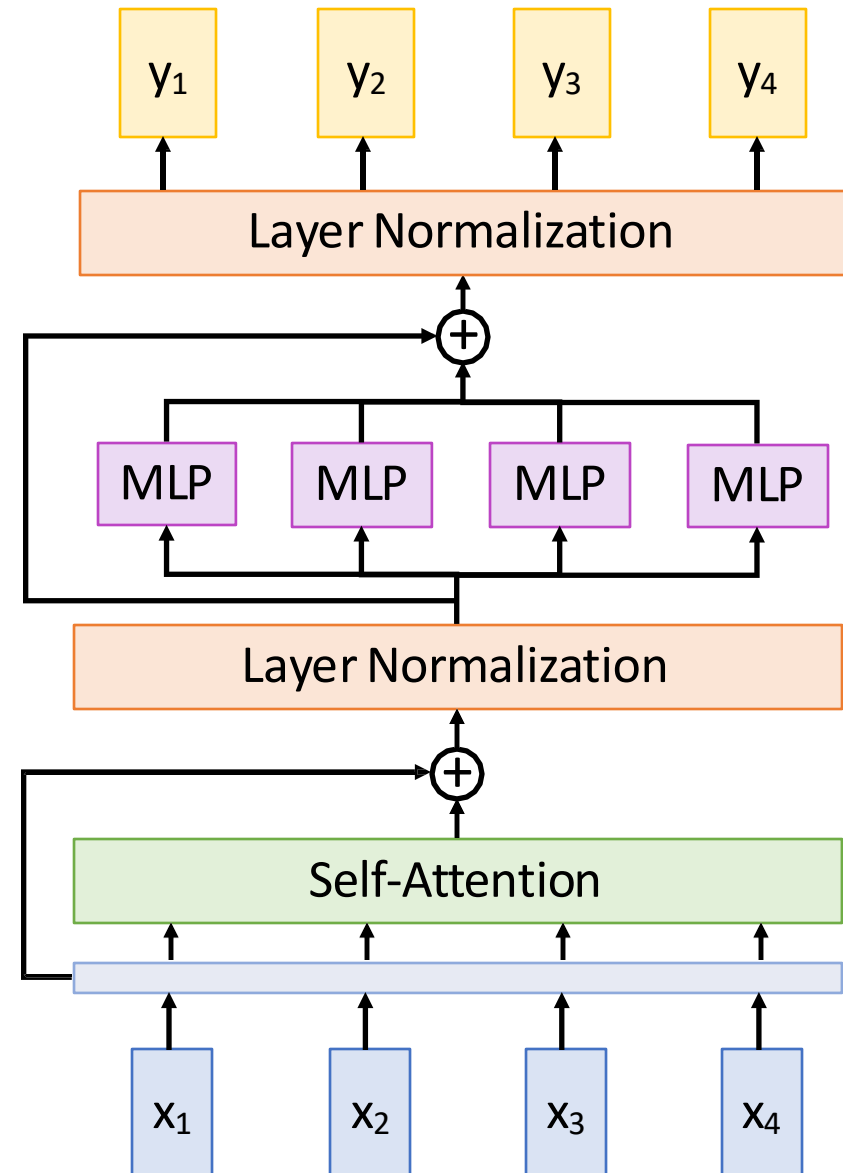
Ba et al, 2016

Residual connection

MLP independently
on each vector

Residual connection

All vectors interact
with each other



The Transformer

Transformer Block:

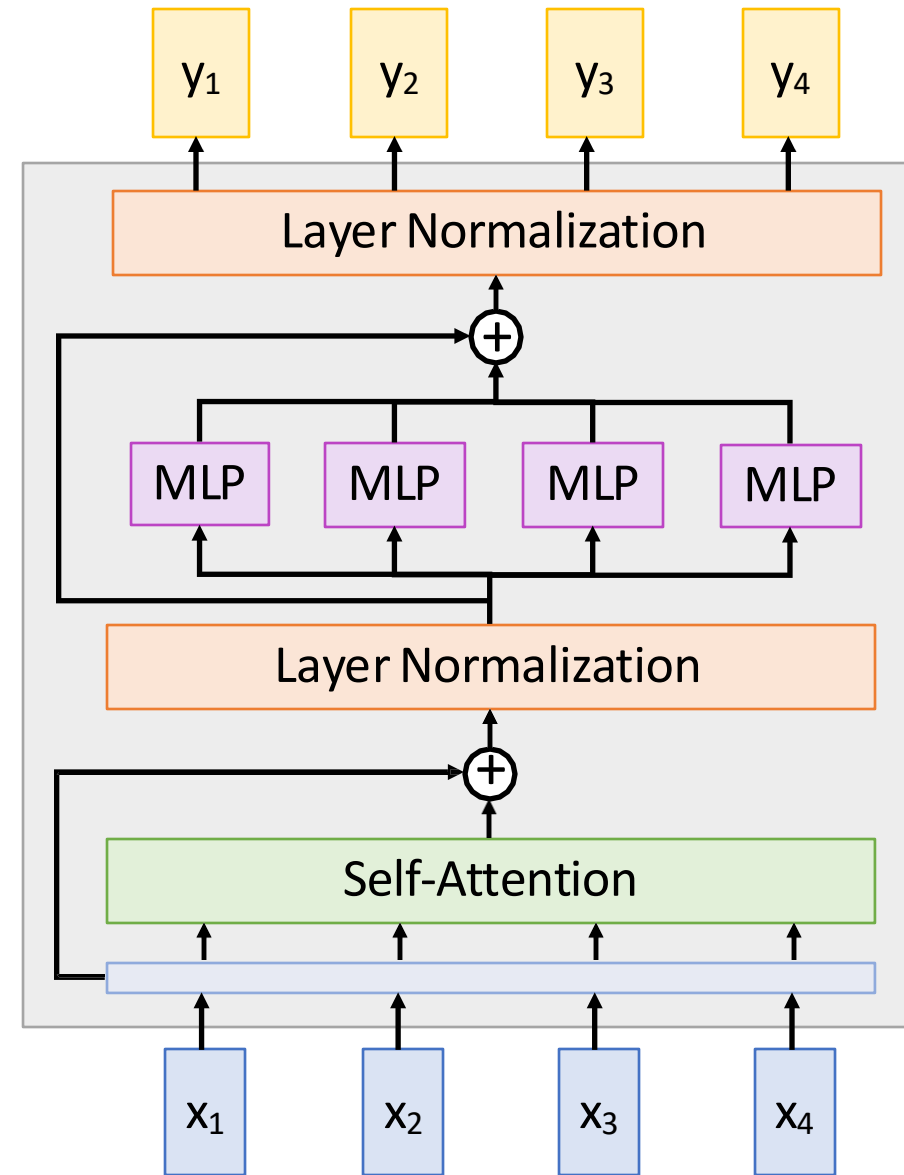
Input: Set of vectors x

Output: Set of vectors y

Self-attention is the only interaction between vectors!

Layer norm and MLP work independently per vector

Highly scalable, highly parallelizable



The Transformer

Transformer Block:

Input: Set of vectors x

Output: Set of vectors y

Self-attention is the only
interaction between vectors!

Layer norm and MLP work
independently per vector

Highly scalable, highly
parallelizable

A **Transformer** is a sequence
of transformer blocks

Vaswani et al:
12 blocks, $D_Q=512$, 6 heads

